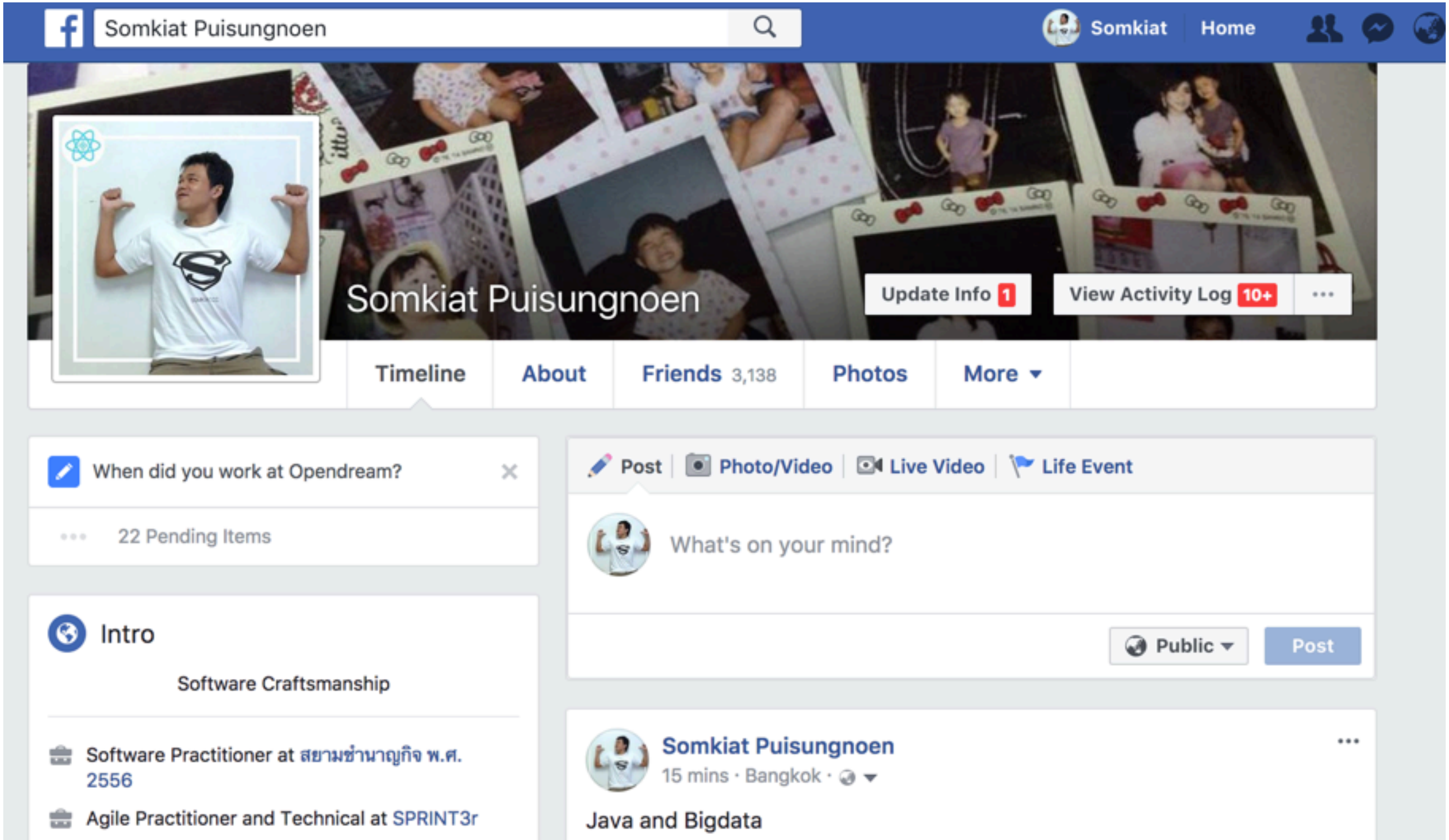
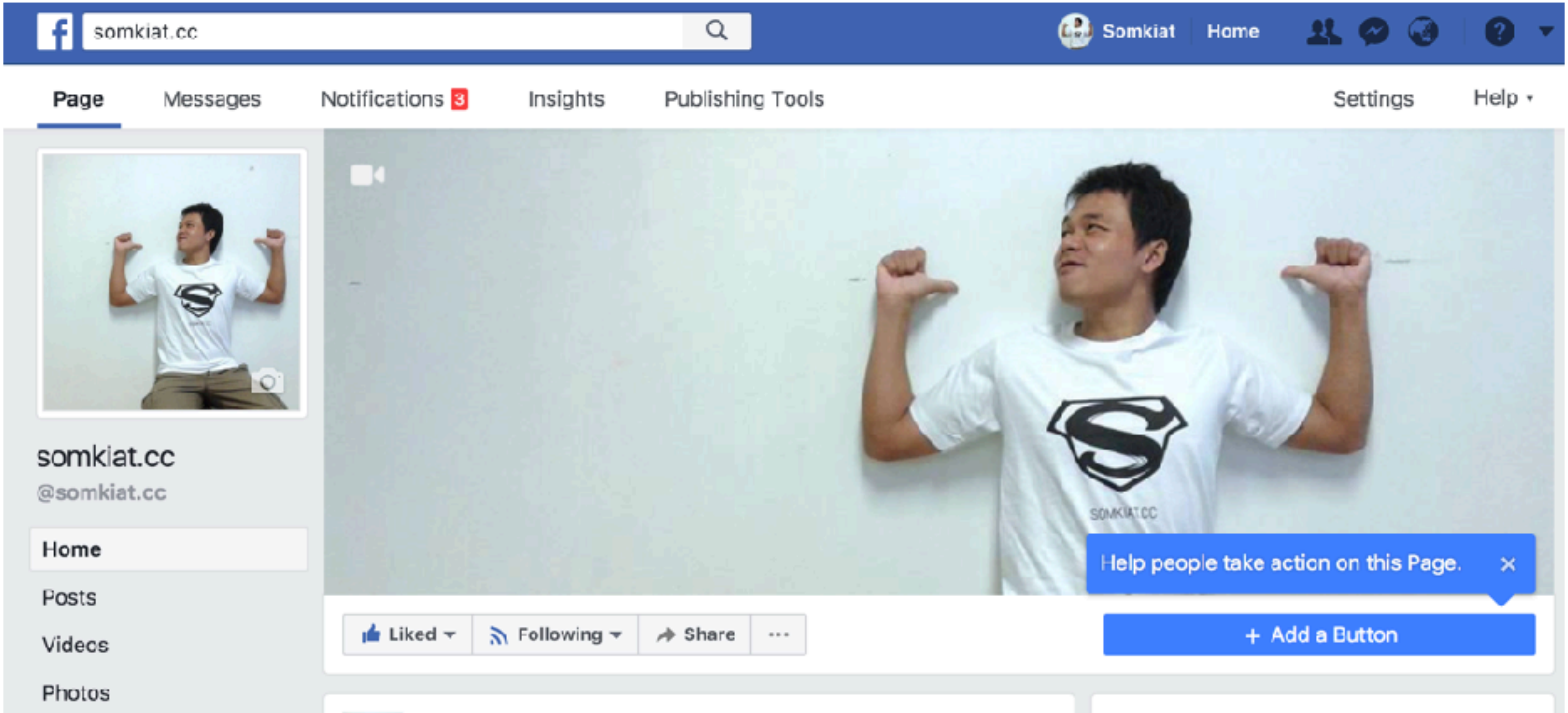
# Fundamental
# Java and Spring Boot


Somkiat Puisungnoen
Somkiat
Home
Somkiat Puisungnoen
Update Info 1
View Activity Log 10+
Timeline
About
Friends 3,138
Photos
More
When did you work at Opendream?
22 Pending Items
Intro
Software Craftsmanship
Software Practitioner at สยามชำนาญกิจ พ.ศ. 2556
Agile Practitioner and Technical at SPRINT3r
Post
Photo/Video
Live Video
Life Event
What's on your mind?
Public
Post
Somkiat Puisungnoen
15 mins · Bangkok ·
Java and Bigdata


somkiat.cc
Somkiat
Home
Page
Messages
Notifications 3
Insights
Publishing Tools
Settings
Help
somkiat.cc
@somkiat.cc
Home
Posts
Videos
Photos
Help people take action on this Page.
Liked
Following
Share
+ Add a Button

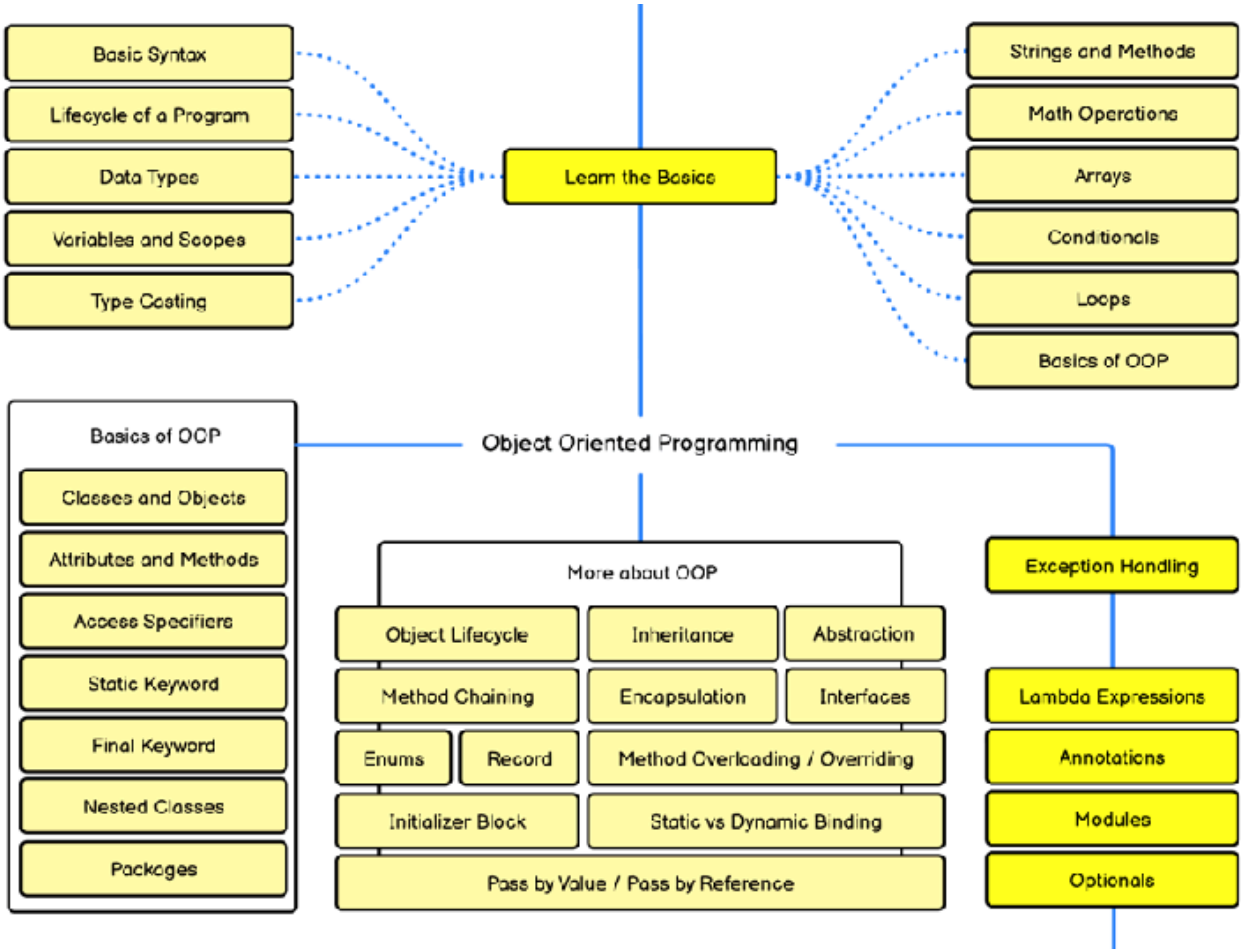
# https://github.com/up1/course-java-spring-2026

# Java Fundamental

# Leaning Java

Modern Java (generic, optional and lambda)
Error handling
Object-Oriented Concepts with Java
Data structure with Java collection framework
Modern Java (generic, optional and lambda)
Testing with JUnit and Mockito

# Java Developer Roadmap




https://roadmap.sh/java

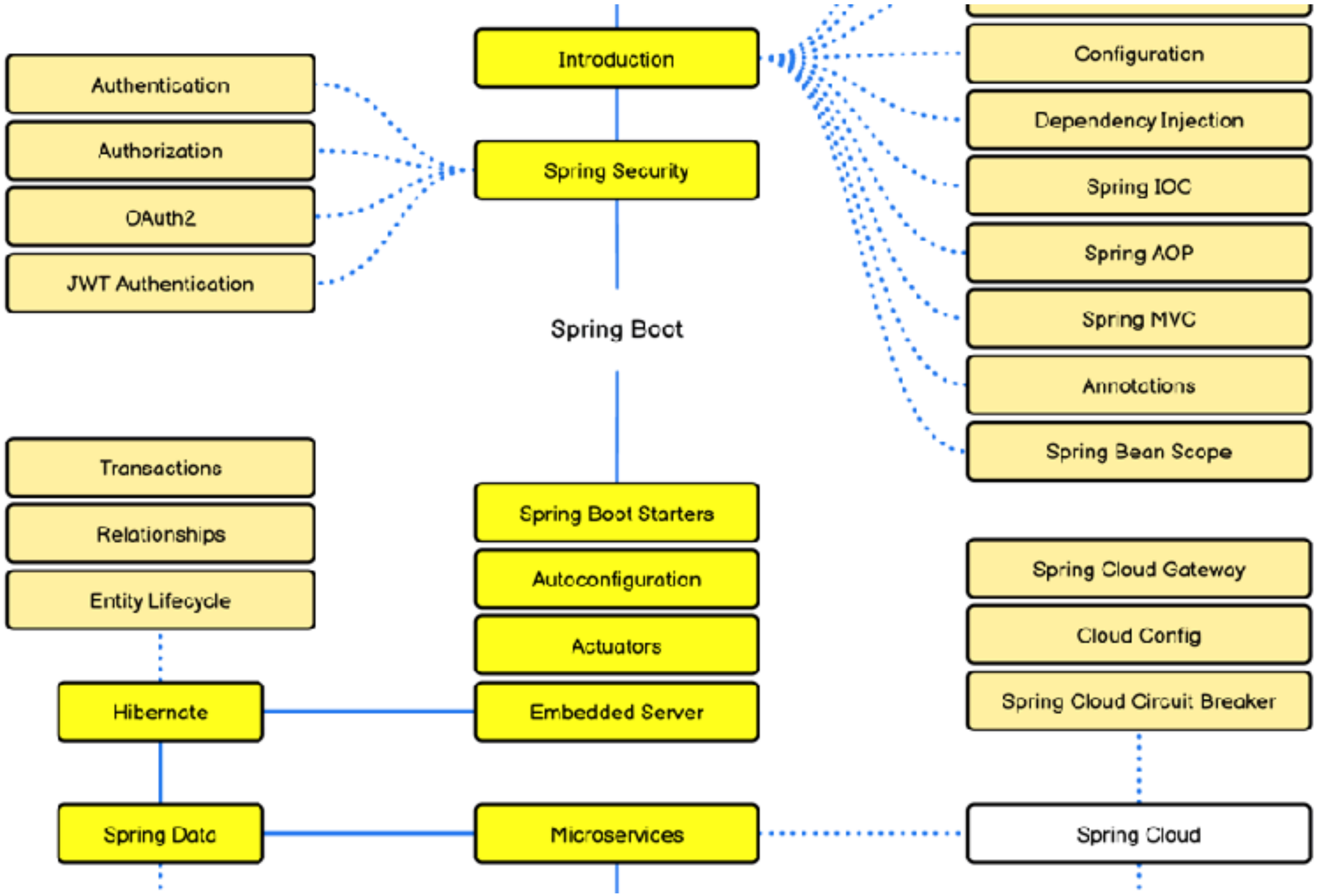
# Basic of REST

REST (REpresentational State Transfer)
6 REST constraints
REST vs RESTful API
Design principles

# Spring Boot

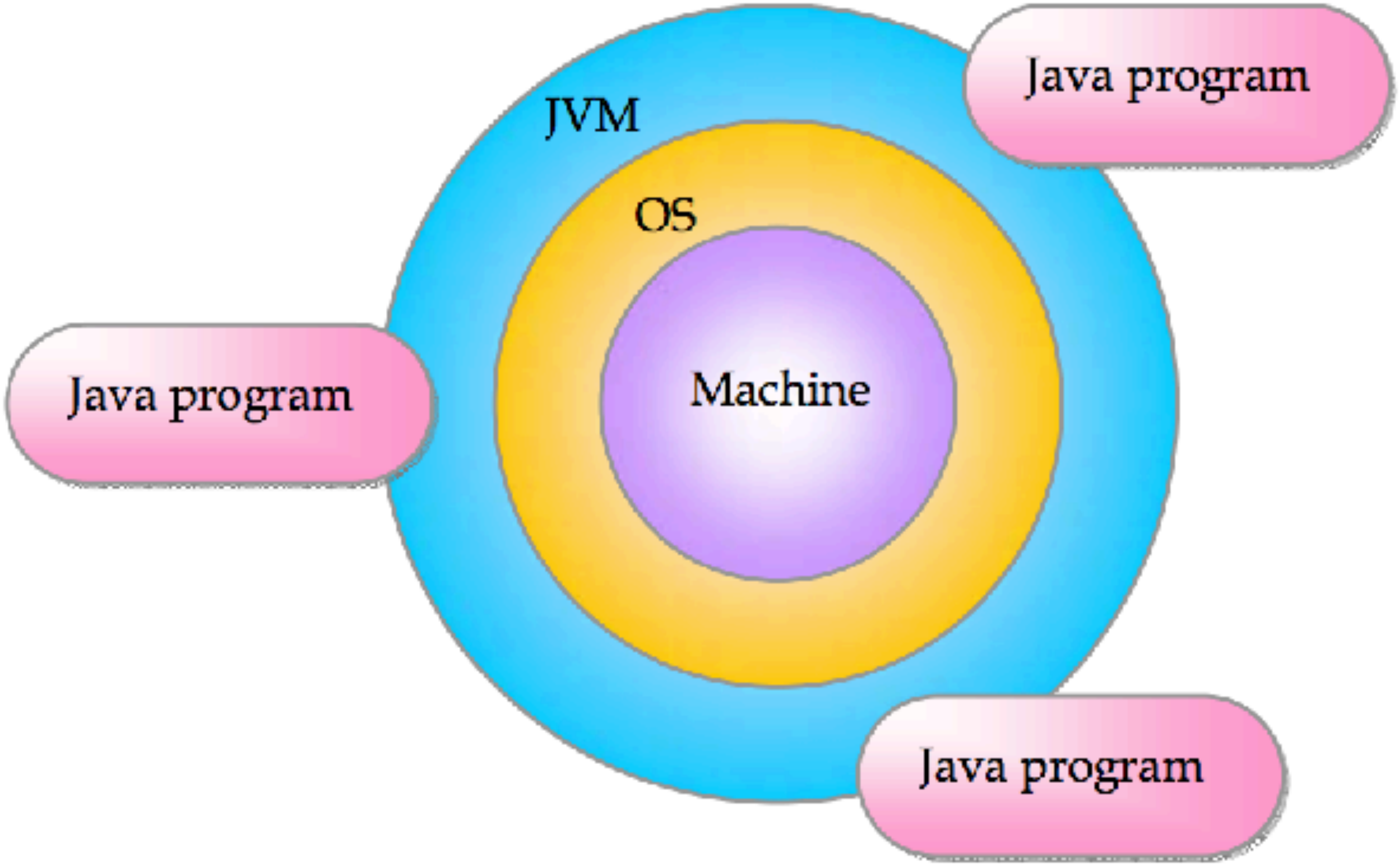
Goals of Spring Boot
Better project structure of Spring Boot
Develop RESTful APIs
Error handling
Working with database with Spring Data
Manage connections and transactions
Testing
Observable service (log, trace, metric)

# Spring Boot Roadmap




https://roadmap.sh/spring-boot

# Write Once Run Anywhere ?

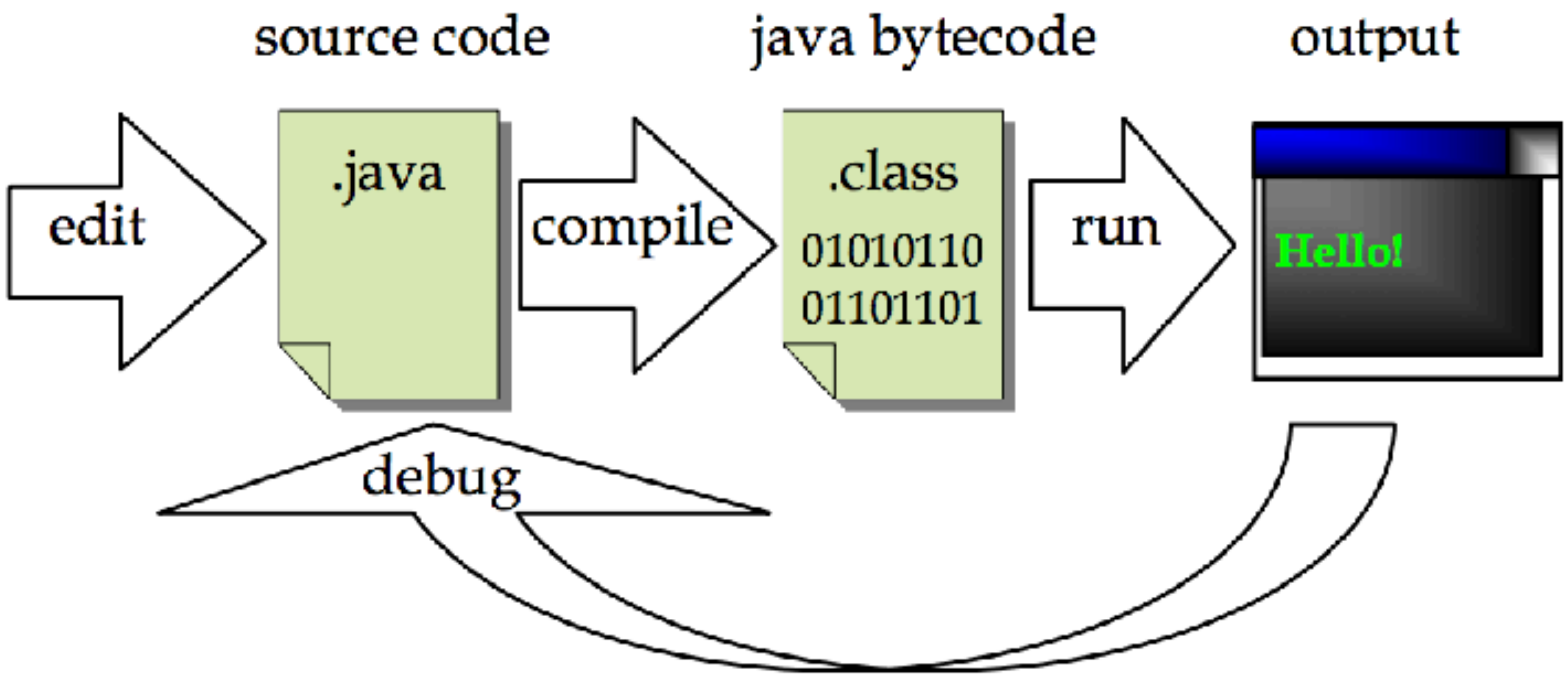


# Structure of Java program

Create file HelloWorld.java

```
public class HelloWorld {

    // Show message in a console
    public static void main(String[] args) {
        System.out.println("Hello World");

    }

}
```

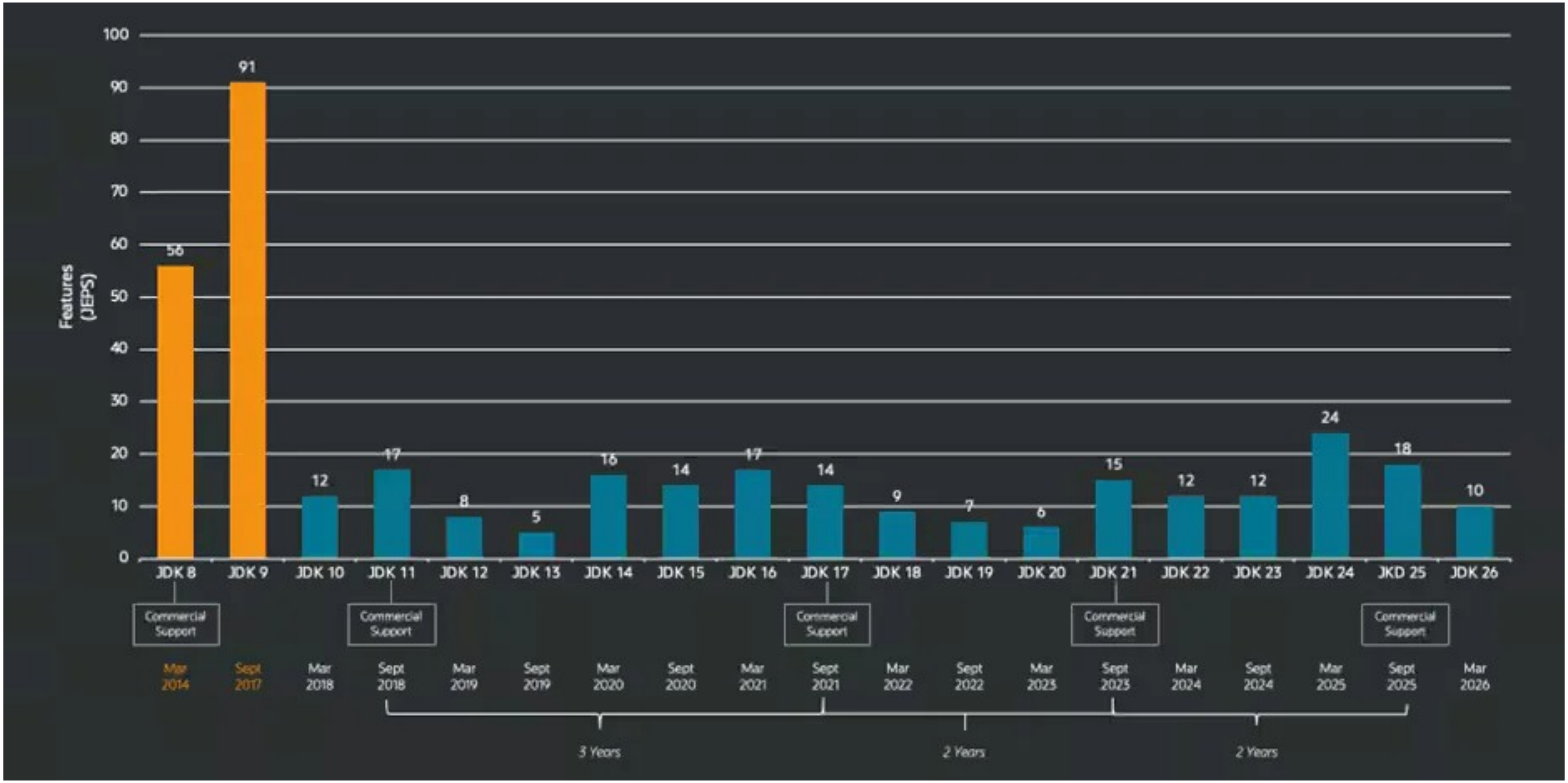
# Java Development Workflow



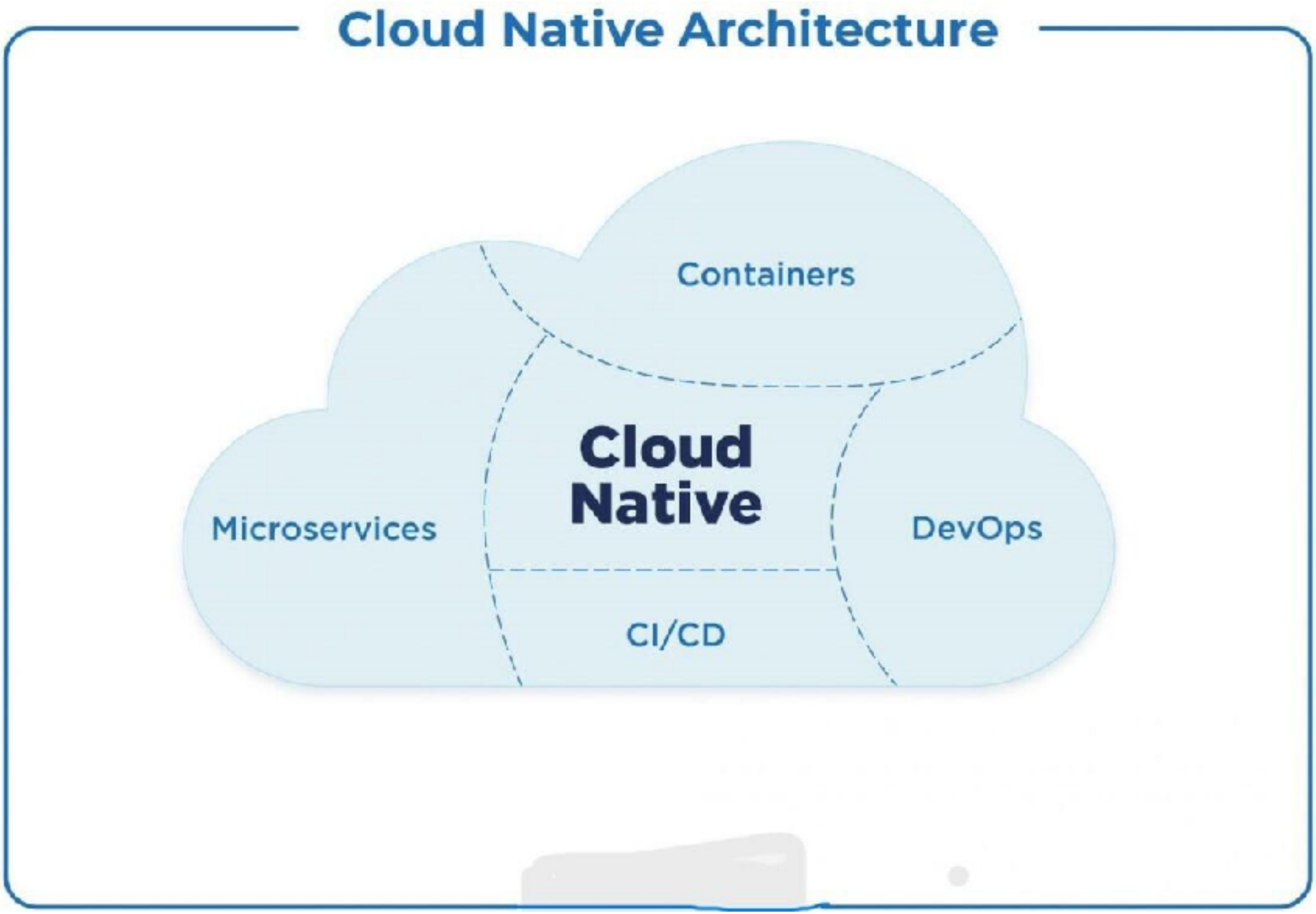

JDK

JRE

# History of Java from 8 to 26




https://www.infoq.com/news/2026/03/java26-released/


Cloud Native Architecture
Containers
Cloud Native
Microservices
DevOps
CI/CD

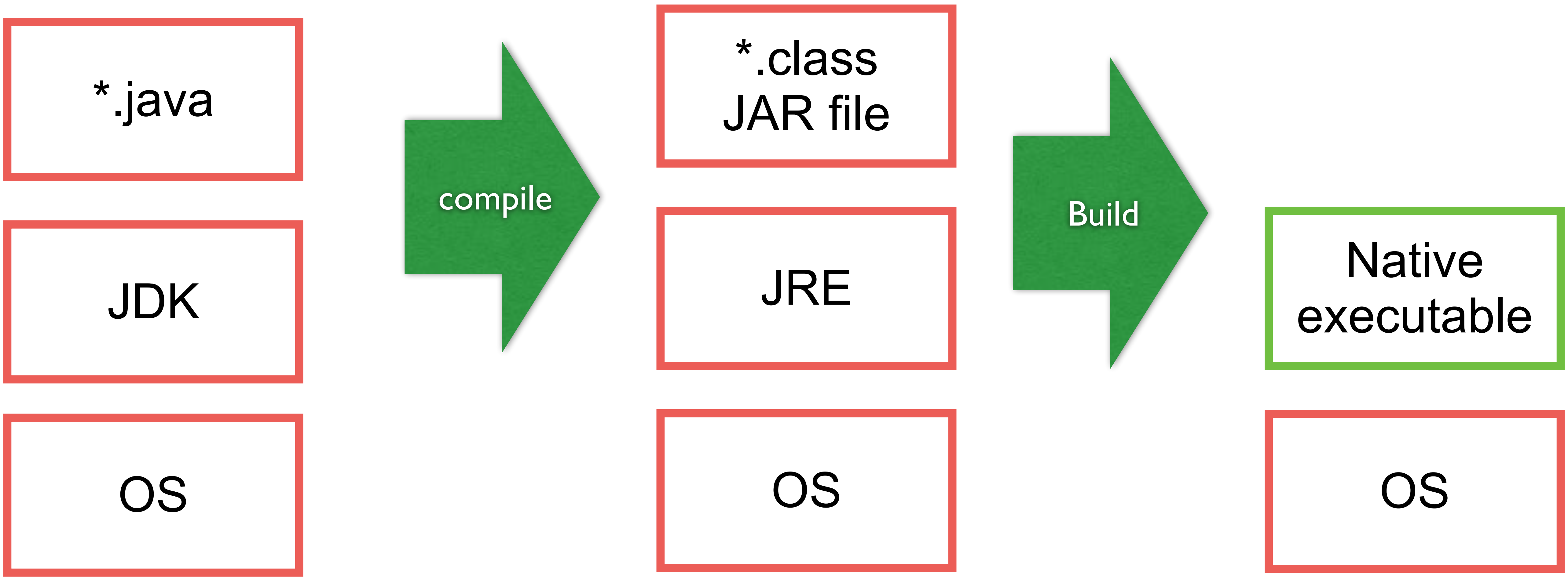
# Java in Cloud Native App

Improve performance and resource usages

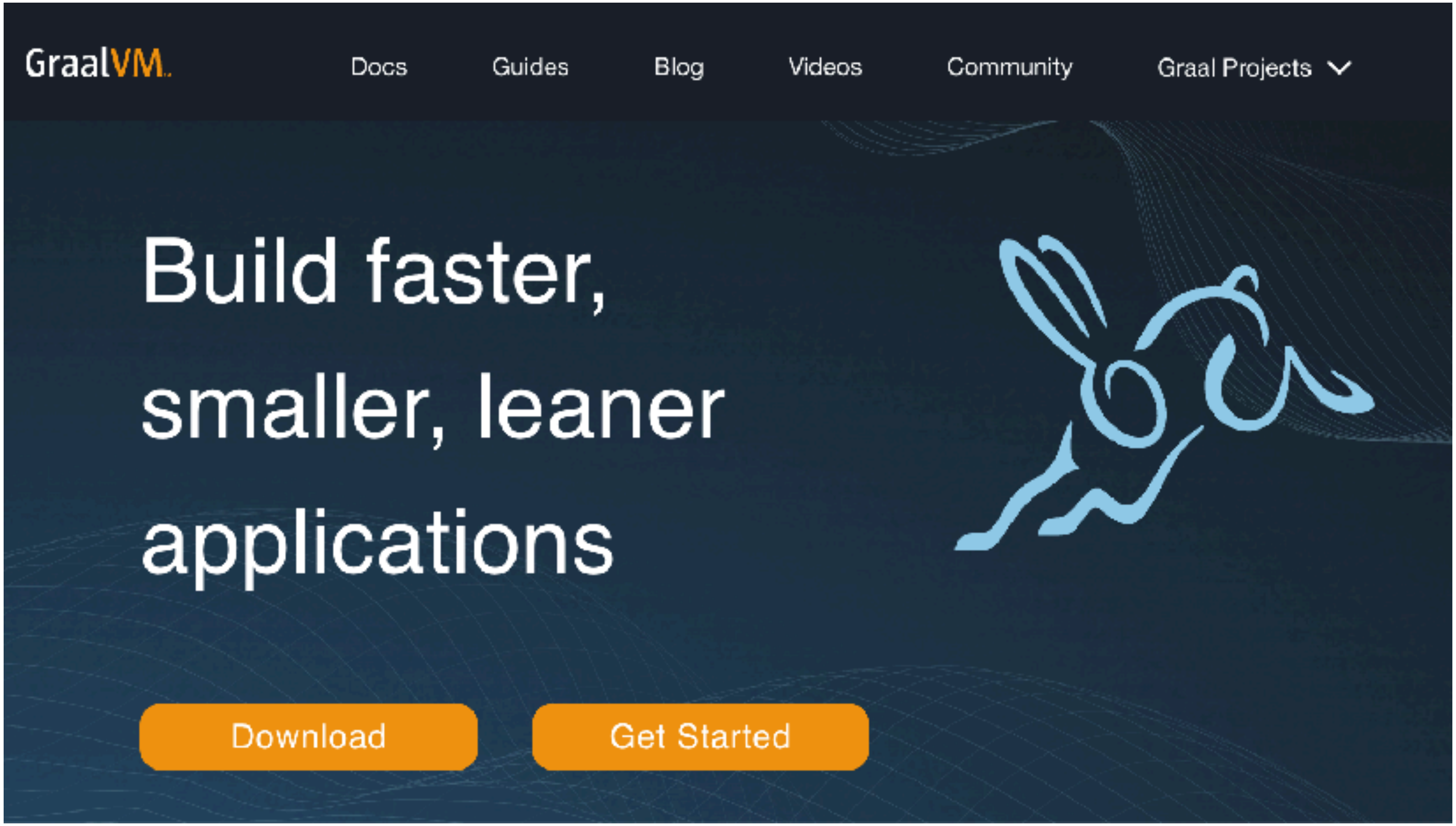
QUARKUS

GraalVM™

# Java in Cloud Native



# Working with GraalVM




https://www.graalvm.org/

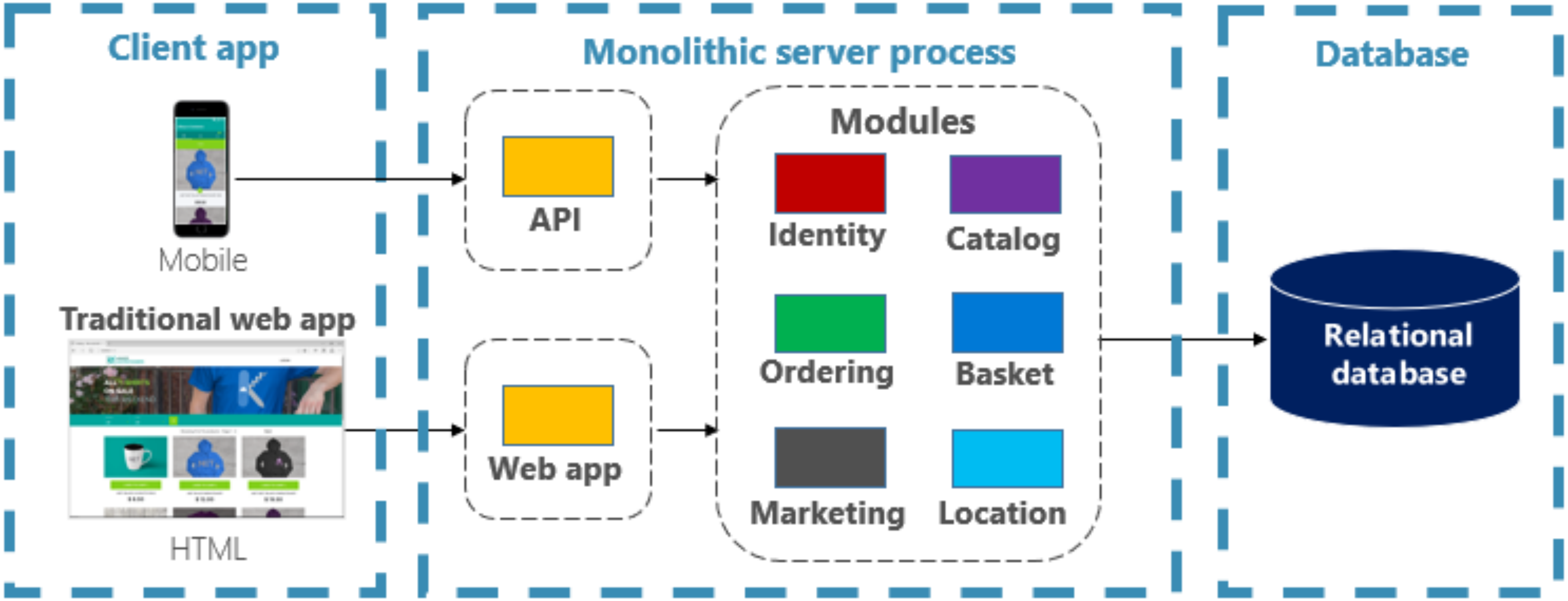
# Native Image of GraalVM

Use less resources (compare with JRE)
Start in milliseconds
Deliver peak performance immediately (no warmup)
Lightweight package
Reduce attack surface

https://www.graalvm.org/latest/reference-manual/native-image/

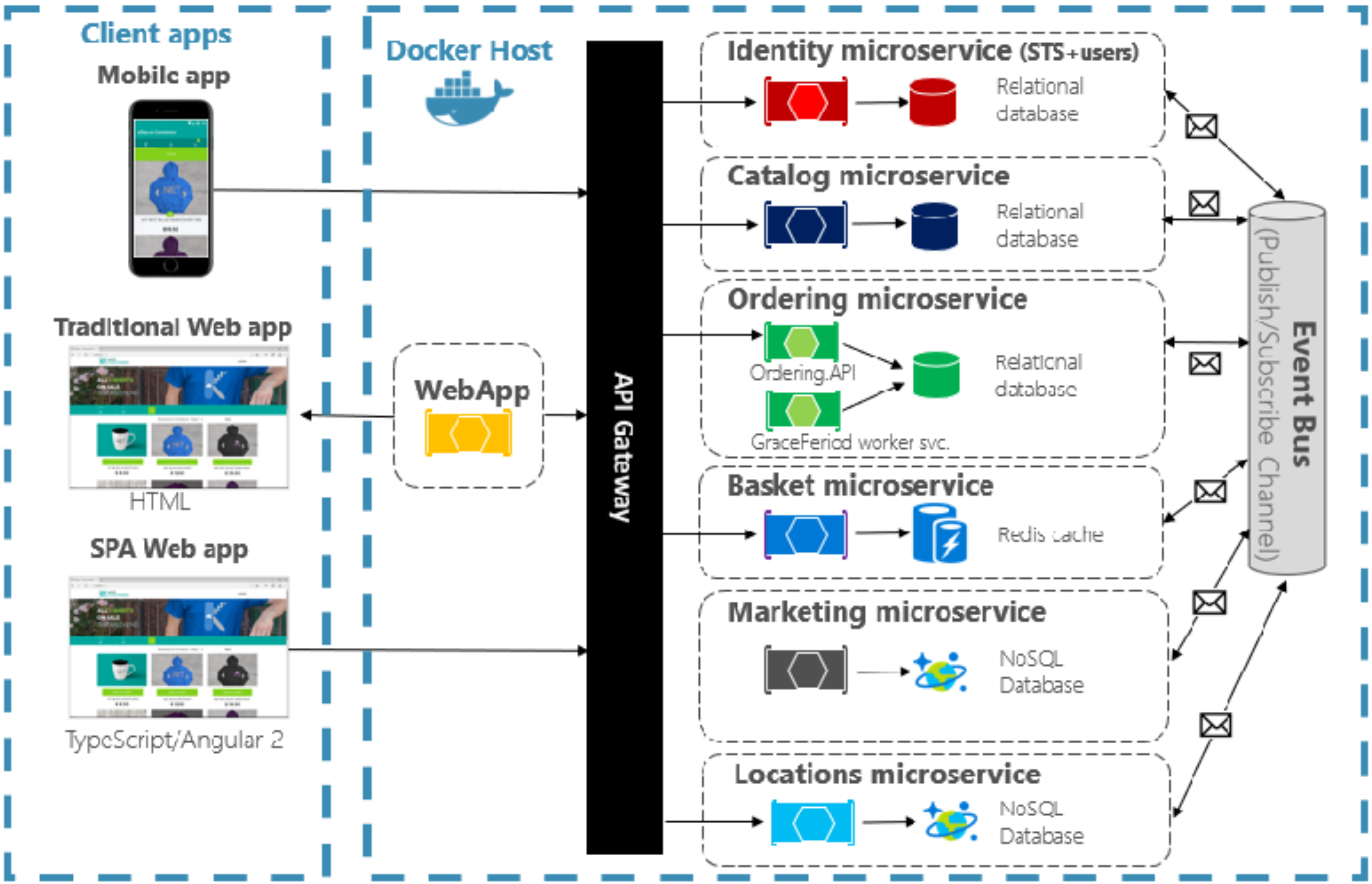
# Cloud Native Architecture Design

# Monolithic




https://learn.microsoft.com/th-th/dotnet/architecture/cloud-native/introduction

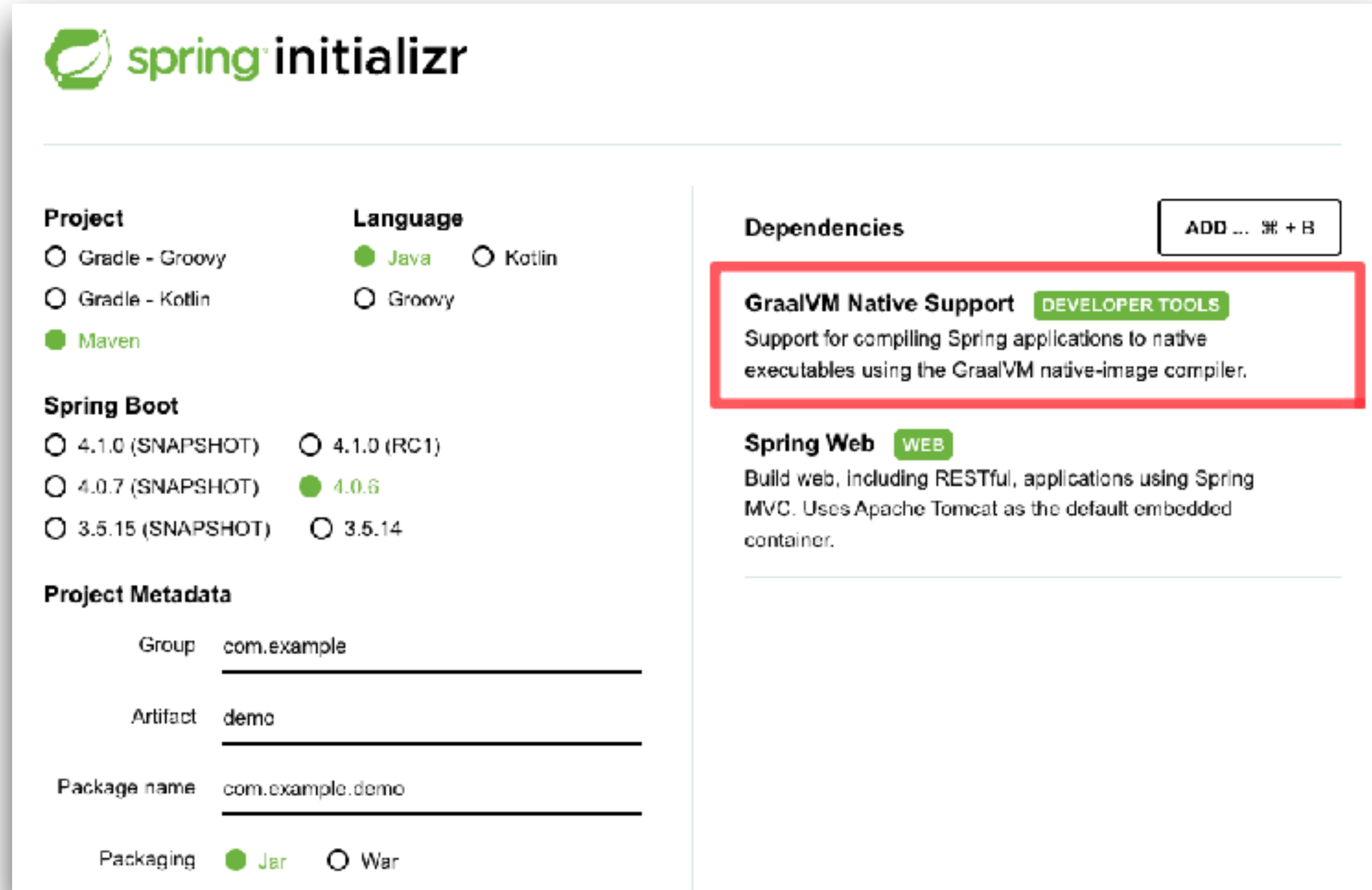
# Microservices




https://learn.microsoft.com/th-th/dotnet/architecture/cloud-native/introduction

# GraalVM in Spring Boot

# GraalVM in Spring Boot




https://start.spring.io/

# Object-Oriented Programming

# Structure of Java program

**Class**

Members or properties

Behavioral or methods

**Main method to start program**

## **Thinking** before create class

## **Thinking** before create class

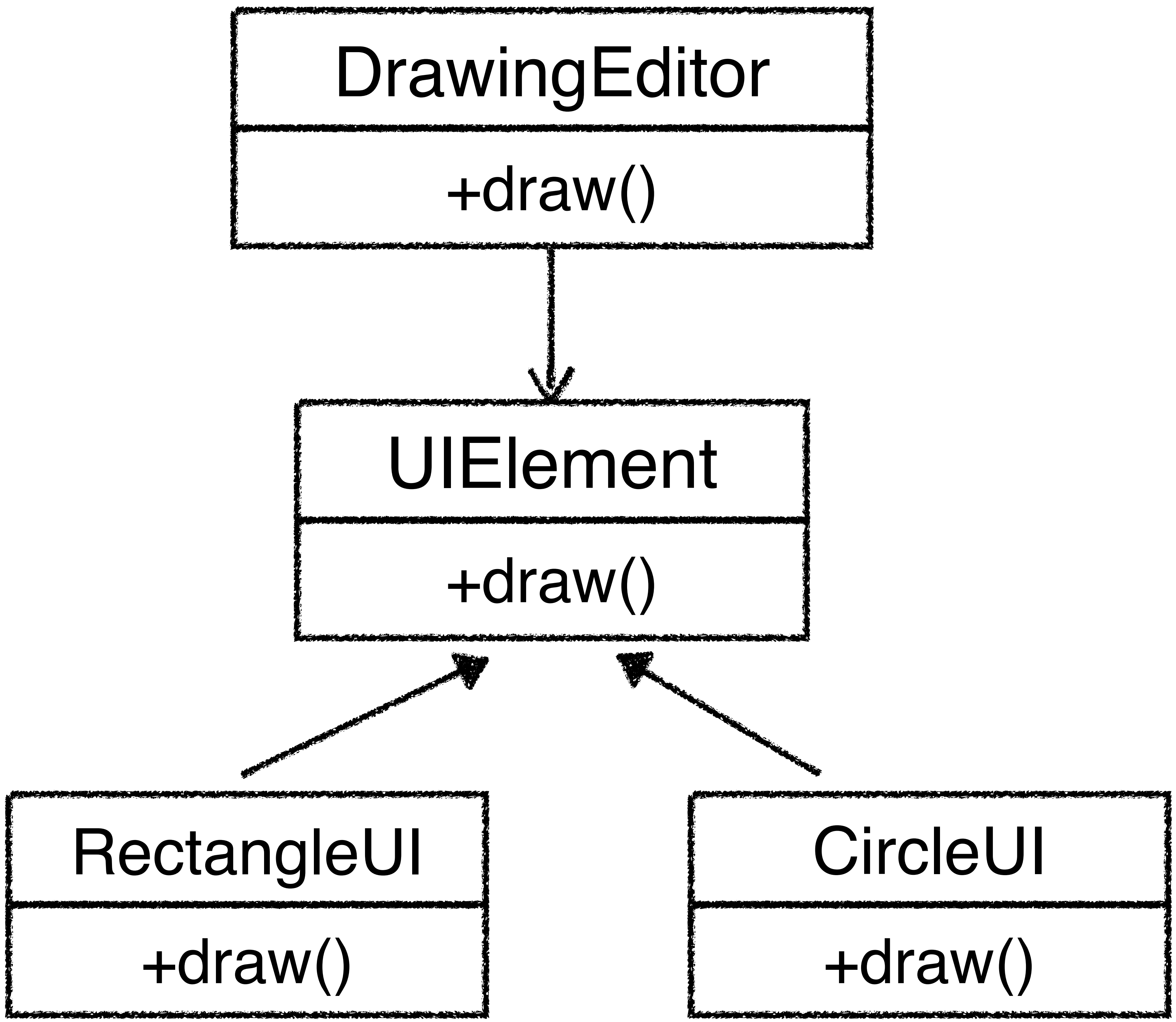
# Single Responsibility Principle

# Open-Closed Principle

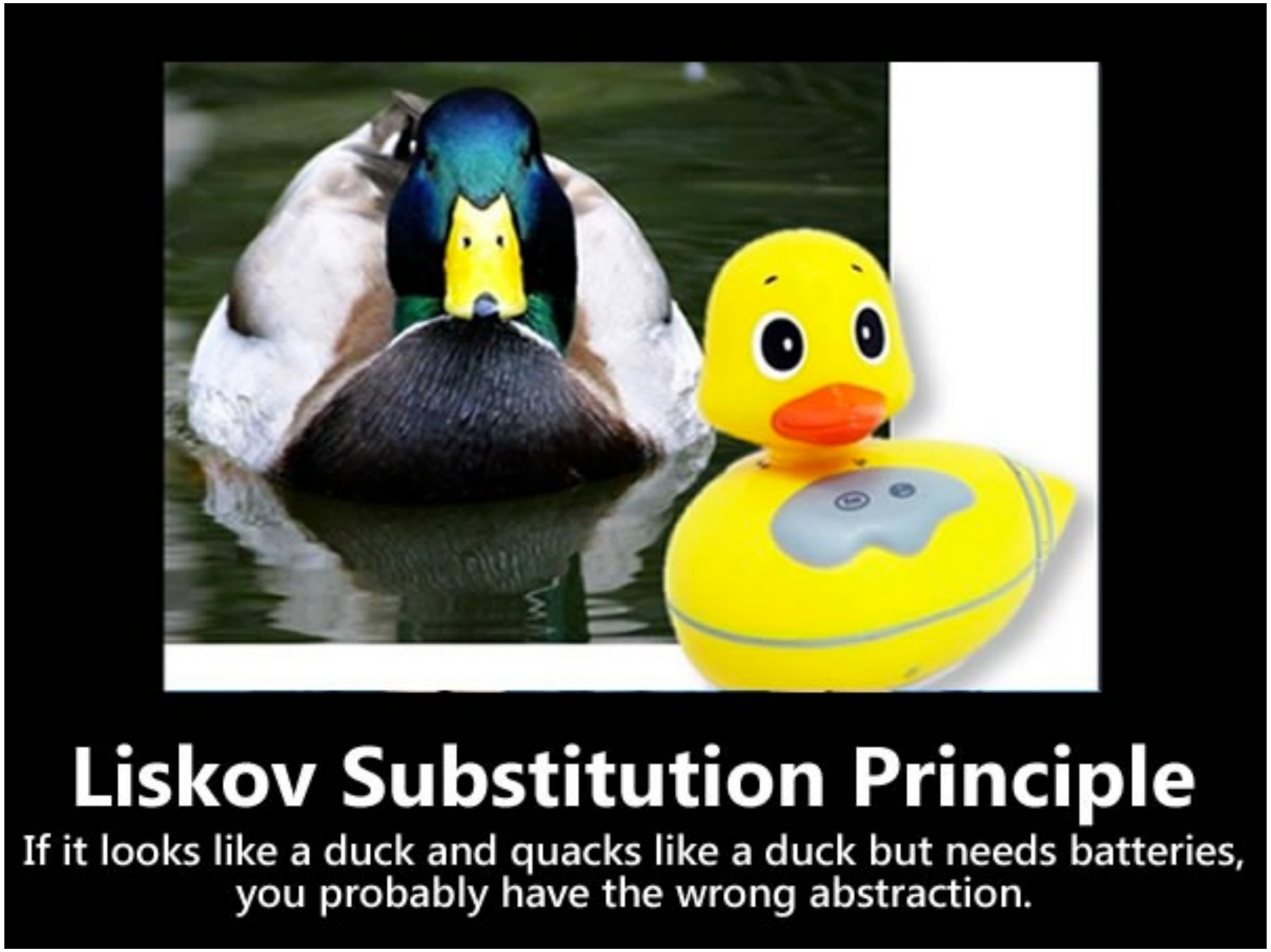
# Example

DrawingEditor

+draw()
-drawCircle()
-drawRectangle()


DrawingEditor
+draw()
UIElement
+draw()
RectangleUI
+draw()
CircleUI
+draw()

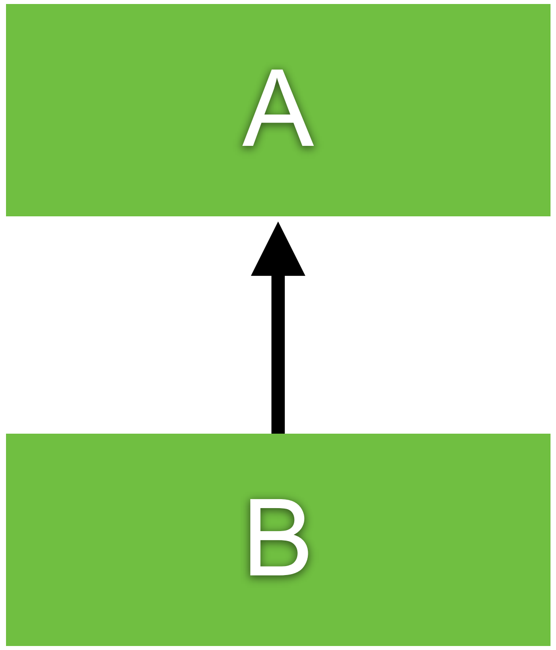
# Liskov Substitution Principle



# Inheritance

Ability to derive a new class from existing class

superclass / base class




subclass / derived class

*Additional properties and behaviors*

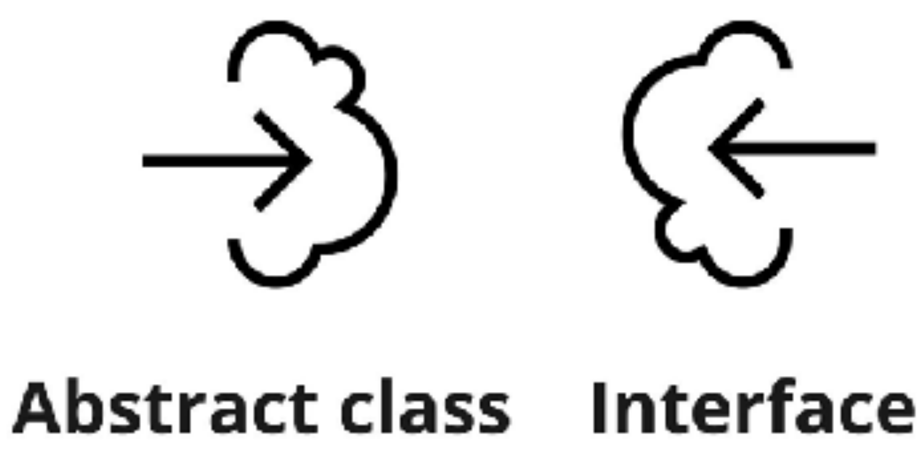
# How to create ?

Extends from base class only one class

```
class BaseClass {

}

class SubClass extends BaseClass {

}
```

# More OOP with Java

Abstract classes
Interfaces



# Abstract classes

Placeholder for a method that fully defined in a subclass

Class have abstract modifier

Method can have abstract modifier

Abstract method can't be private

Abstract method can't have body

*"Class that contains an abstract method is called **Abstract class**"*

# Classes

*“Class that contains an abstract method is called* ***Abstract class”***

*“Class that not contains an abstract method is called* ***Concrete class”***

# Abstract classes

```
abstract class Employee {

    private int id;

    public abstract double getSalary();

    public boolean sameSalary(Employee other) {
        return this.getSalary() == other.getSalary();
    }

}
```

# Pitfall of abstract class

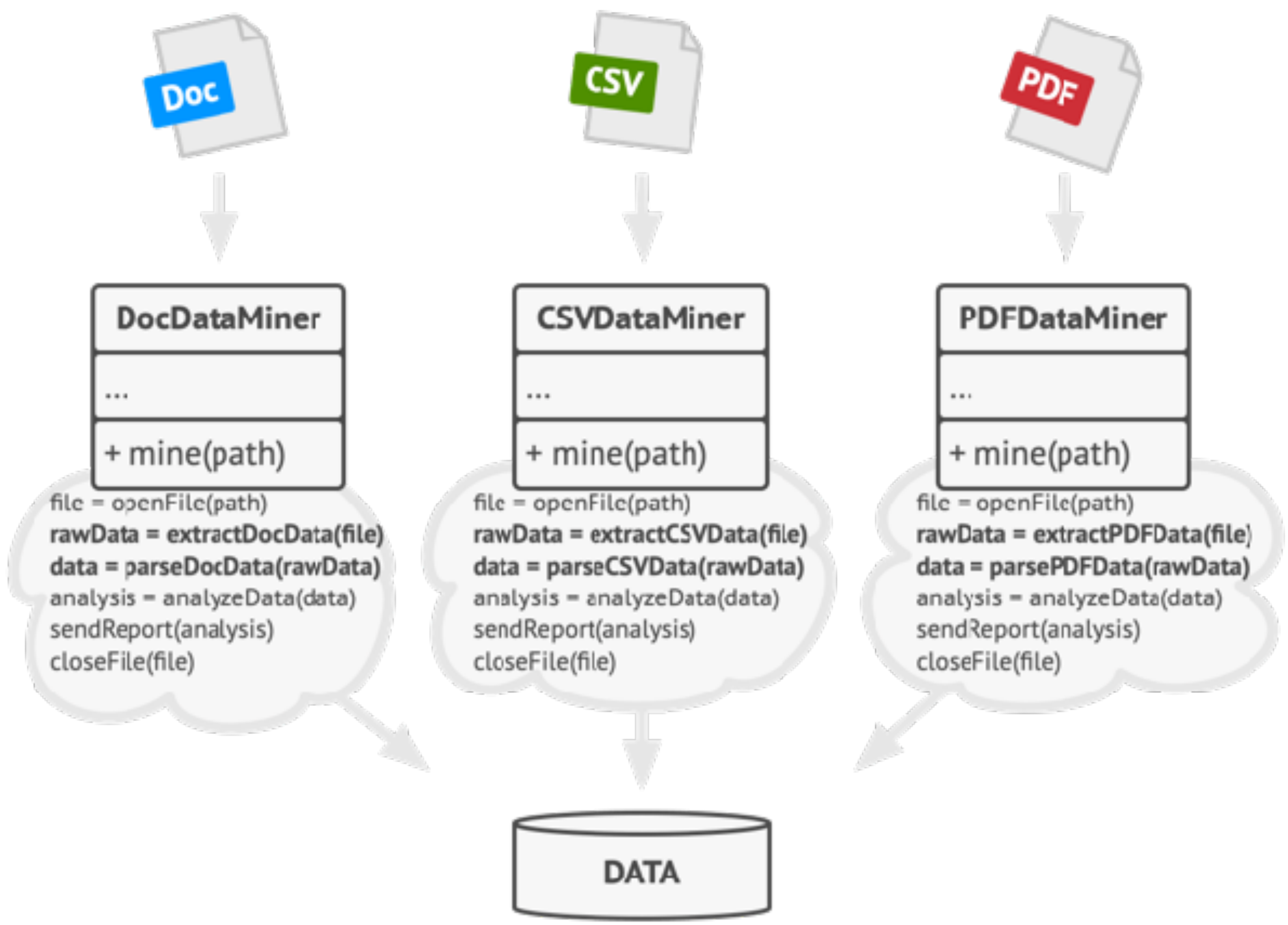
You can't create instances of an abstract class
Abstract class can only use to derive more classes

# Purpose of abstract class

Create a function as base class
But can be extended by sub class

```
abstract class Report {
    public abstract void createHeader();
    public abstract void createBody();
    public abstract void createFooter();

    public void generate() {
        createHeader();
        createBody();
        createFooter();
    }
}
```

# Template Method Pattern




https://refactoring.guru/design-patterns/template-method

# Interfaces

Like an extreme abstract class

All of methods in an interface are abstract by default

Not a class

No instance variables

```
interface Animal {
    void eat();
    void sleep();
}
```

# Using interface from class (1)

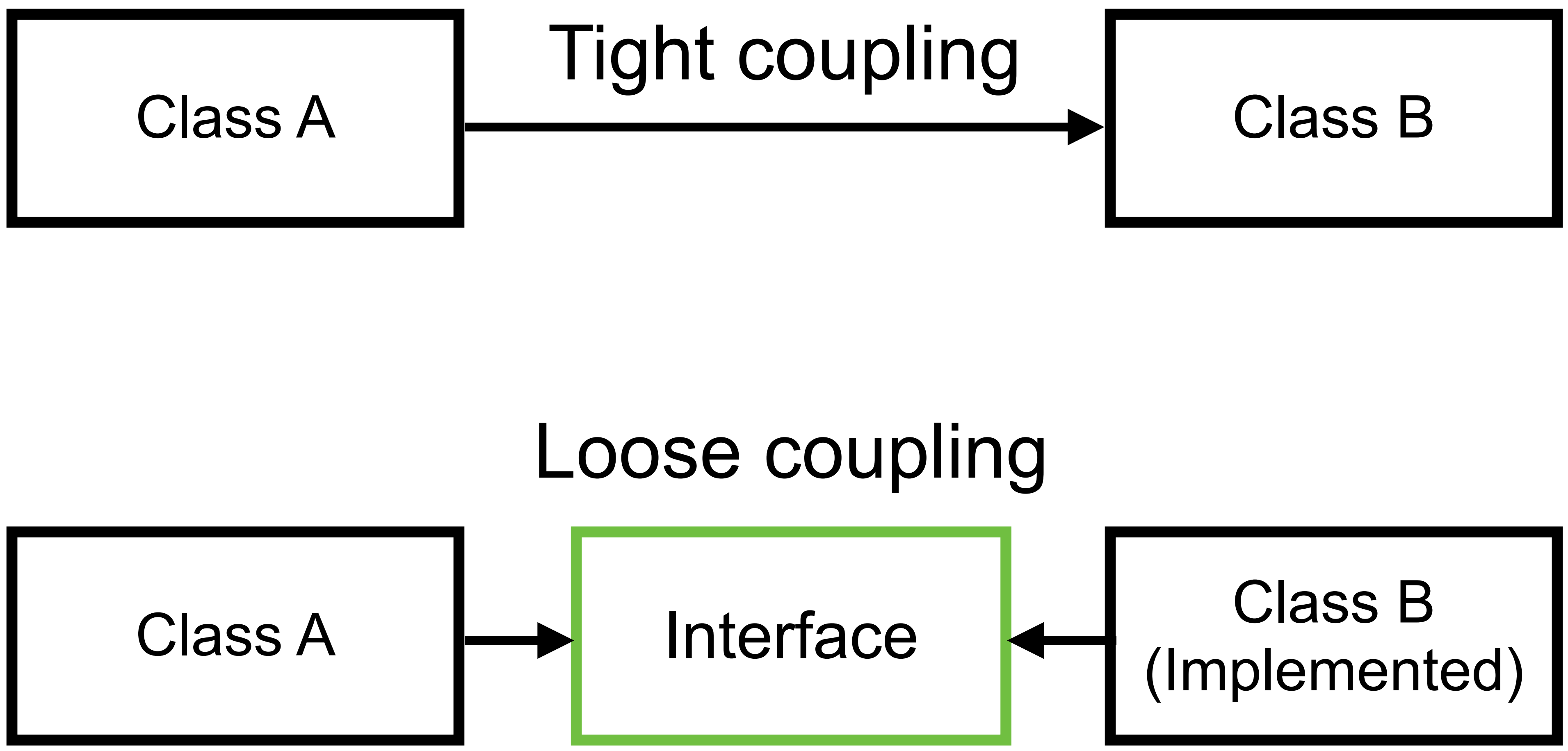
```java
class Cat implements Animal {

    @Override
    public void eat() {
    }

    @Override
    public void sleep() {
    }

}
```

# Using interface from class (2)

```
class Cat implements Animal {

    @Override
    public void eat() {
    }

    @Override
    public void sleep() {
    }

}
```

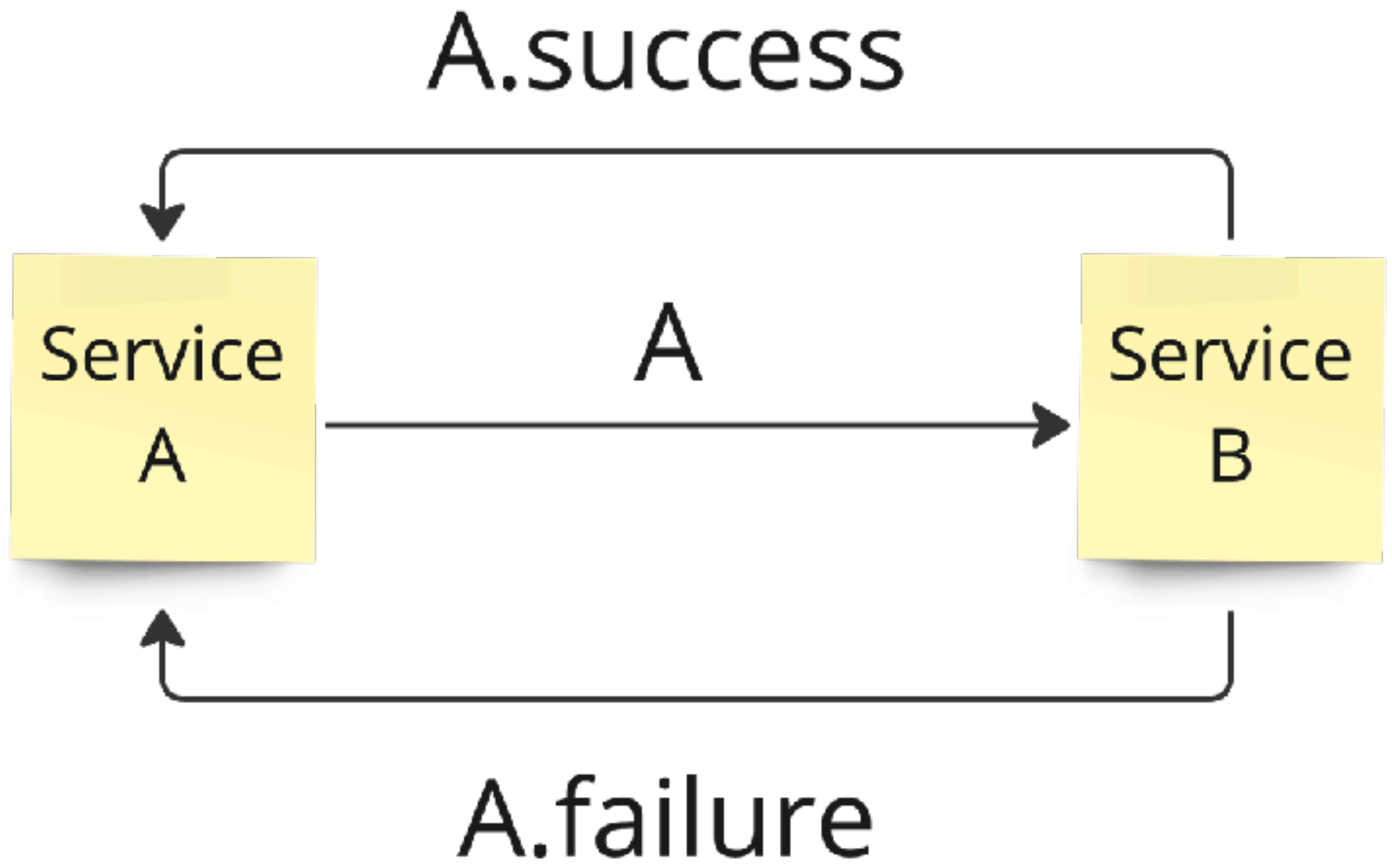
*Need to implements all method from interface*

# When to use interface ?

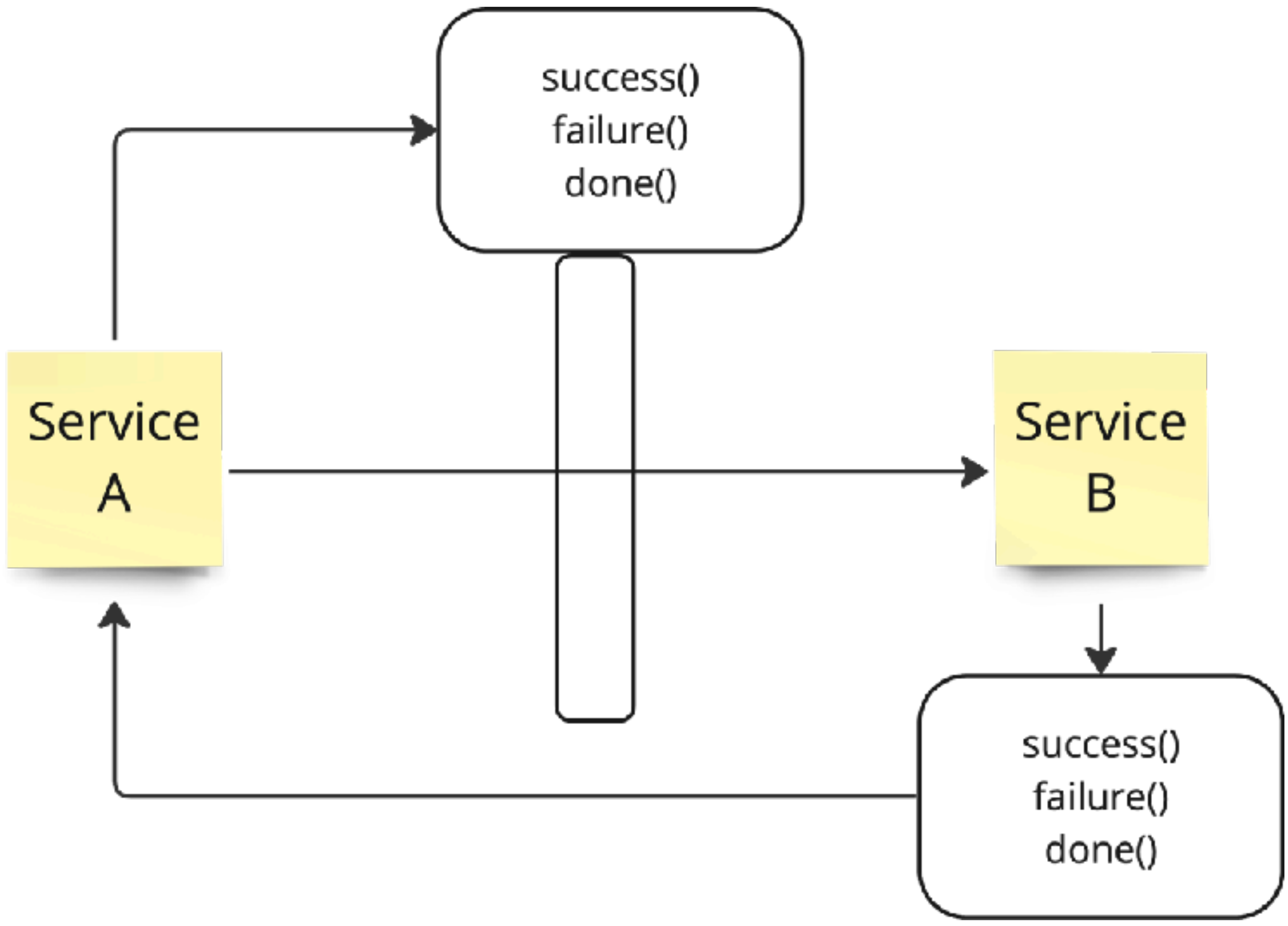


# Callback with Interface

# Call back with class (1)

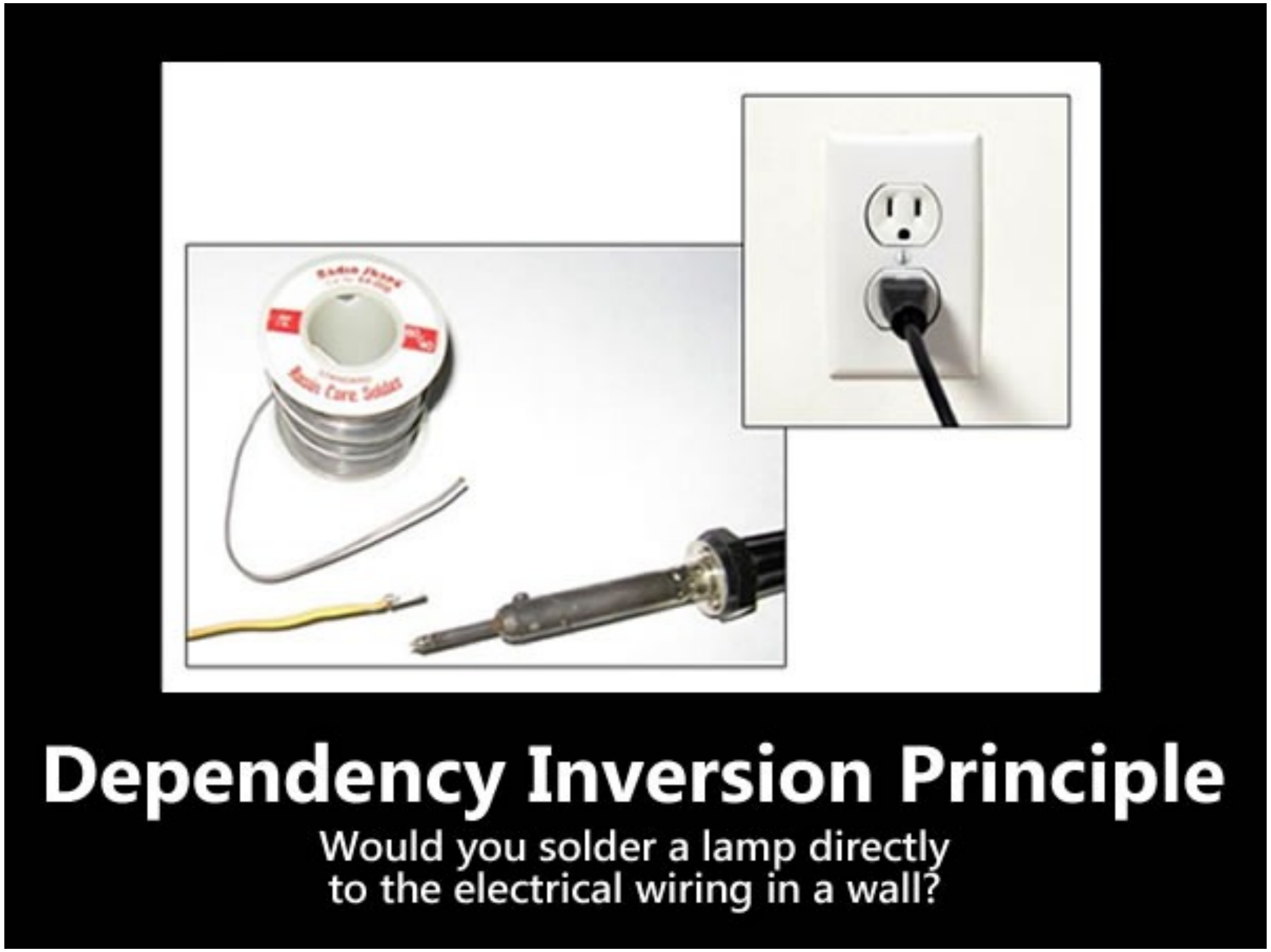


# Call back with interface (2)

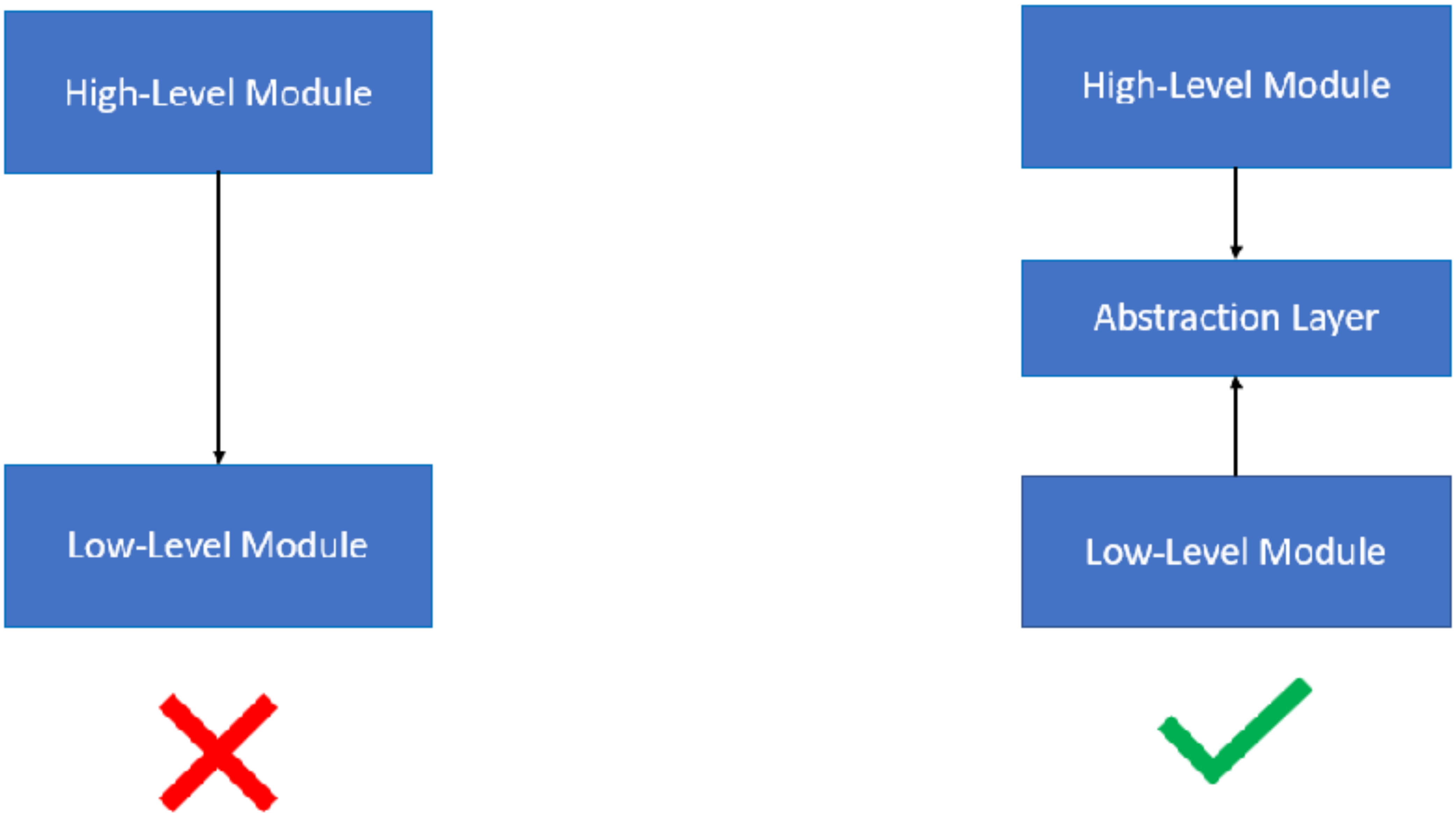


# Interface Segregation Principle

# Dependency Inversion Principle

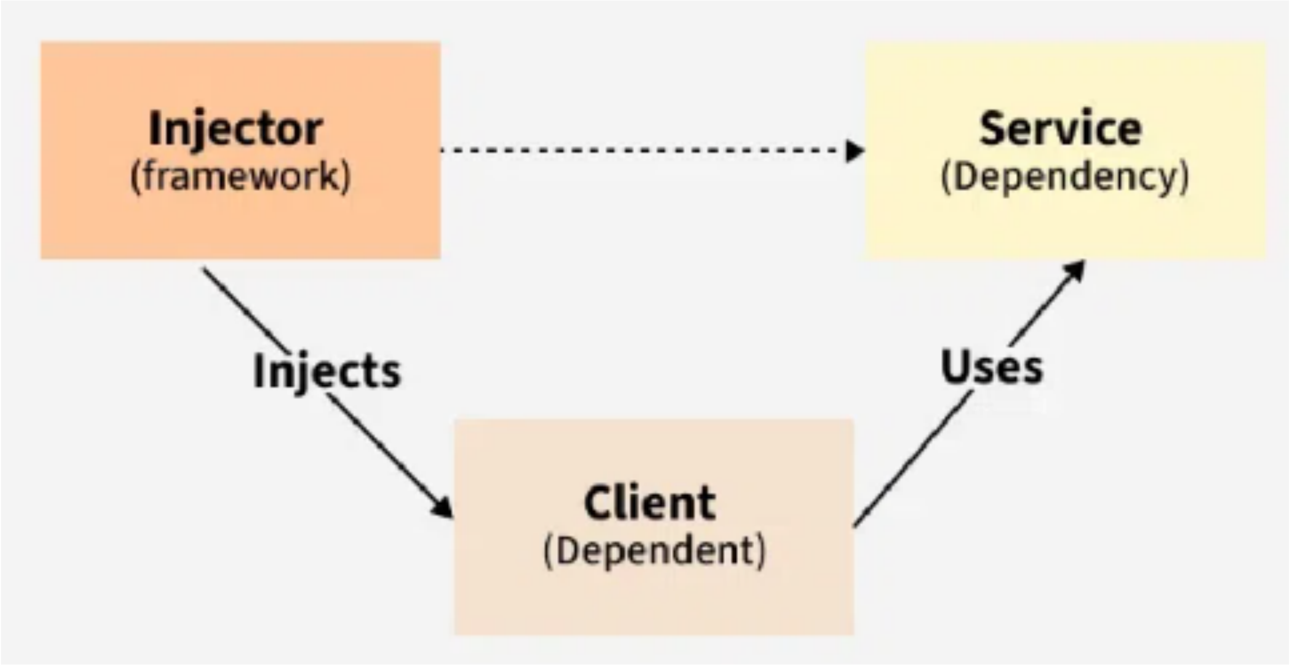


# Depend on abstraction



# Dependency Injection

Property injection
Constructor injection
Method injection
Interface injection



# Java Collection Framework

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/doc-files/coll-overview.html

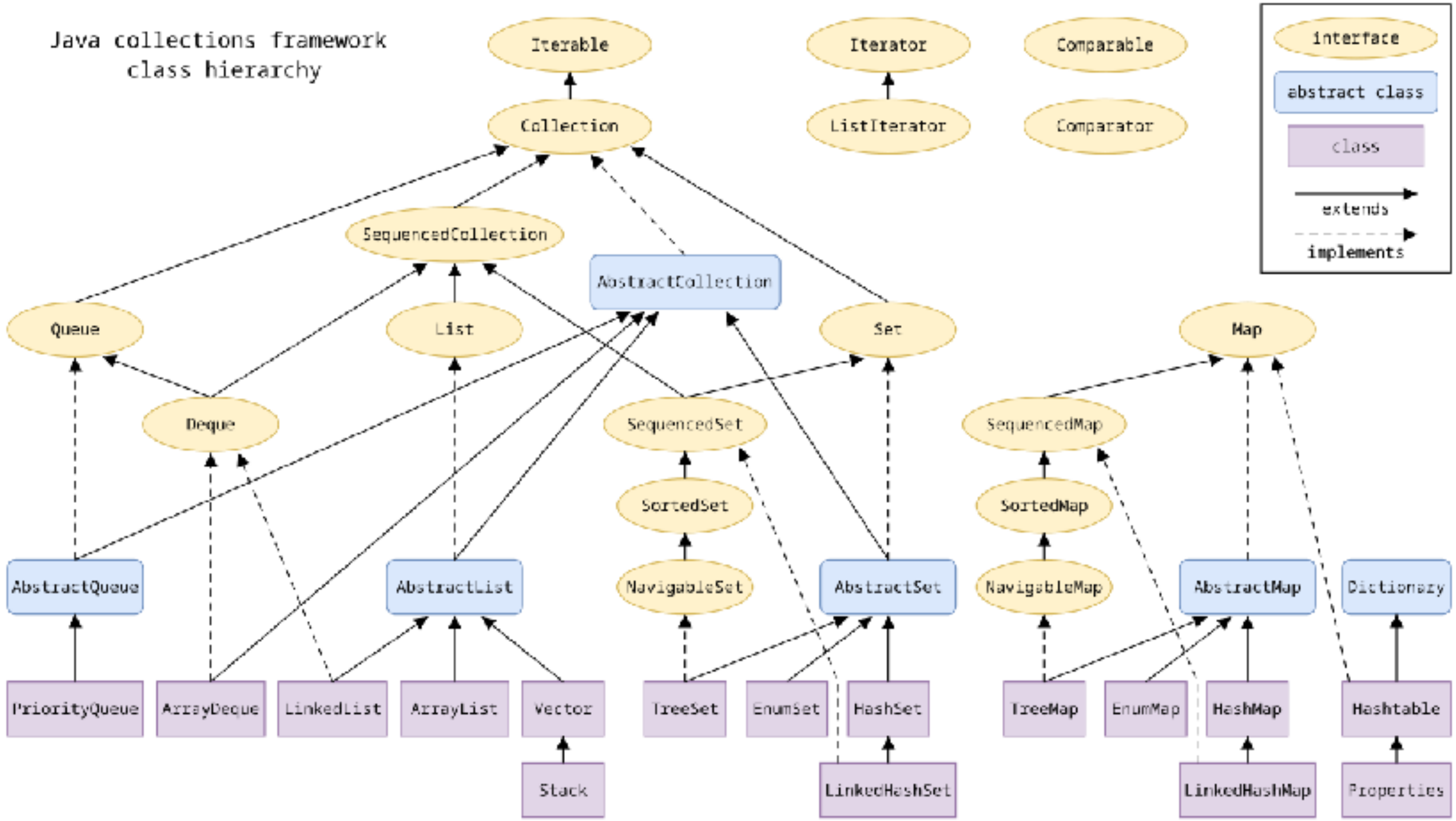
# Java Collection Framework

**General purpose implementations**

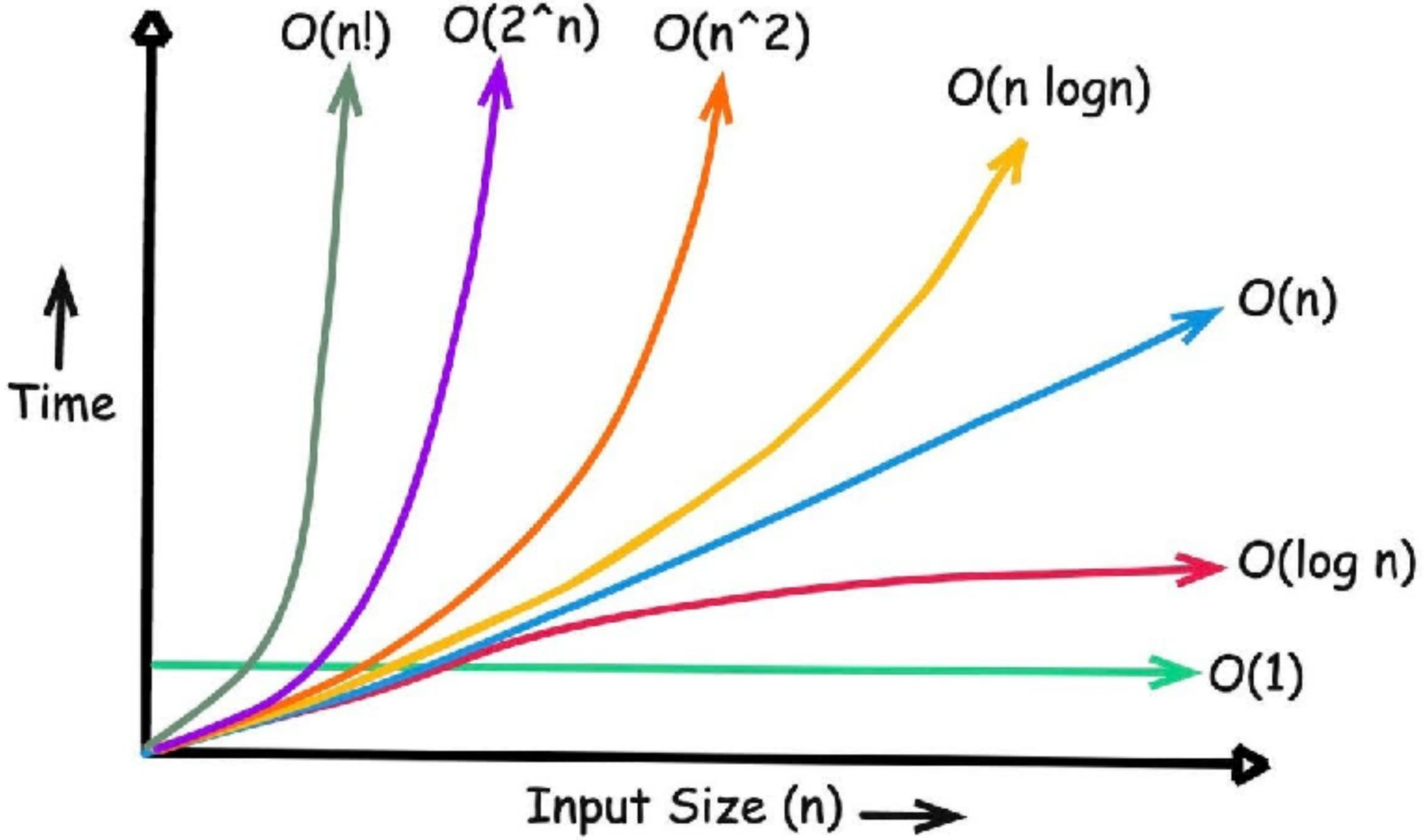
| Interface | Hash Table | Resizable Array | Balanced Tree | Linked List | Hash Table + Linked List |
|---|---|---|---|---|---|
| Set | HashSet | | TreeSet | | LinkedHashSet |
| List | | ArrayList | | LinkedList | |
| Queue, Deque | | ArrayDeque | | LinkedList | |
| Map | HashMap | | TreeMap | | LinkedHashMap |

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/doc-files/coll-overview.html

# Java Collection Framework



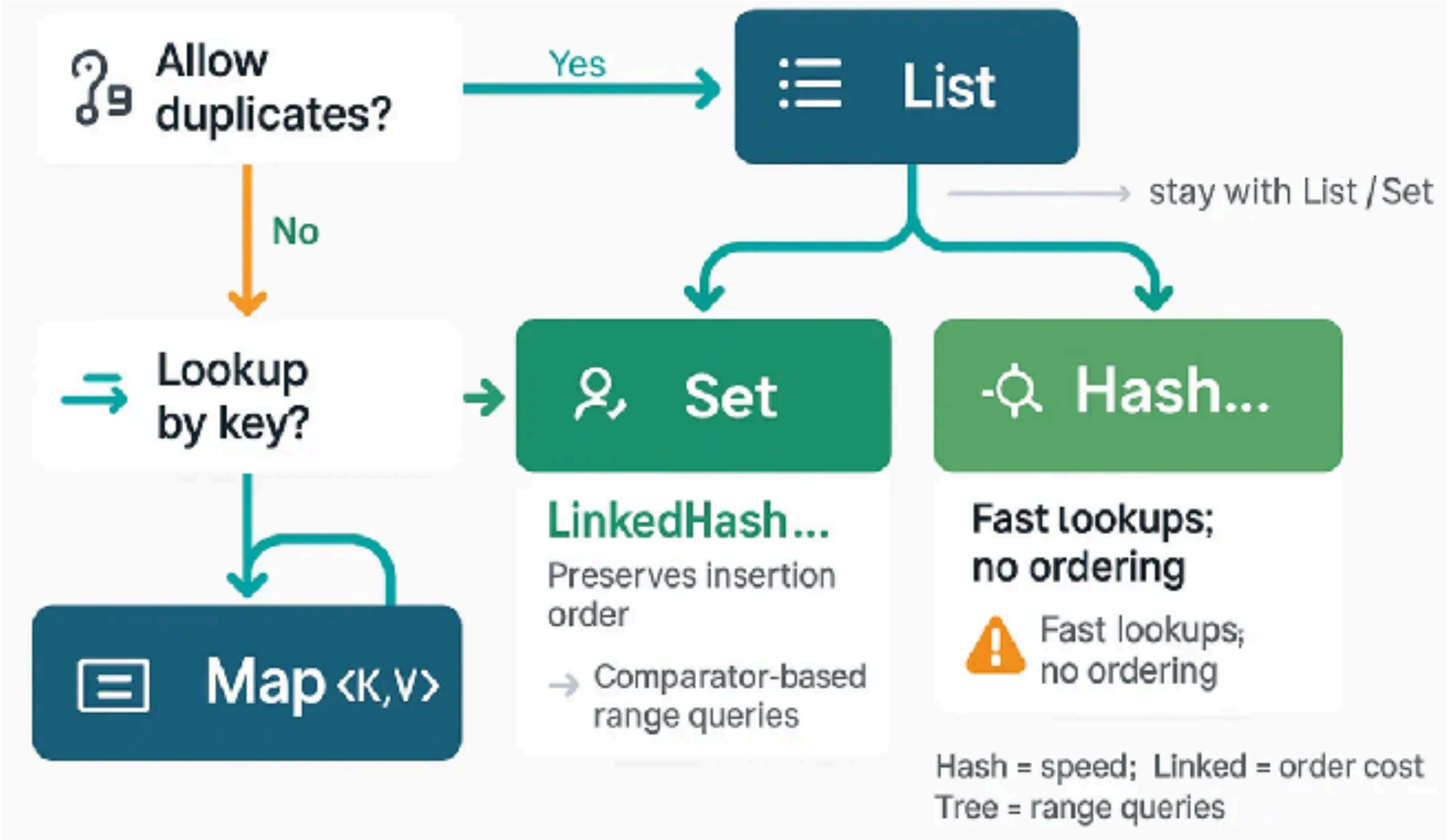
# Big O Notation



# Working with use cases

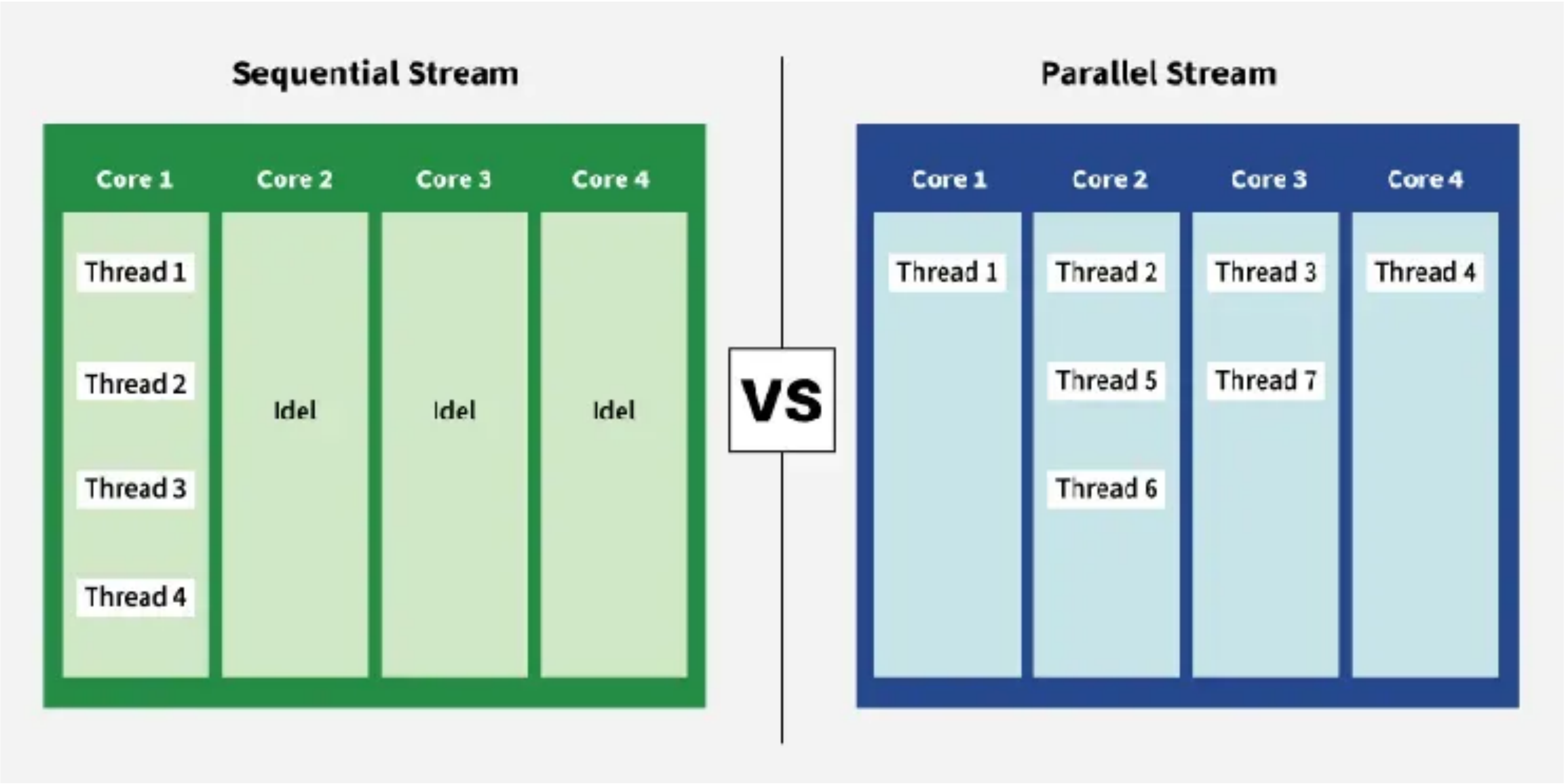
# List vs Map

## Lookup optimization



# Java Stream

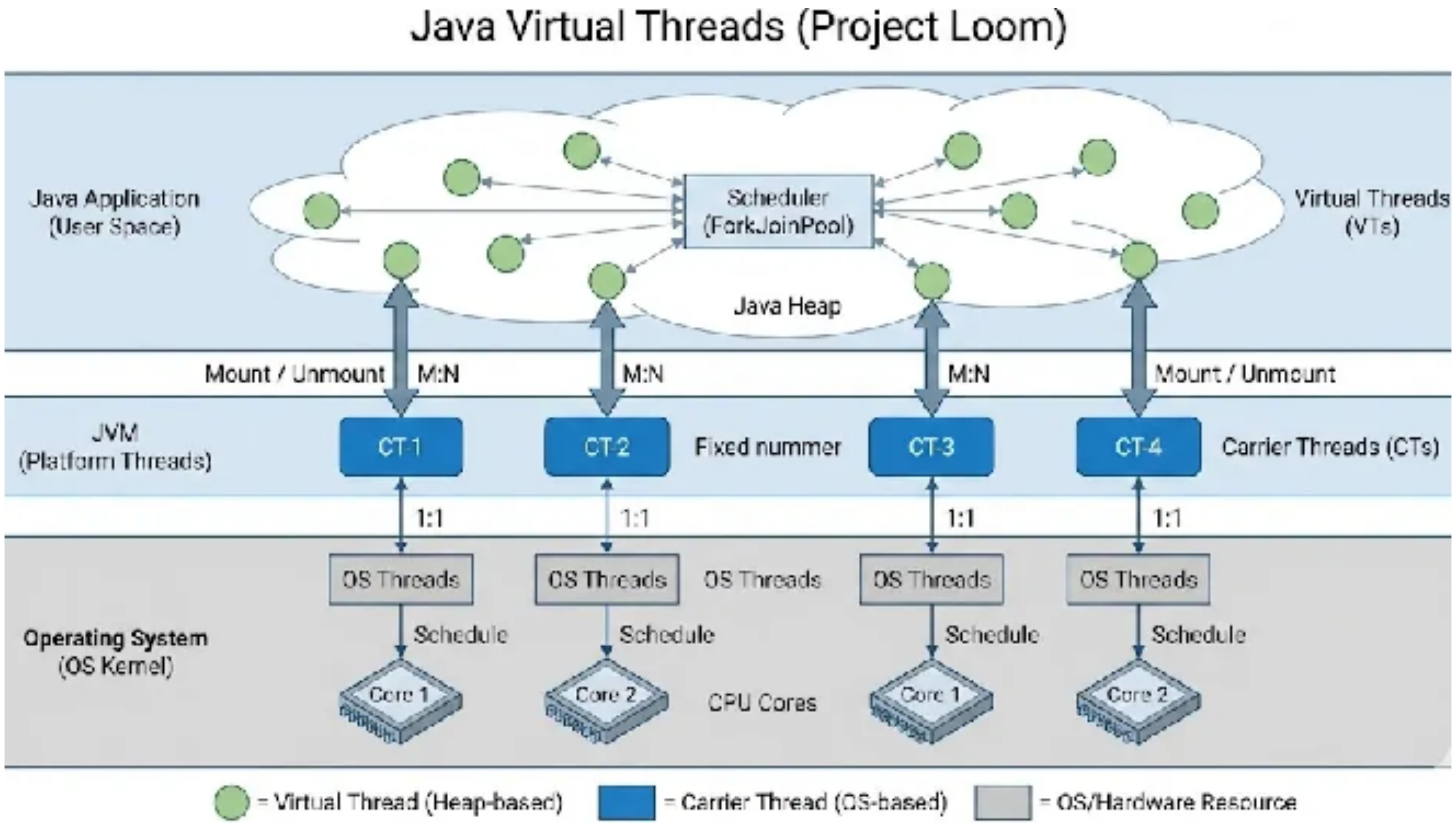
## Sequencial vs Parallel




https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/stream/Stream.html

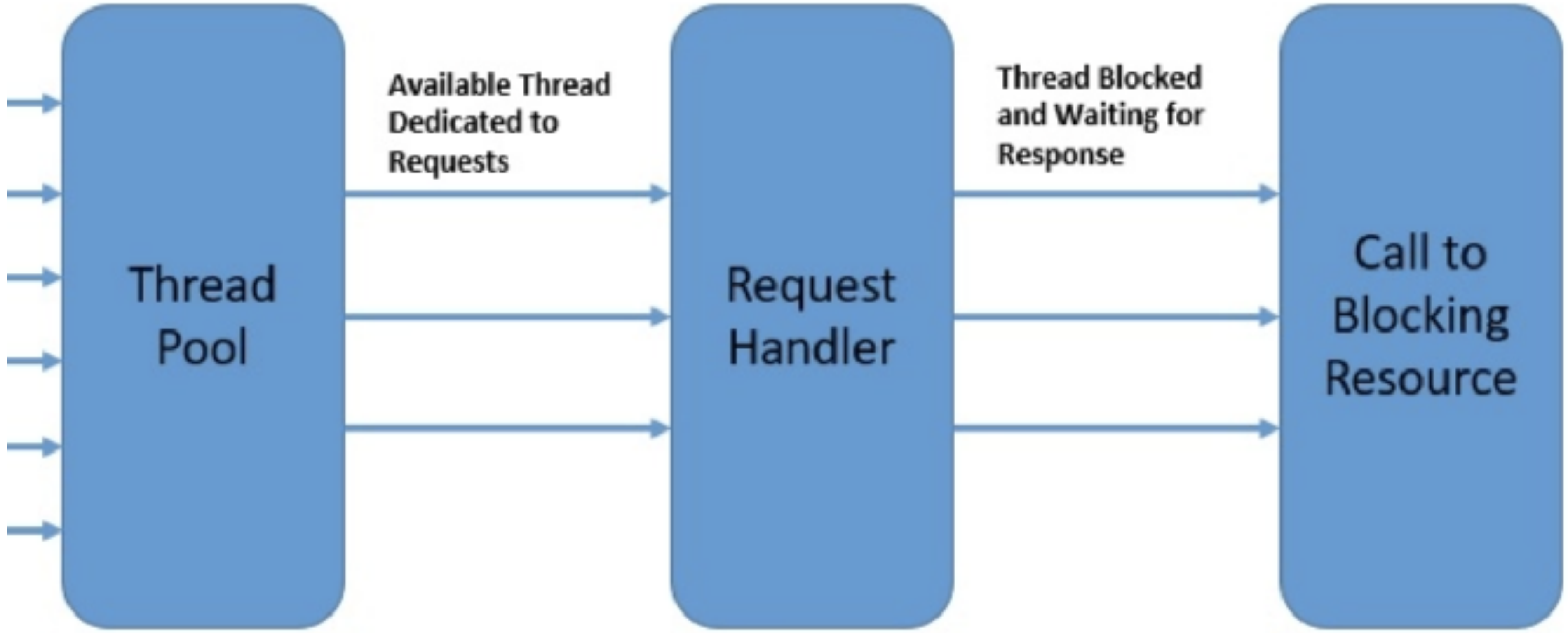
# Java Virtual Threads (Project Loom)

Java 21
Lightweight, JVM-managed thread




https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html

# Spring Boot with Virtual Thread !!

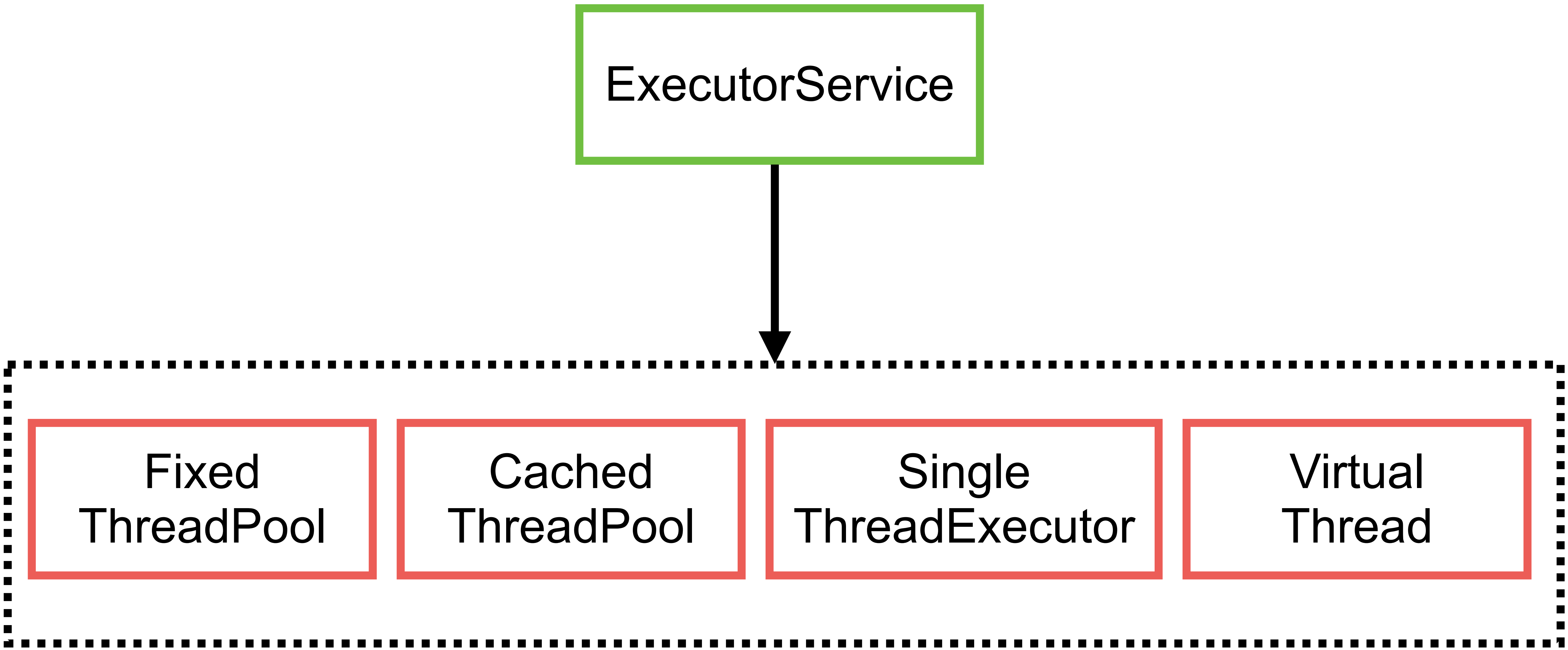



https://www.somkiat.cc/virtual-thread-in-spring-boot-3/

# Platform vs Virtual Thread

| Feature | Platform thread | Virtual thread (Java21+) |
| --- | --- | --- |
| **Mapping** | 1:1<br>with OS threads | M:N<br>Many virtual threads to few OS threads |
| **Creation cost** | High<br>memory and CPU | Low |
| **Memory footprint** | 1 MB per thread<br>(Stack size) | < 1 KB<br>(Stored on heap) |
| **Use case** | CPU-intensive tasks | I/O intensive tasks (blocking call) |
| **Scalability** | Limit by OS resources | Scalable to millions |
| **Pooling ?** | Required to prevent resource exhaustion | Create a new one per task |

# High level concurrency framework



# Executor Service

Thread pooling

Task management (runnable, callable)

Life cycle control

Working with try-with-resources (AutoClosable)

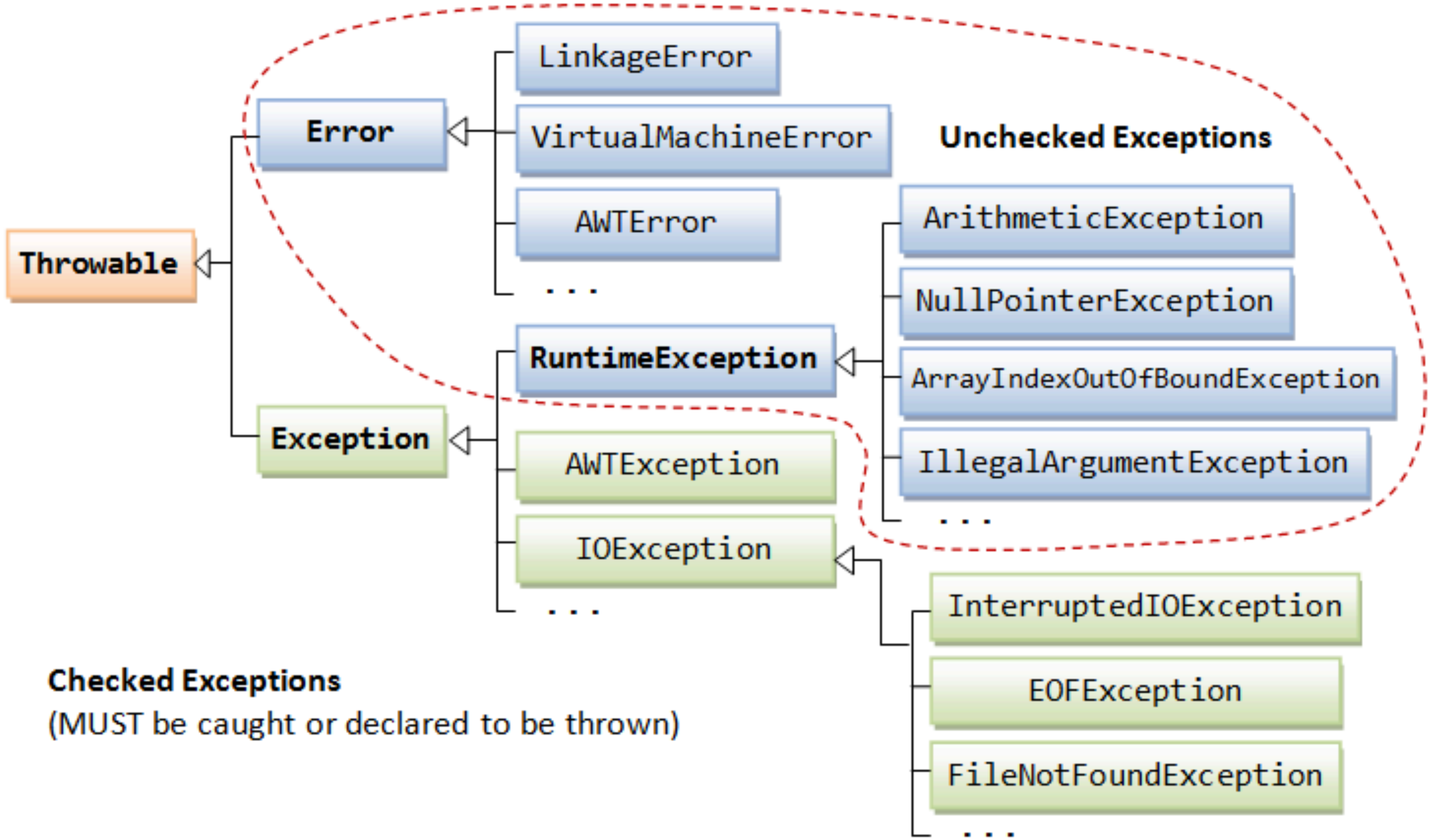
https://docs.oracle.com/en/java/javase/25/docs/api//java.base/java/util/concurrent/ExecutorService.html

# Exception Handling

# Types of Exception

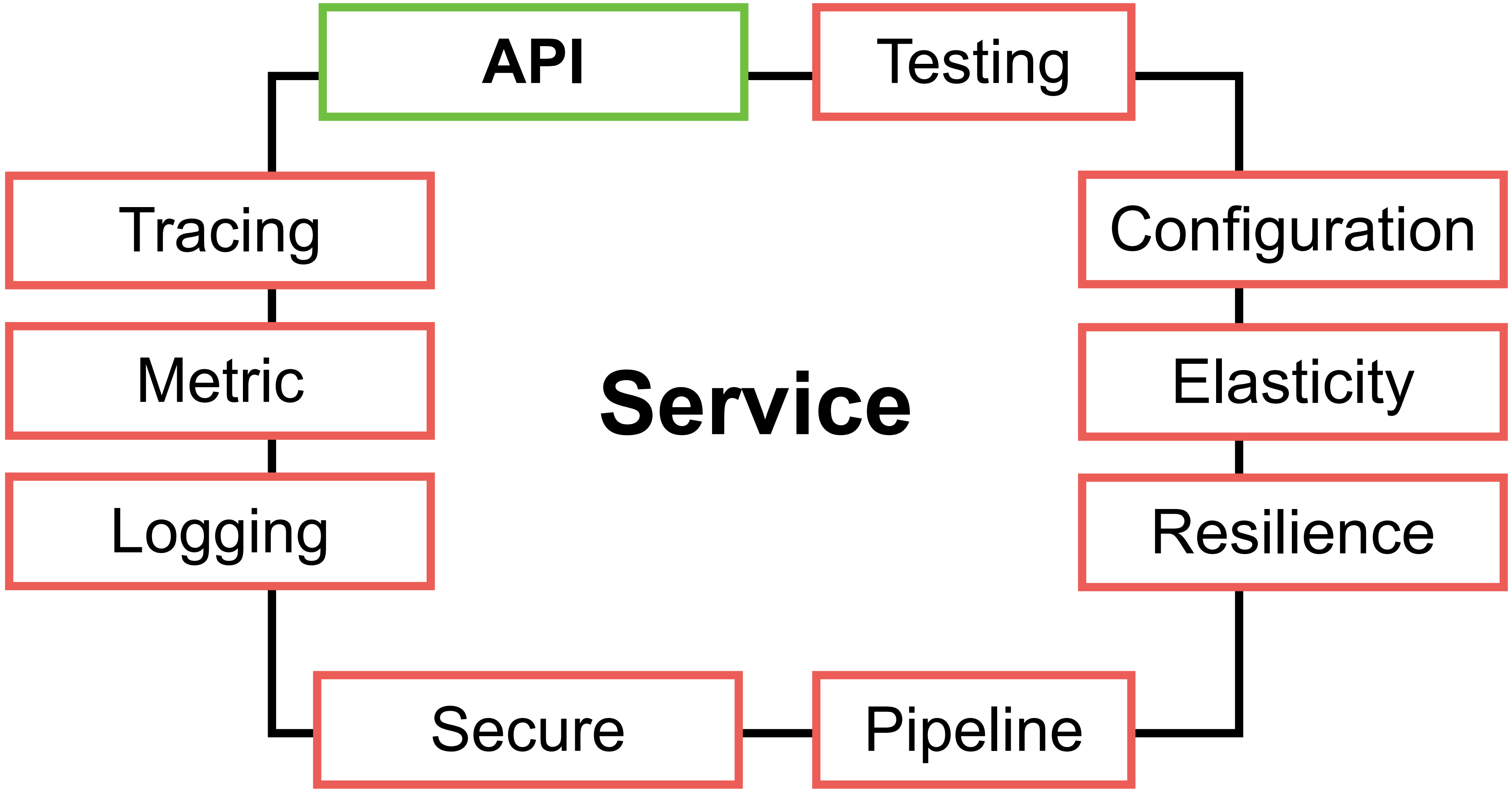
- Compile-time
- Runtime
- Logical

# Exception Classes



# REST API with Spring Boot

# Properties of Service



# APIs

Application Programming Interface

| **REST API** | gRPC | Messaging |
|---|---|---|

# REST

**RE**presentation **S**tate **T**ransfer

The style of **software architecture** behind RESTful services

Defined in 2000 by Roy Fielding

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

# REST Request & Response

**Request Method**

GET, POST, PUT, DELETE


**Response Format**

XML or JSON

# HTTP Methods meaning

| Method | Meaning |
|---|---|
| GET | Read data |
| POST | Create/Insert new data |
| PUT/PATCH | Update data or insert if a new id |
| DELETE | Delete data |

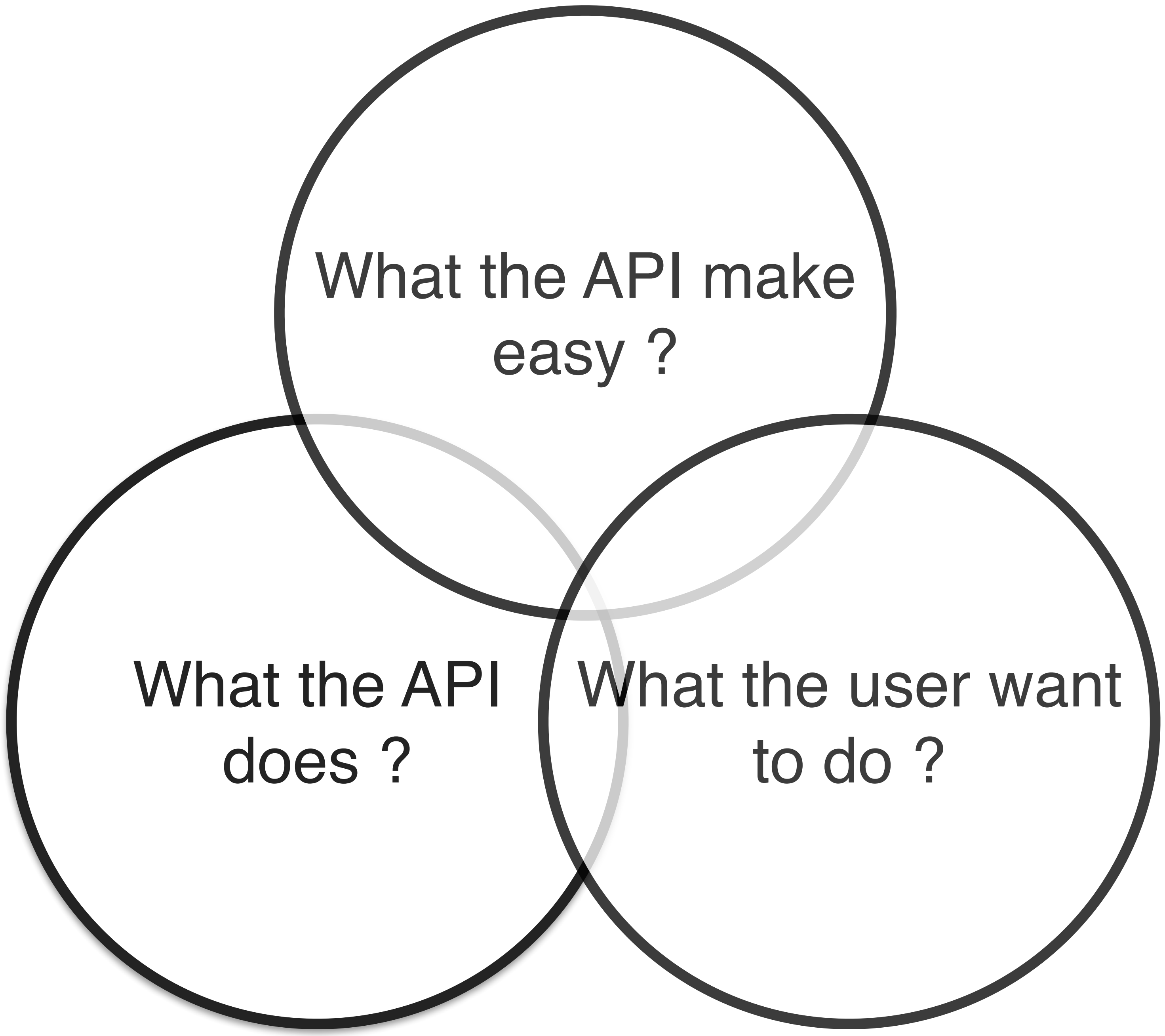
# Response format ?

# Status Code

| Code | Description |
| --- | --- |
| 1xx | Information |
| 2xx | Successfu |
| 3xx | Redirection |
| 4xx | Client error |
| 5xx | Server error |

https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

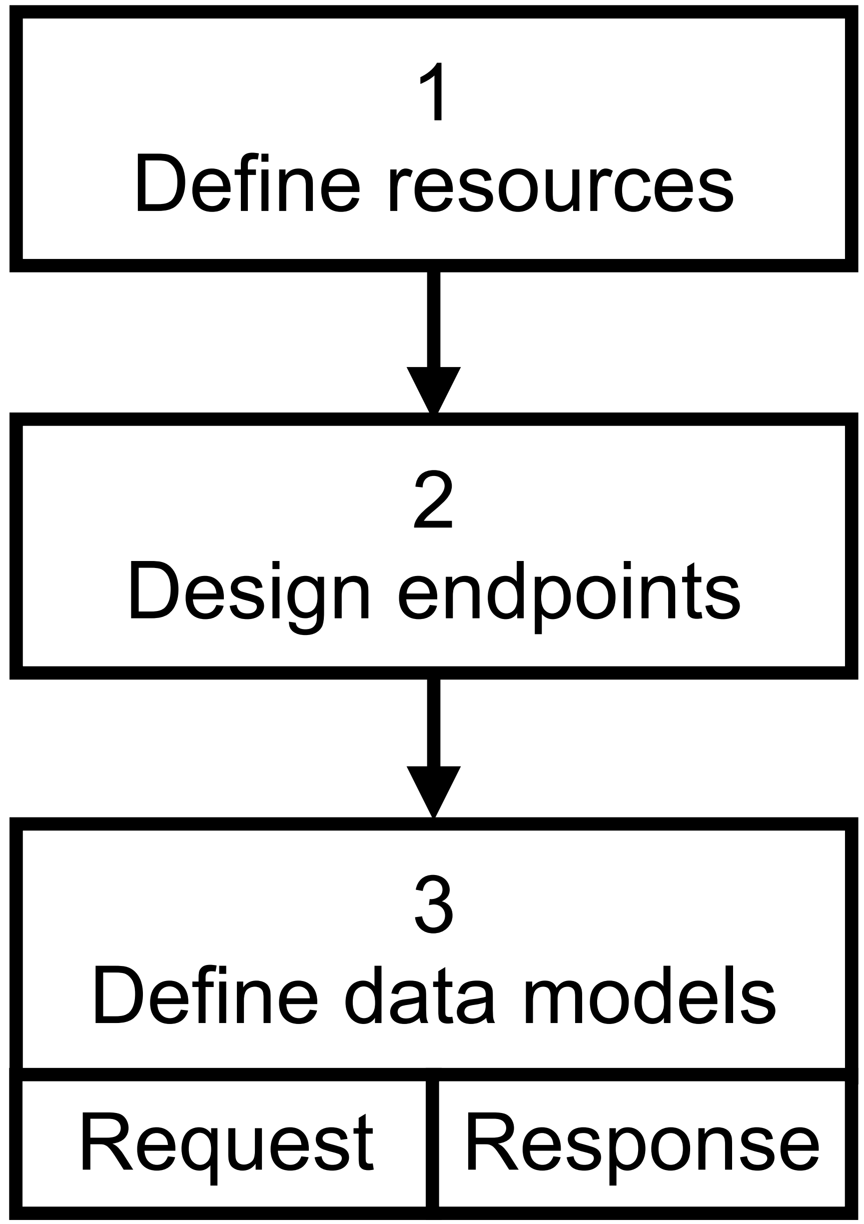
# Good APIs ?


What the API make easy ?
What the API does ?
What the user want to do ?

# Principles of API Design

- Consistency
- Statelessness
- Resource-oriented design
- Use standard HTTP methods
- Versioning

# Steps to Design APIs



# More !!

Authentication
Authorization

Pagination and
filtering

Rate Limiting

Pagination and
filtering

Error handling

Documentation

Testing

Monitoring and
Observability

# Implementation

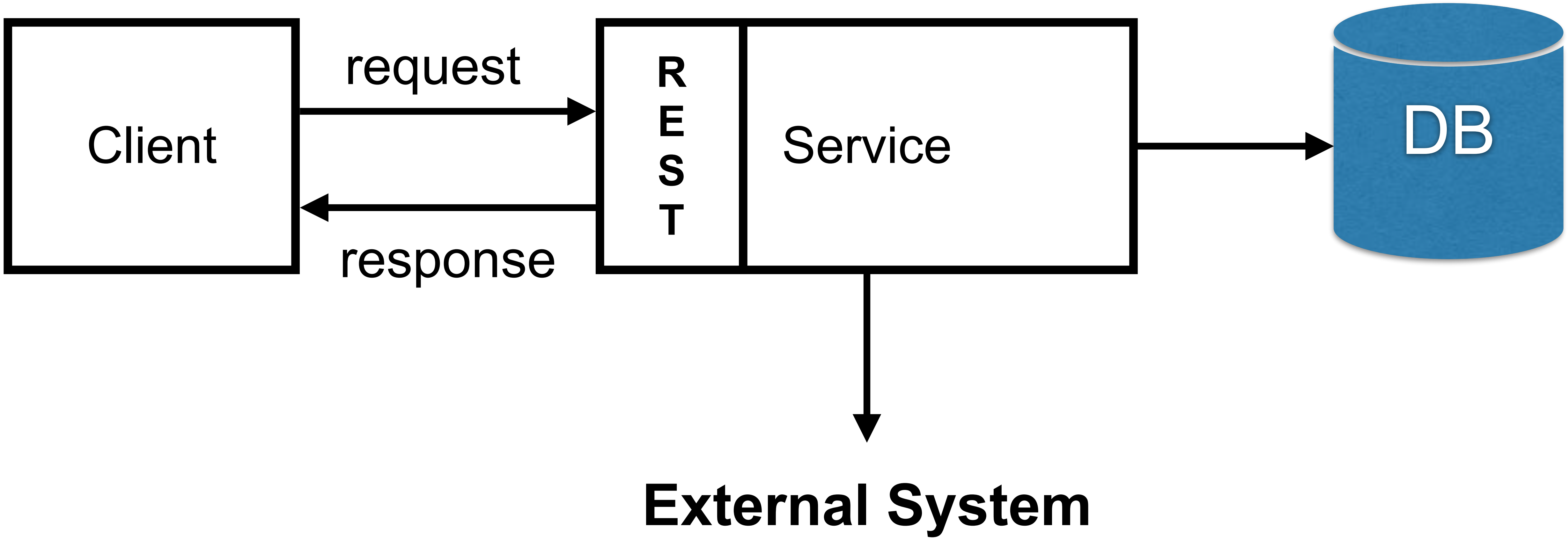
| Coding | Testing |
|---|---|

Observability

Performance

Security

# Structure of Service



# Spring Boot Structure (1)

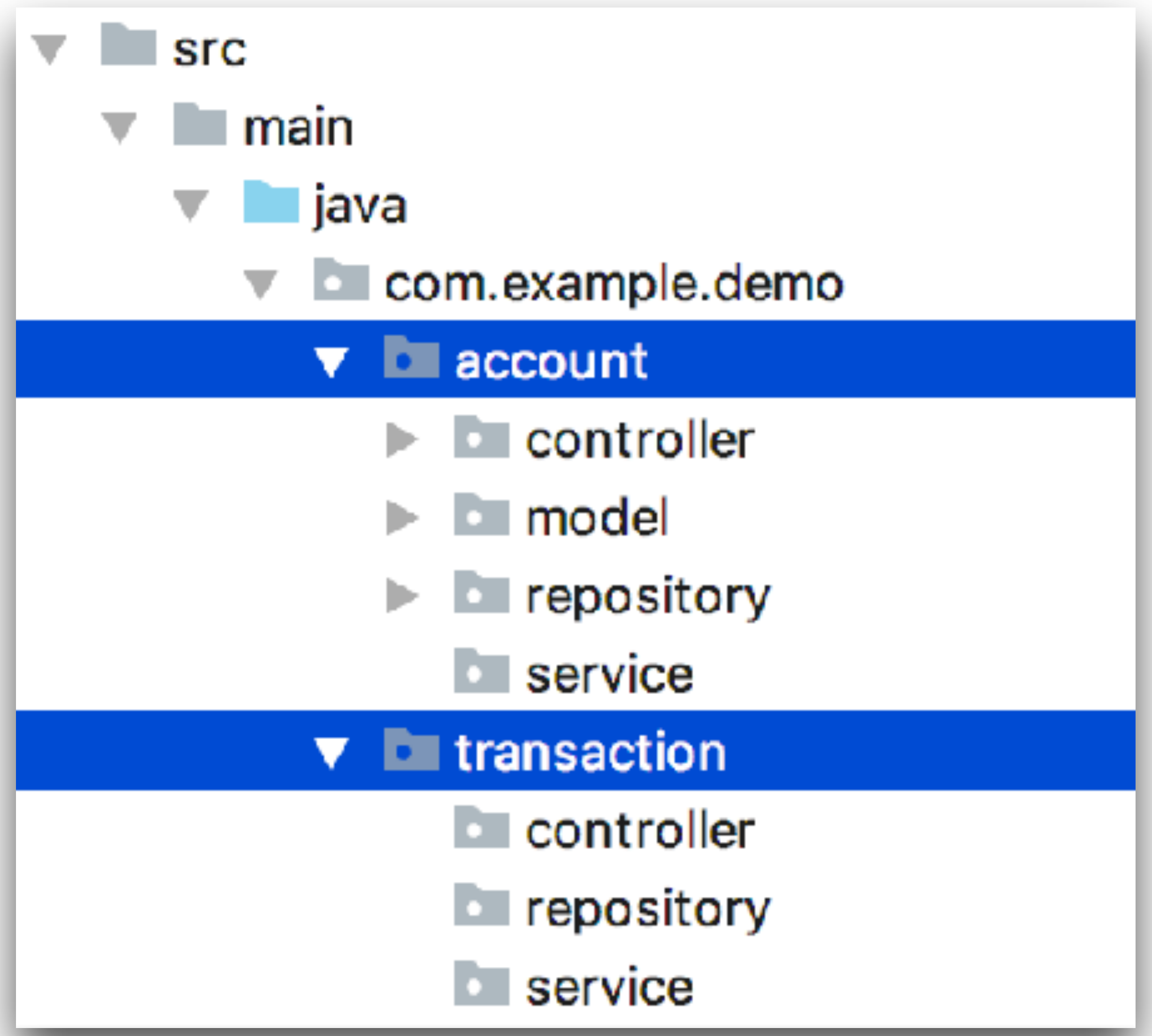
Separate by function/domain/feature

```
feature1
|    - controller
|    - service
|    - repository
feature2
|    - controller
|    - service
|    - repositor
```

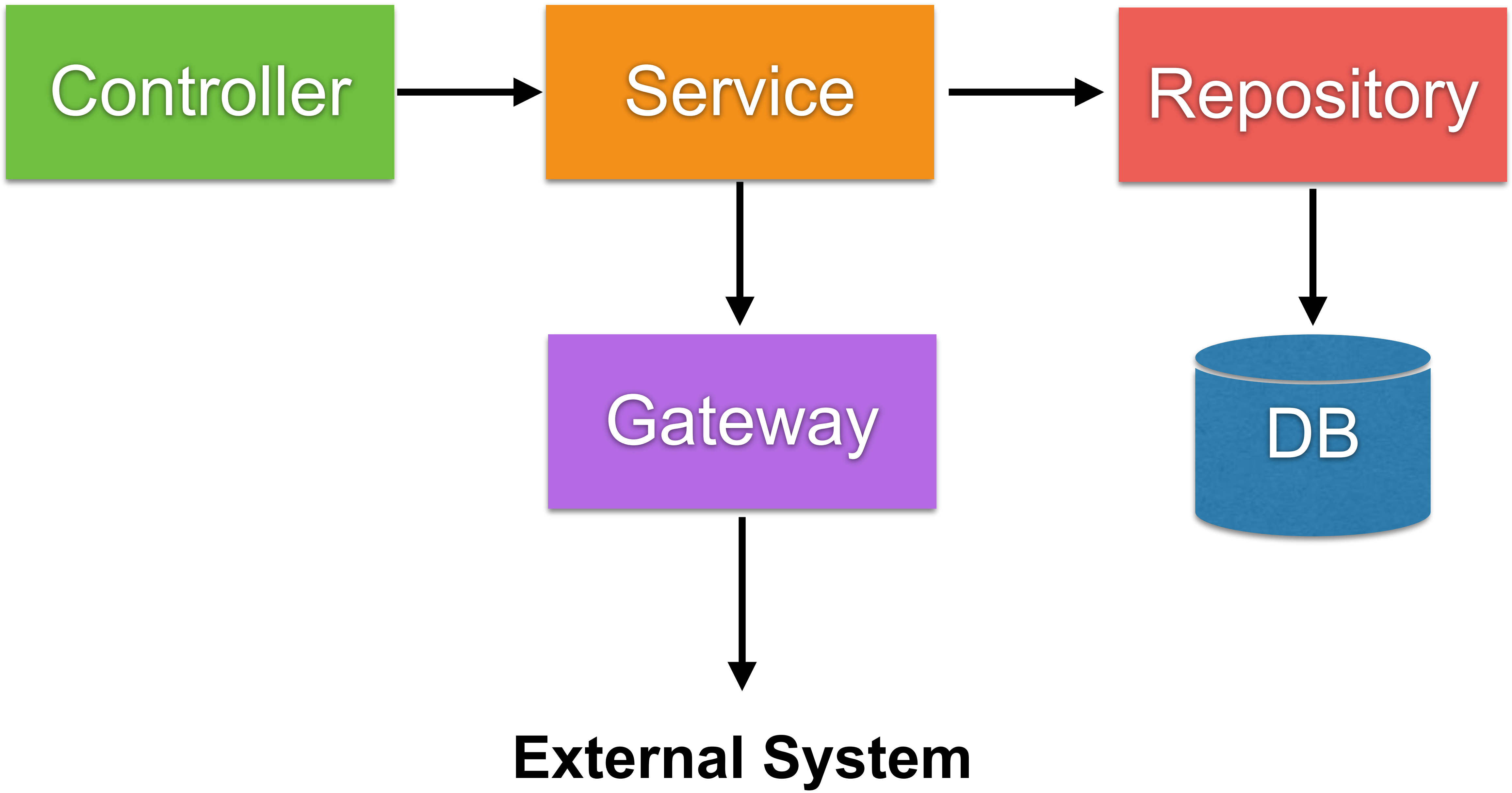
https://docs.spring.io/spring-boot/reference/using/structuring-your-code.html

# Spring Boot Structure (2)

Separate by function/domain/feature



# Structure of Spring Boot project



# Controller

Request and Response
Validation input
Delegate to others classes such as service and repository

# Service

Control the flow of main business logics
Manage database transaction
Don’t reuse service

# Repository

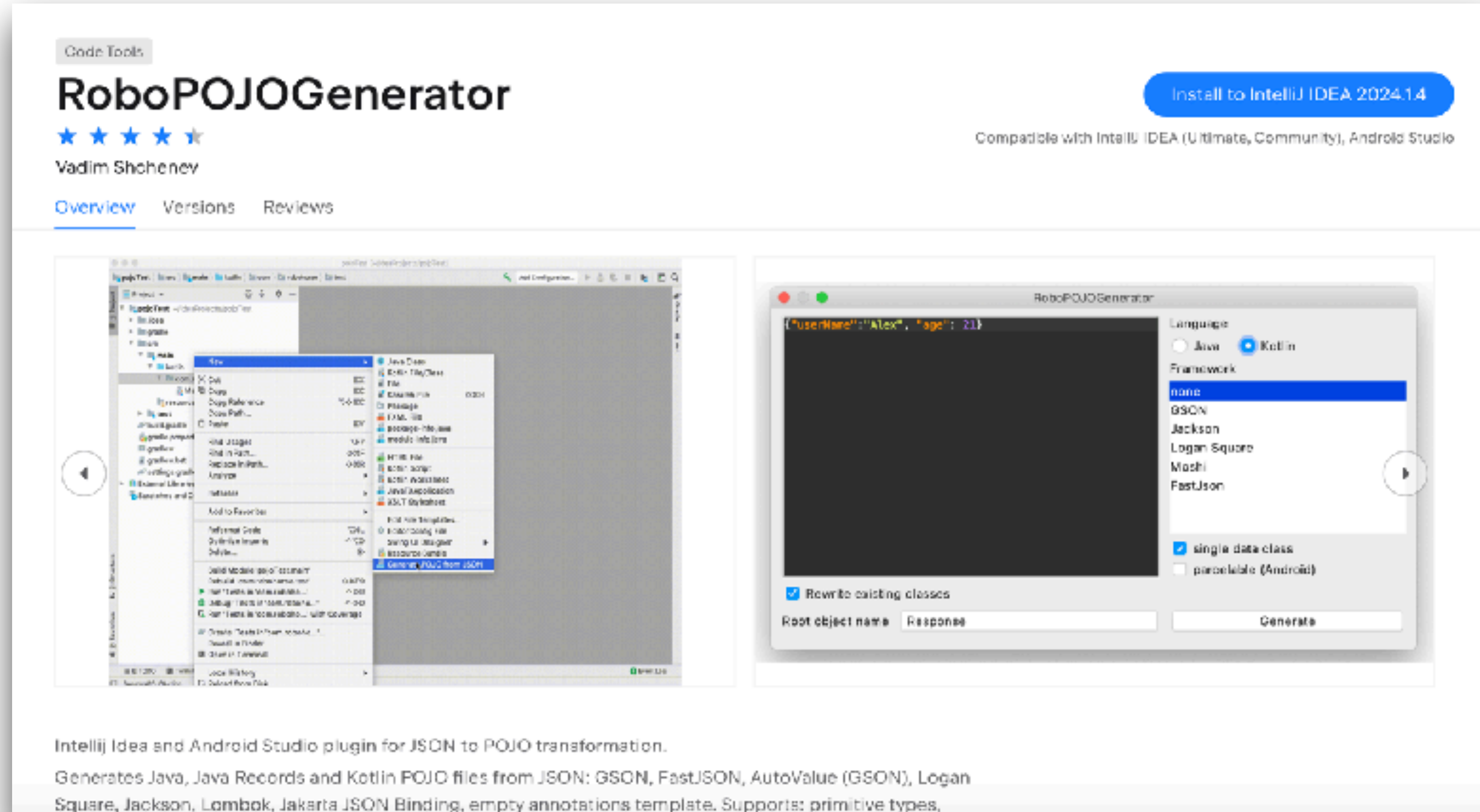
Manage data in data store such as RDBMS and NoSQL

# Gateway

Call external service via network such as WebServices and RESTful APIs

# Convert JSON to POJO

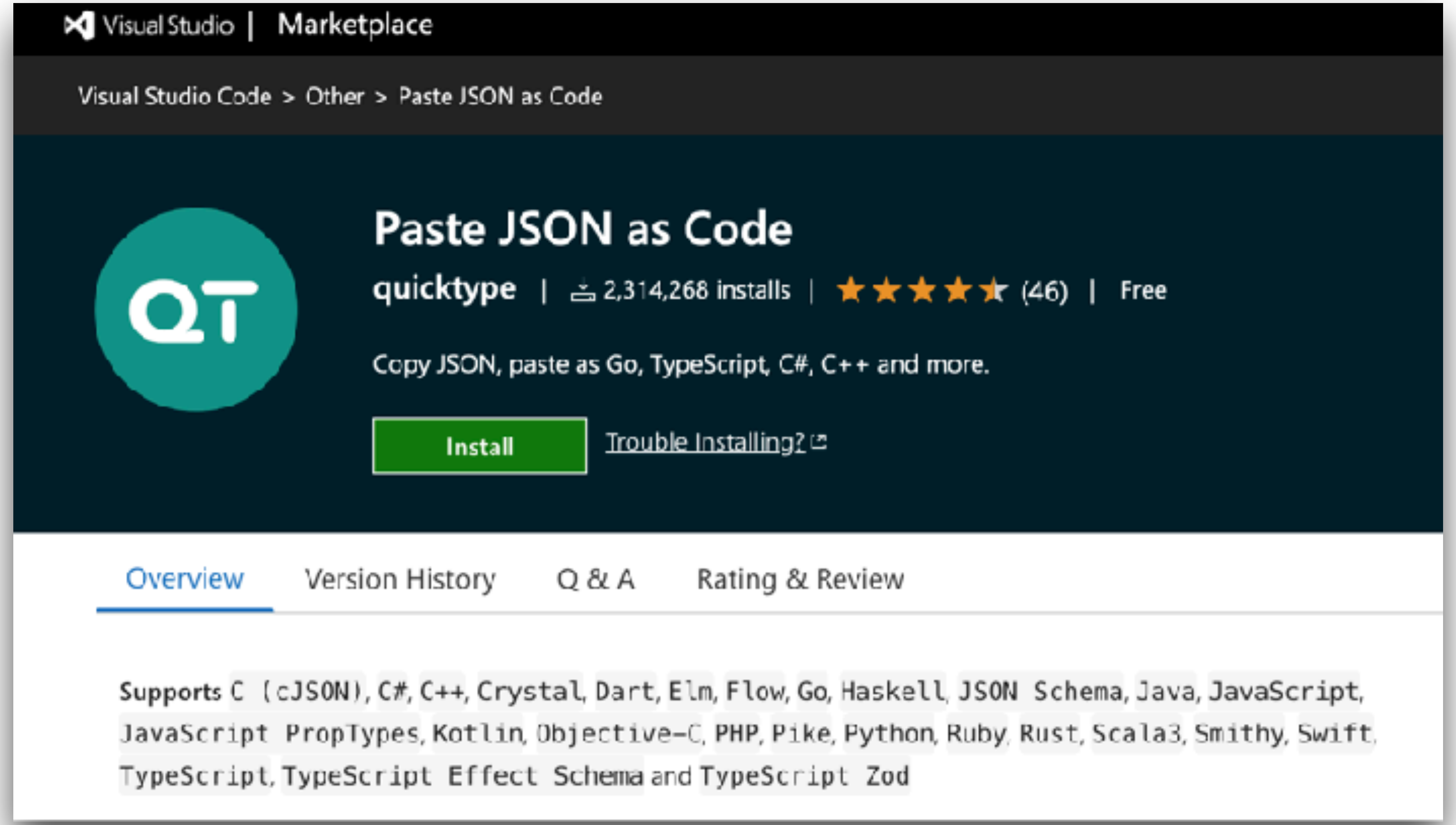
## IntelliJ IDEA




https://plugins.jetbrains.com/plugin/8634-robopojogenerator

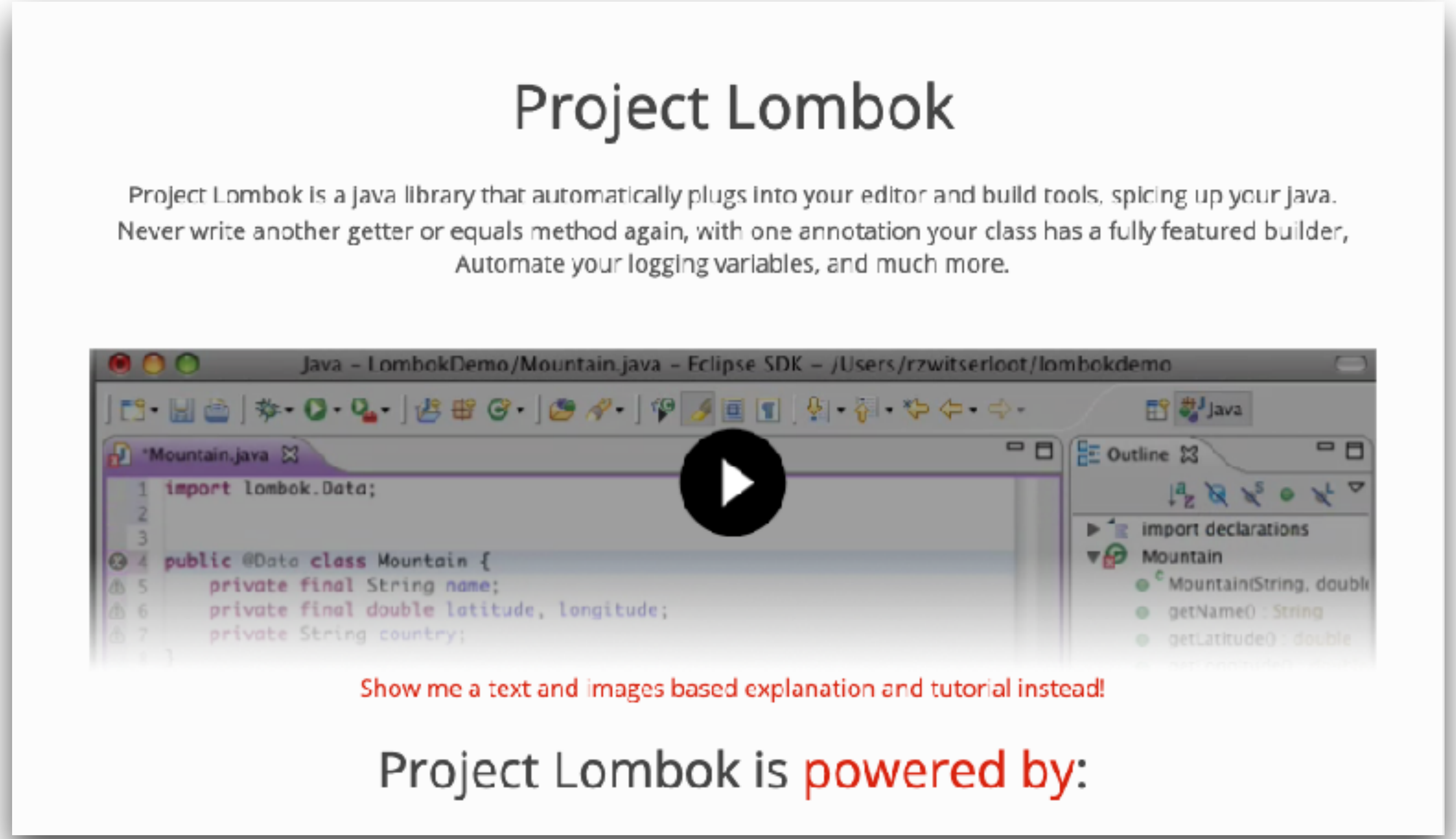
# Convert JSON to POJO

## VS Code



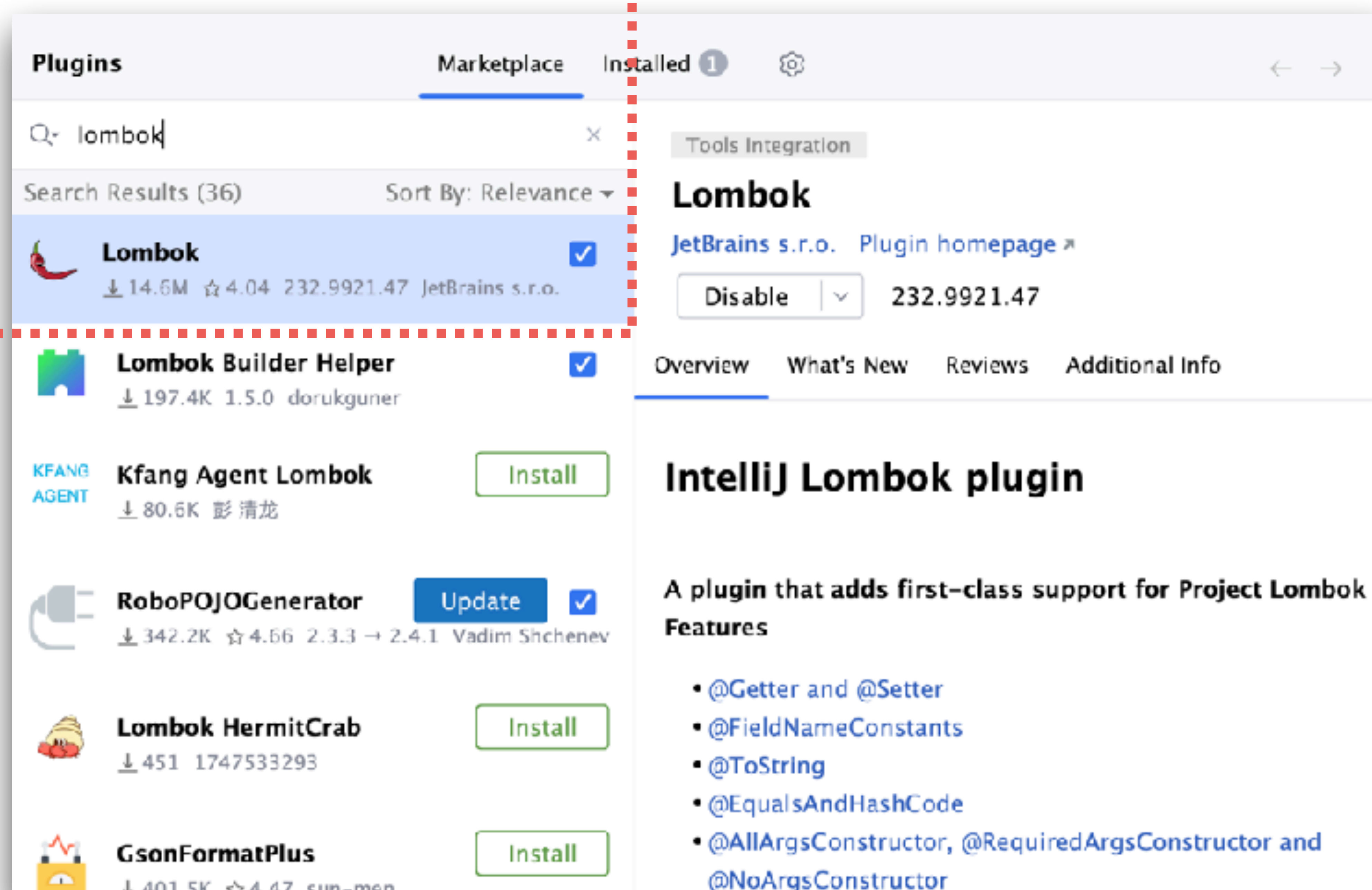

https://marketplace.visualstudio.com/items?itemName=quicktype.quicktype

# Using Lombok




https://projectlombok.org/

# IntelliJ IDEA plug-in

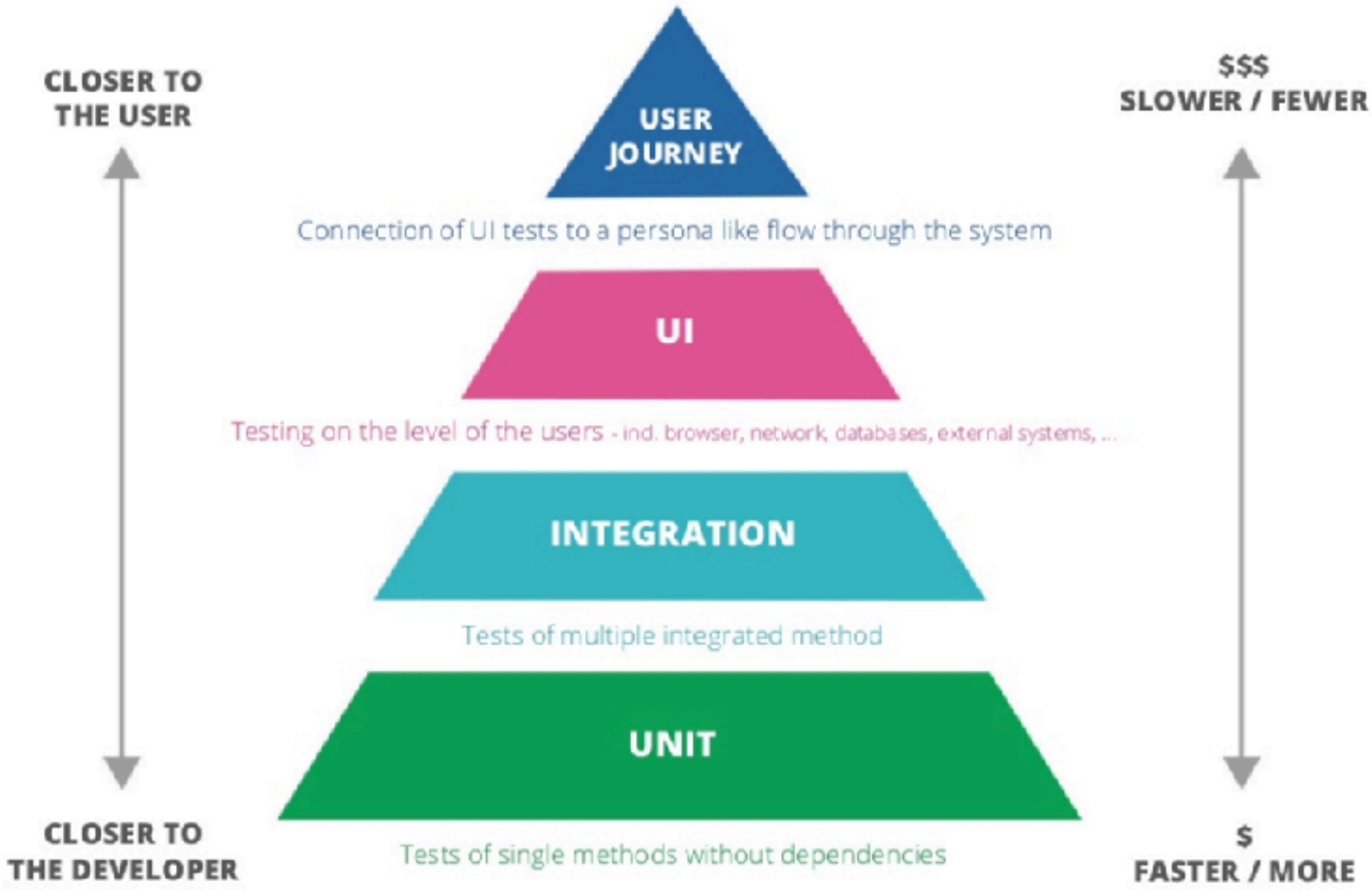



https://plugins.jetbrains.com/plugin/6317-lombok

# Use Lombok

```java
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class UserRequest {
    private String email;
    private String password;
}
```

# Testing with Spring Boot



CLOSER TO THE USER
$$$ SLOWER / FEWER
USER JOURNEY
Connection of UI tests to a persona like flow through the system
UI
Testing on the level of the users - incl. browser, network, databases, external systems, ...
INTEGRATION
Tests of multiple integrated method
UNIT
Tests of single methods without dependencies
CLOSER TO THE DEVELOPER
$ FASTER / MORE

# Spring Boot Testing

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
</dependency>
```

6

https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing

# Testing in Spring Boot ?

**@SpringBootTest**
@WebMVCTest
@JsonTest
@DataJpaTest
@RestClientTest

https://docs.spring.io/spring-boot/reference/testing/spring-boot-applications.html

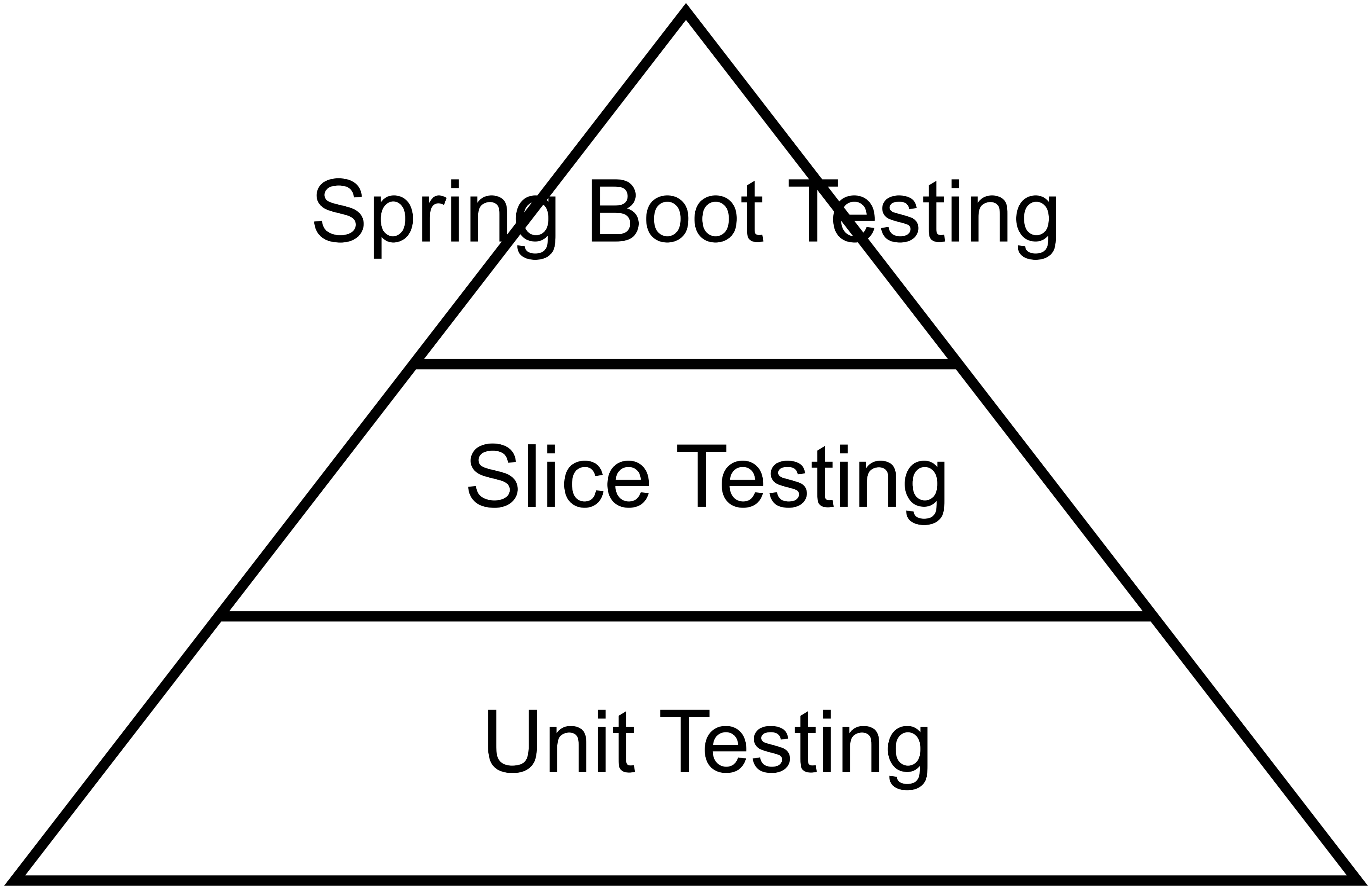
# Testing in Spring Boot

@SpringBootTest

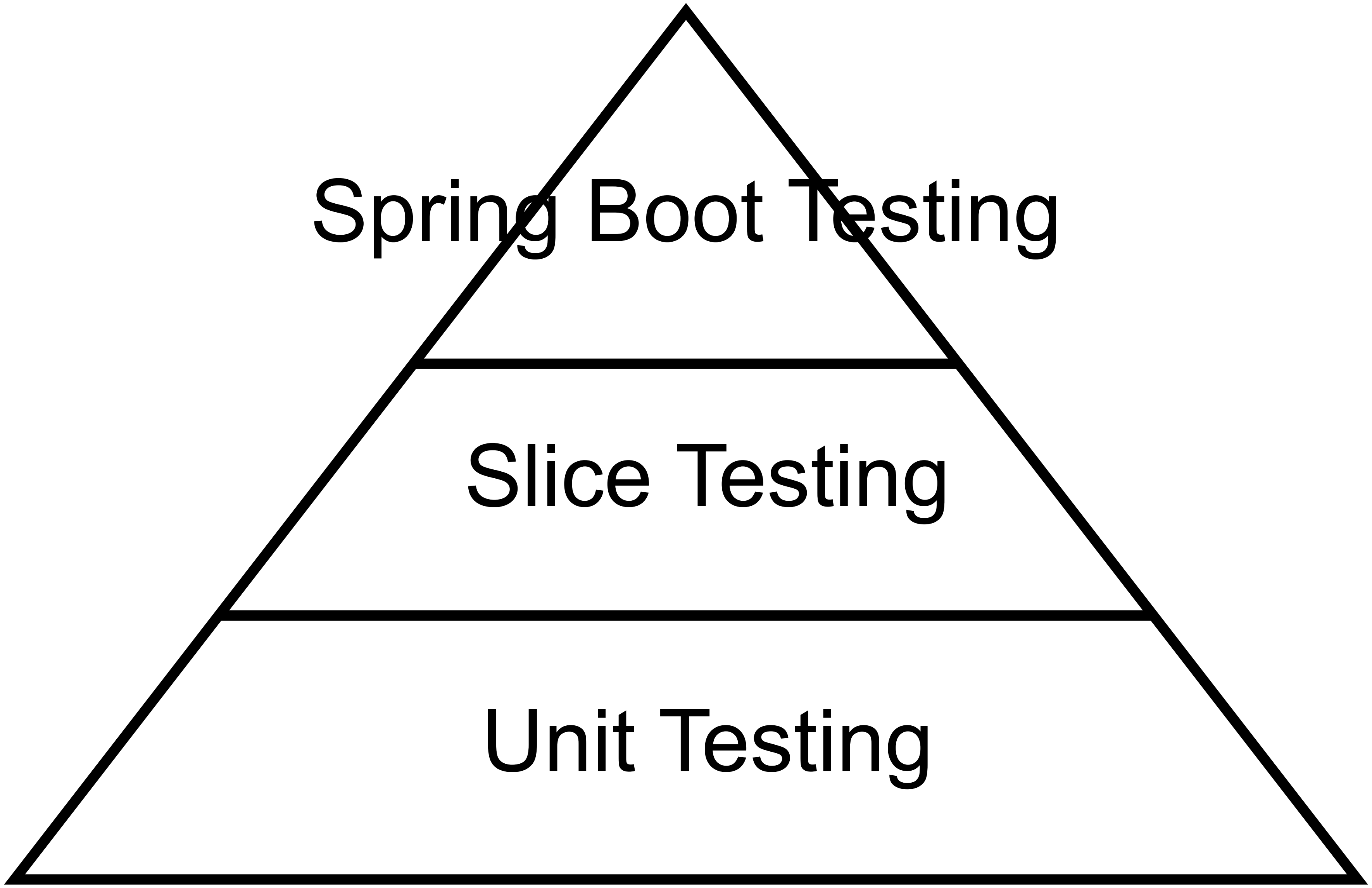
@WebMVCTest
@JsonTest
@DataJpaTest
@RestClientTest

**Slice testing**

# Testing in Spring Boot



# Testing in Spring Boot

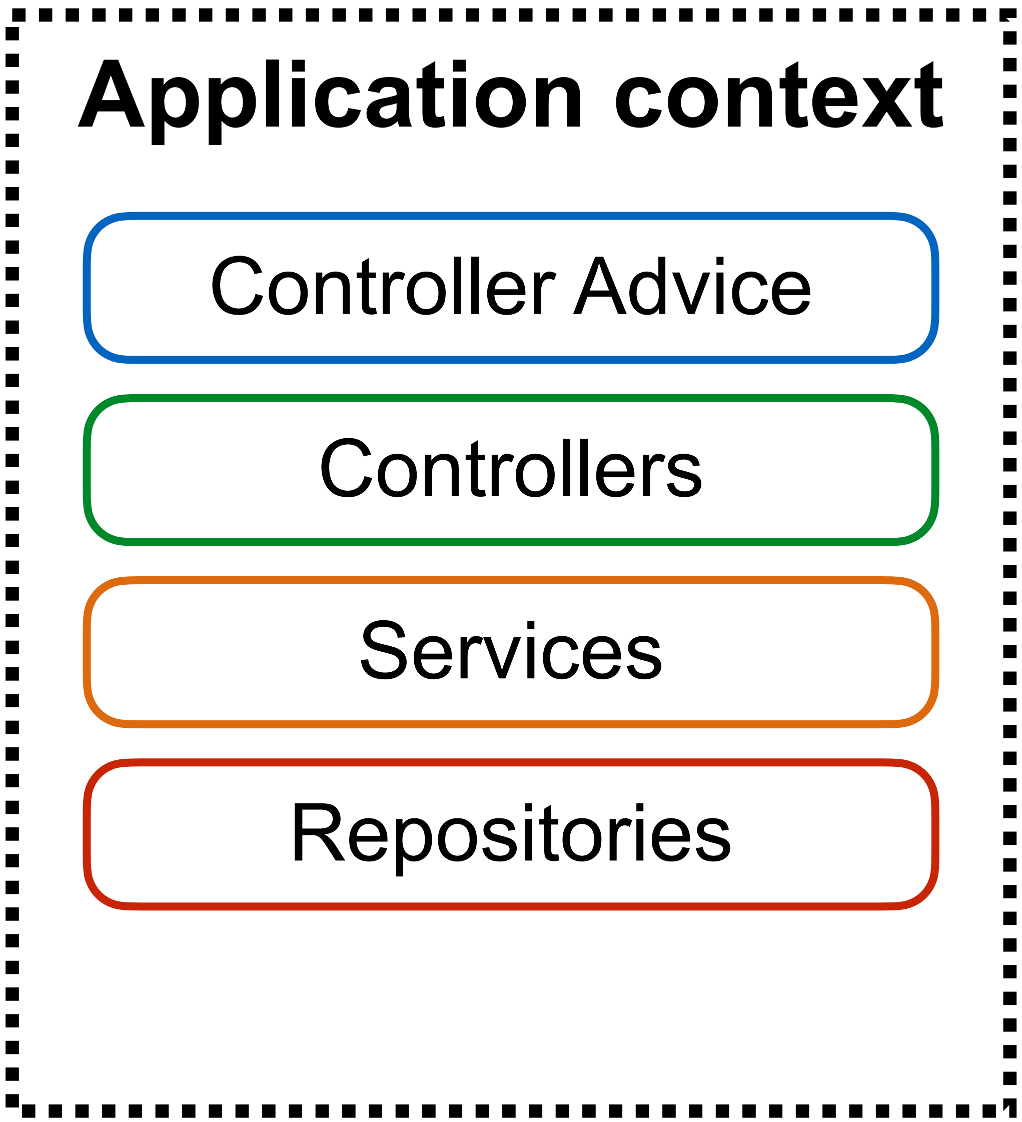



mockito

# Controller testing

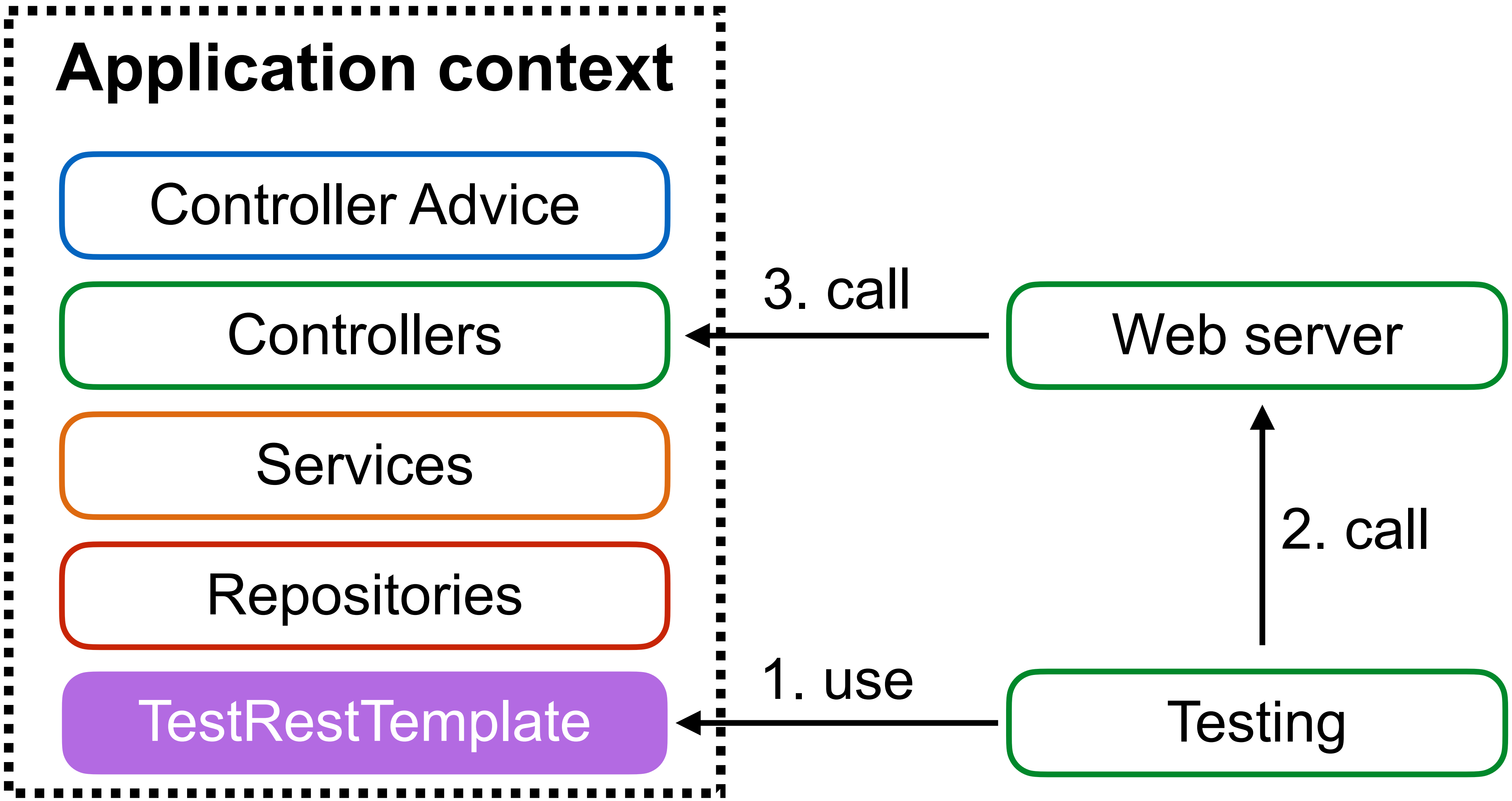
1. Spring Boot Testing
2. Slice Testing with MockMvc
3. Unit Testing

# 1. SpringBootTest




https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/#features.testing

# 1. SpringBootTest



# SpringBootTest #1

```java
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment
        = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @Test
    public void sayHi() {
        // Action :: Call controller
        Hello actualResult
                = testRestTemplate.getForObject("/hello/somkiat",
                                                  Hello.class);

        // Assertion :: Check result with expected result
        assertEquals("Hello, somkiat", actualResult.getMessage());

    }
}
```

# SpringBootTest #2

```
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment
        = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @Test
    public void sayHi() {
        // Action :: Call controller
        Hello actualResult
                = testRestTemplate.getForObject("/hello/somkiat",
                                                Hello.class);

        // Assertion :: Check result with expected result
        assertEquals("Hello, somkiat", actualResult.getMessage());

    }
}
```

# SpringBootTest #3

```
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment
        = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @Test
    public void sayHi() {
        // Action :: Call controller
        Hello actualResult
                = testRestTemplate.getForObject("/hello/somkiat",
                                                Hello.class);

        // Assertion :: Check result with expected result
        assertEquals("Hello, somkiat", actualResult.getMessage());

    }
}
```

# Error handling

# Working with Error Responses

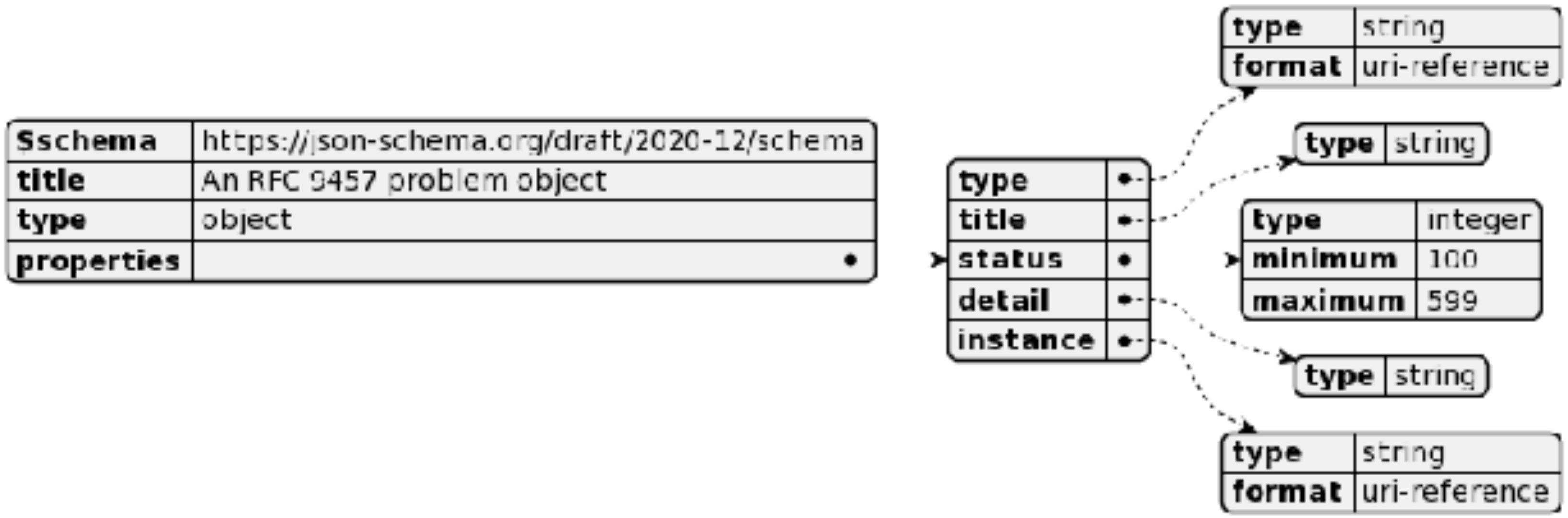
ErrorResponse

ProblemDetail

Customization

https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html

# ProblemDetail




```json
{
  "type":"https://problems-registry.smartbear.com/missing-body-property",
  "status":400,
  "title":"Missing body property",
  "detail":"The request is missing an expected body property.",
  "code":"400-09",
  "instance":"/logs/regisrations/d24b2953-ce05-488e-bf31-67de50d3d085",
  "errors":[
    {
        "detail":"The body property {name} is required",
        "pointer":"/name"
    }
  ]
}
```

https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/http/ProblemDetail.html

# ProblemDetail

Enable ProblemDetail for error by default

**application.properties**

```
spring.mvc.problemdetails.enabled=true
```

# Example in ControllerAdvice

```
@RestControllerAdvice
class HelloControllerAdvice {

    @ExceptionHandler(InvalidInputException.class)
    public ProblemDetail handleInvalidInputException(
            InvalidInputException e, WebRequest request) {

        ProblemDetail problemDetail
                = ProblemDetail
                .forStatusAndDetail(HttpStatus.BAD_REQUEST, e.getMessage());
        problemDetail.setInstance(URI.create("demo-instance"));
        problemDetail.setTitle("demo");

        return problemDetail;
    }
}
```

# Error handling

```java
@Service
public class UserService {

    private AccountRepository accountRepository;

    @Autowired
    public UserService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    public Account getAccount(int id) {
        Optional<Account> account = accountRepository.findById(id);
        if(account.isPresent()) {
            return account.get();
        }
        throw new MyAccountNotFoundException(
                String.format("Account id=[%d] not found", id));
    }
```

# MyAccountNotFoundException

```
public class MyAccountNotFoundException
            extends RuntimeException {

    public MyAccountNotFoundException(String message) {
        super(message);
    }

}
```

# Response Status

404 = Not Found

| Status | Description |
| --- | --- |
| 400 | Request body doesn’t meet API spec |
| 401 | Authentication/Authorization fail |
| 403 | User can’t perform the operation |
| 404 | Resource does not exist |
| 405 | Unsupported operation |
| 500 | Error on server |

# Handling error in Spring Boot

```java
@RestControllerAdvice
public class AccountControllerHandler {
```

1

```java
    @ExceptionHandler(MyAccountNotFoundException.class)
    public ResponseEntity<ExceptionResponse> accountNotFound(
            MyAccountNotFoundException exception) {

        ExceptionResponse response =
                new ExceptionResponse(exception.getMessage(),
                                          "More detail");

        return new ResponseEntity<ExceptionResponse>(response,
                                                        HttpStatus.NOT_FOUND);

    }

}
```

# Handling error in Spring Boot

```java
@RestControllerAdvice
public class AccountControllerHandler {

    @ExceptionHandler(MyAccountNotFoundException.class)
    public ResponseEntity<ExceptionResponse> accountNotFound(
            MyAccountNotFoundException exception) {

        ExceptionResponse response =
                new ExceptionResponse(exception.getMessage(),
                                      "More detail");

        return new ResponseEntity<ExceptionResponse>(response,
                                                     HttpStatus.NOT_FOUND);

    }
}
```

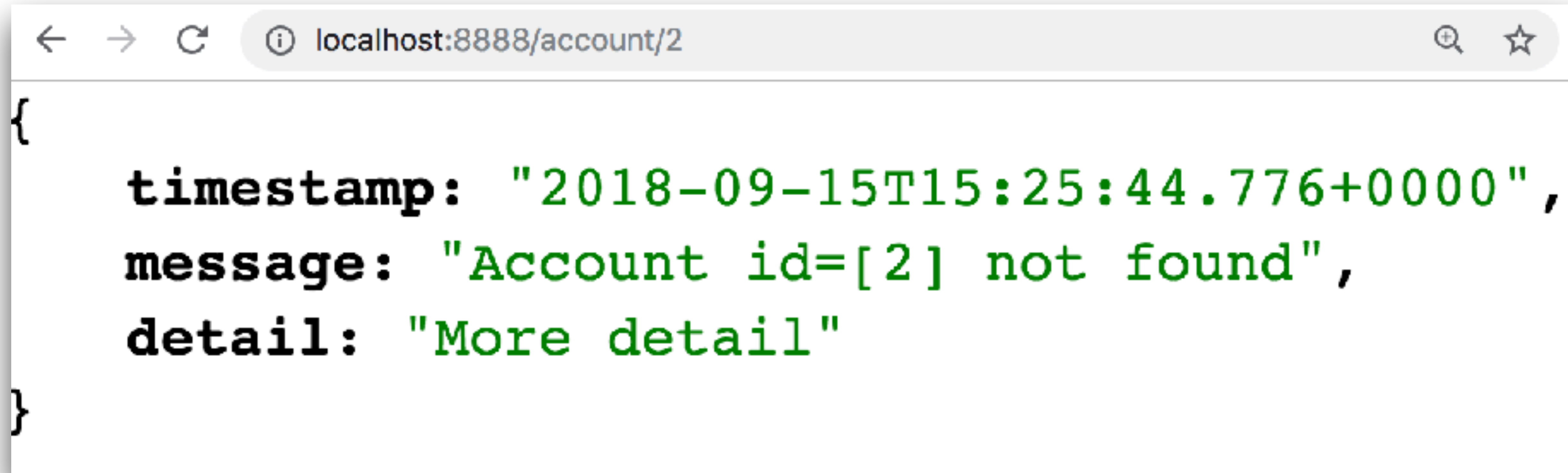
2

# ExceptionReponse

## Response format of error

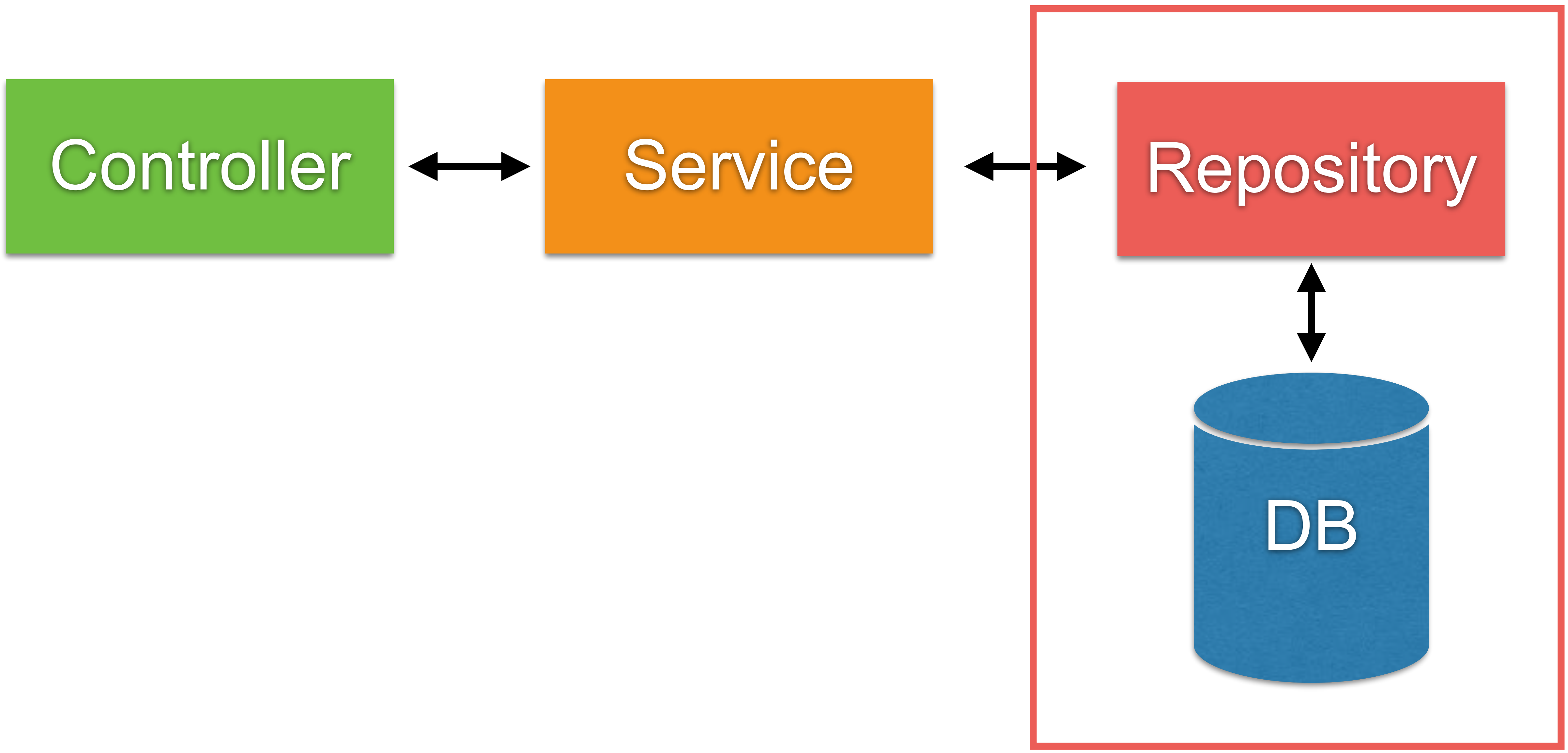
```
public class ExceptionResponse{

    private Date timestamp = new Date();
    private String message;
    private String detail;

    public ExceptionResponse(String message, String detail) {
        this.message = message;
        this.detail = detail;
    }
```

# Result of API



# Working with Database

# Repository Layer in Spring



# Basic of JDBC

```
// 1. Load jdbc driver
Class.forName("postgresql");

// 2. Create connection
Connection connection = DriverManager.getConnection("", "", "");

// 3. Prepared Statement
String sql = "SELECT * FROM TABLE WHERE name=?";
PreparedStatement pStmt = connection.prepareStatement(sql);

// 4. Query
ResultSet resultSet = pStmt.executeQuery();
while(resultSet.next()) {

}

// 5. Release resource
if(resultSet != null) {
    resultSet.close();
    resultSet = null;
}
```

# Framework !!

```java
// 1. Load jdbc driver
Class.forName("postgresql");

// 2. Create connection
Connection connection = DriverManager.getConnection("", "", "");
```

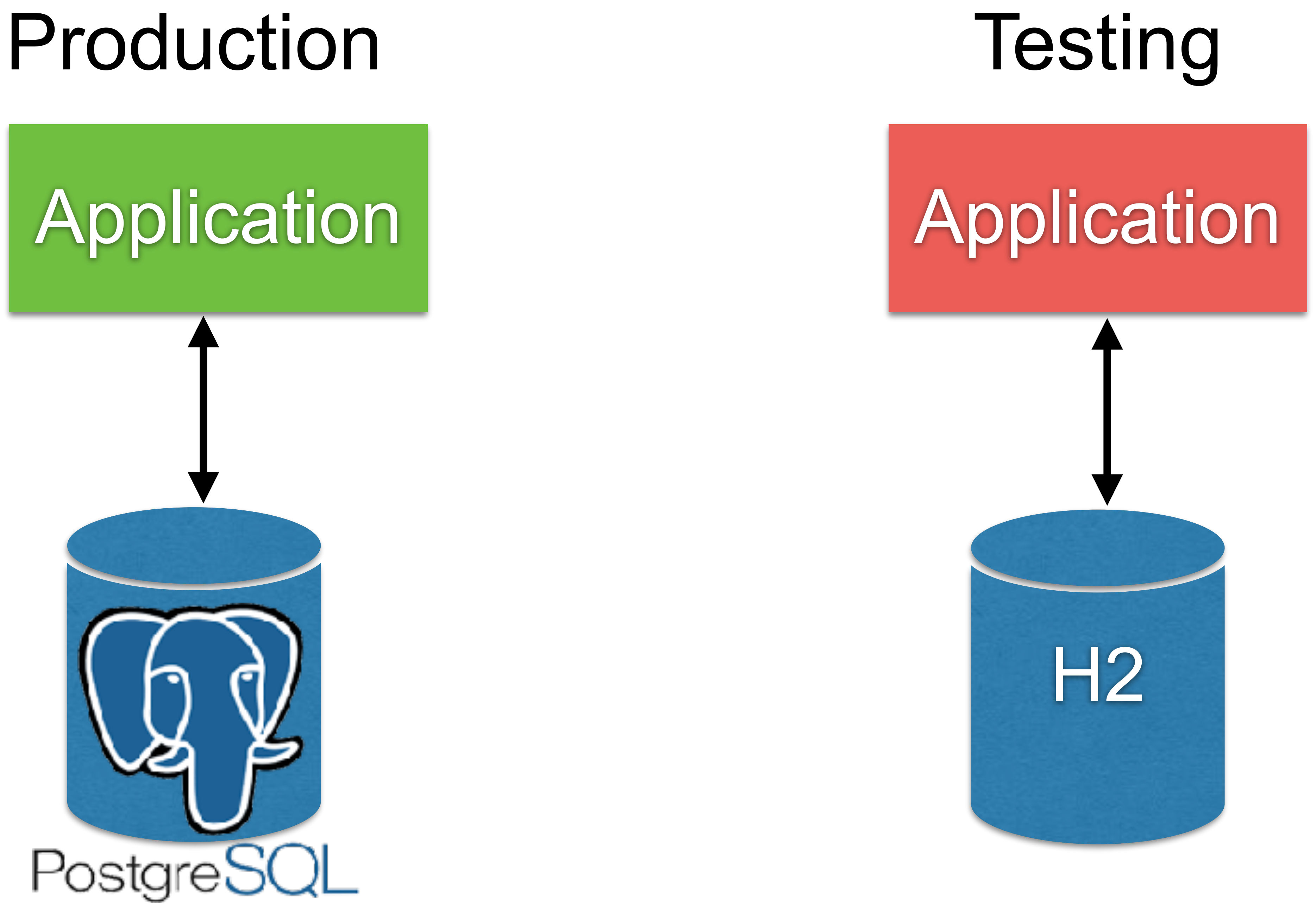
Manage by Framework

```java
// 3. Prepared Statement
String sql = "SELECT * FROM TABLE WHERE name=?";
PreparedStatement pStmt = connection.prepareStatement(sql);

// 4. Query
ResultSet resultSet = pStmt.executeQuery();
while(resultSet.next()) {

}
```

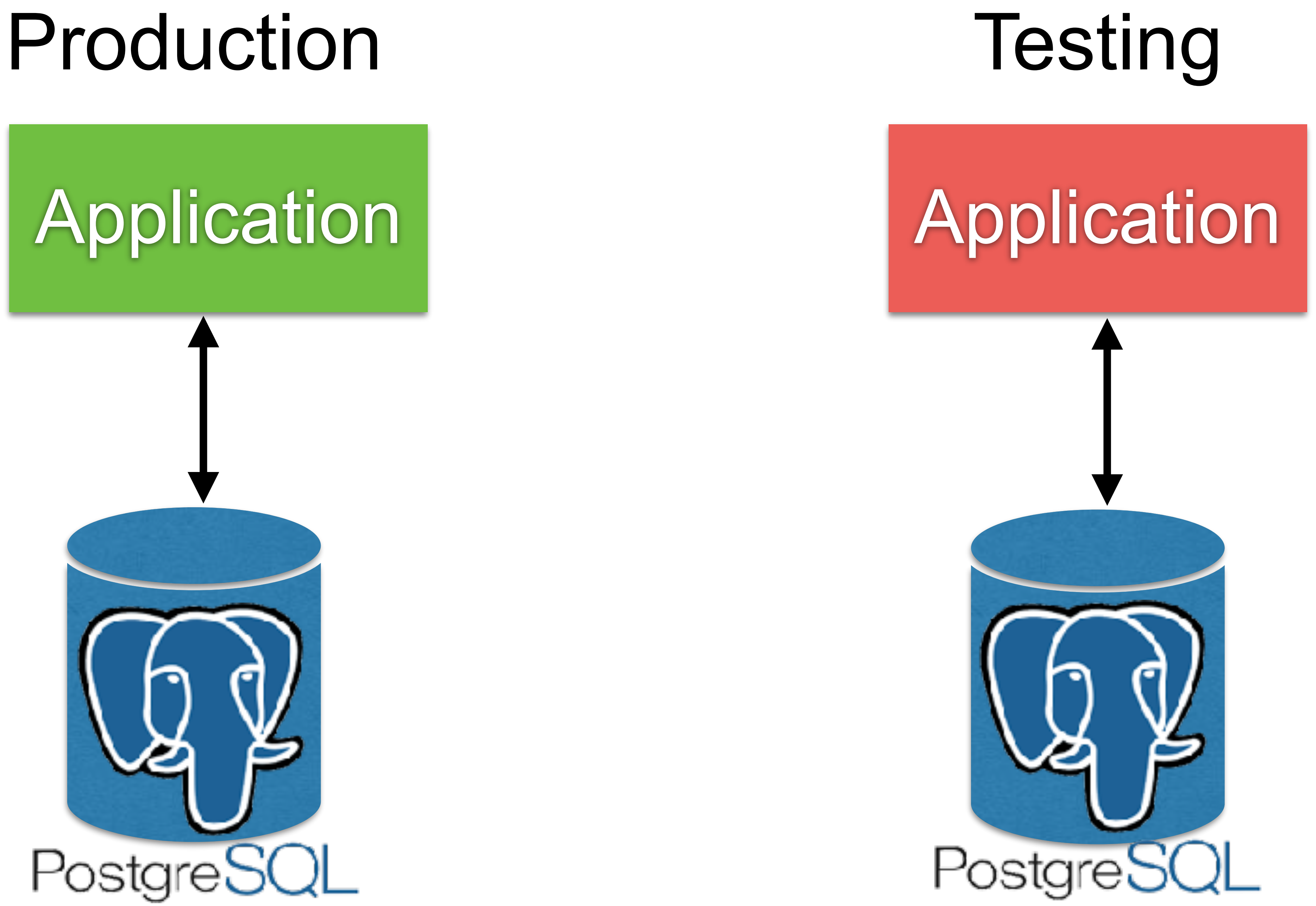
```java
// 5. Release resource
if(resultSet != null) {
    resultSet.close();
    resultSet = null;
}
```

Manage by Framework

# Working with Database ?



# Working with Database ?



# Working with repository

We're using Spring Data

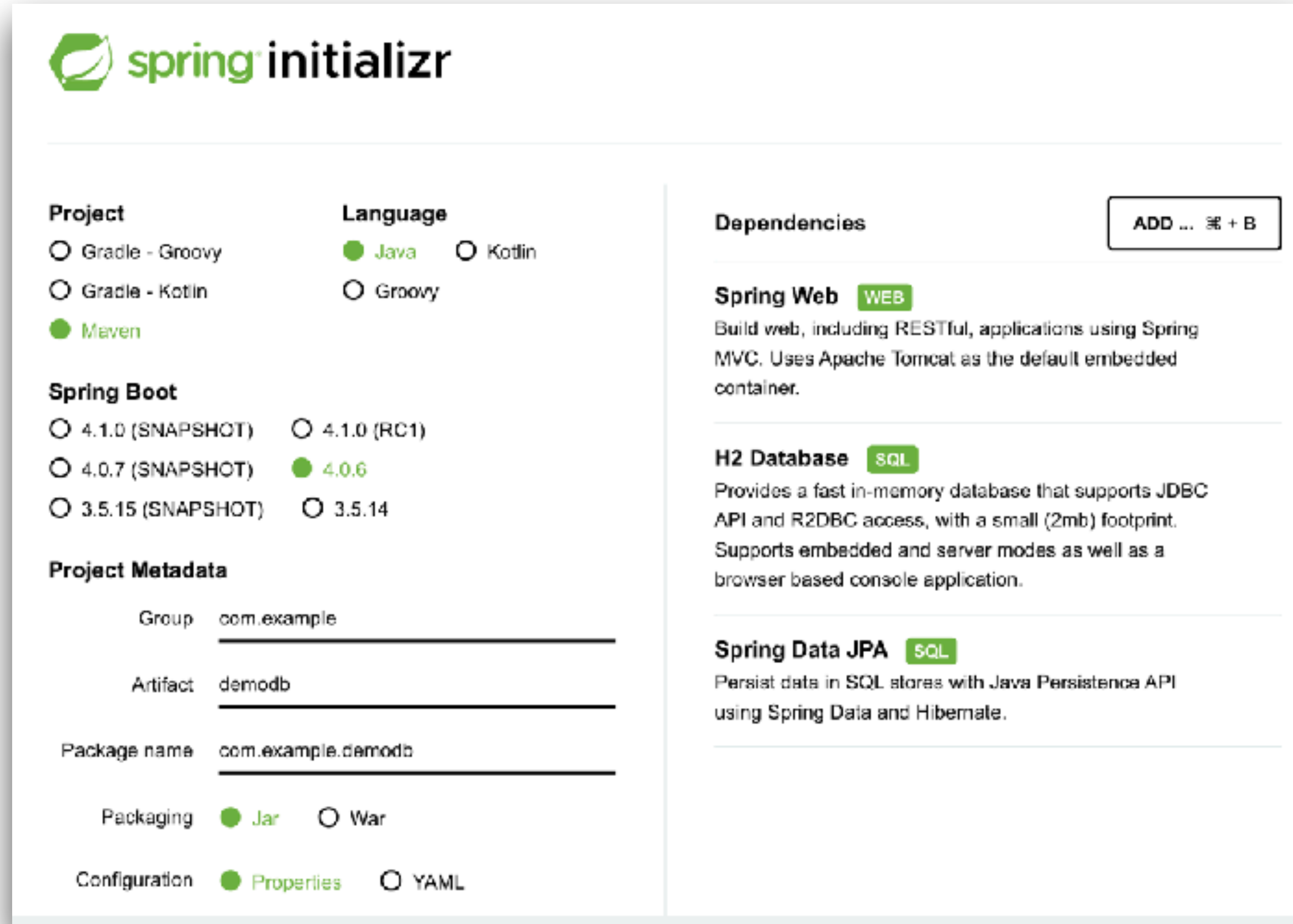
+

https://spring.io/projects/spring-data

# Working with Database

Manage transactions with @Transactional
Manage multiple datasources

# Create Project



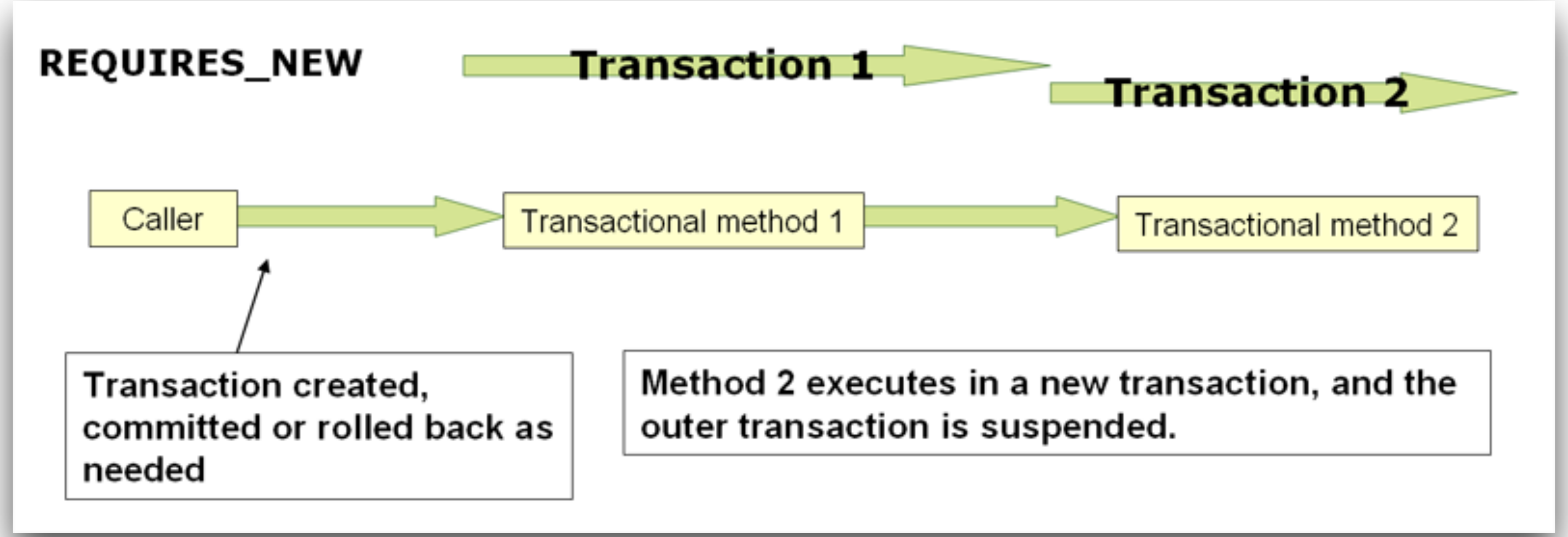

https://start.spring.io/

# Transaction Propagation (1)

## Define how a transaction boundary with @Transaction

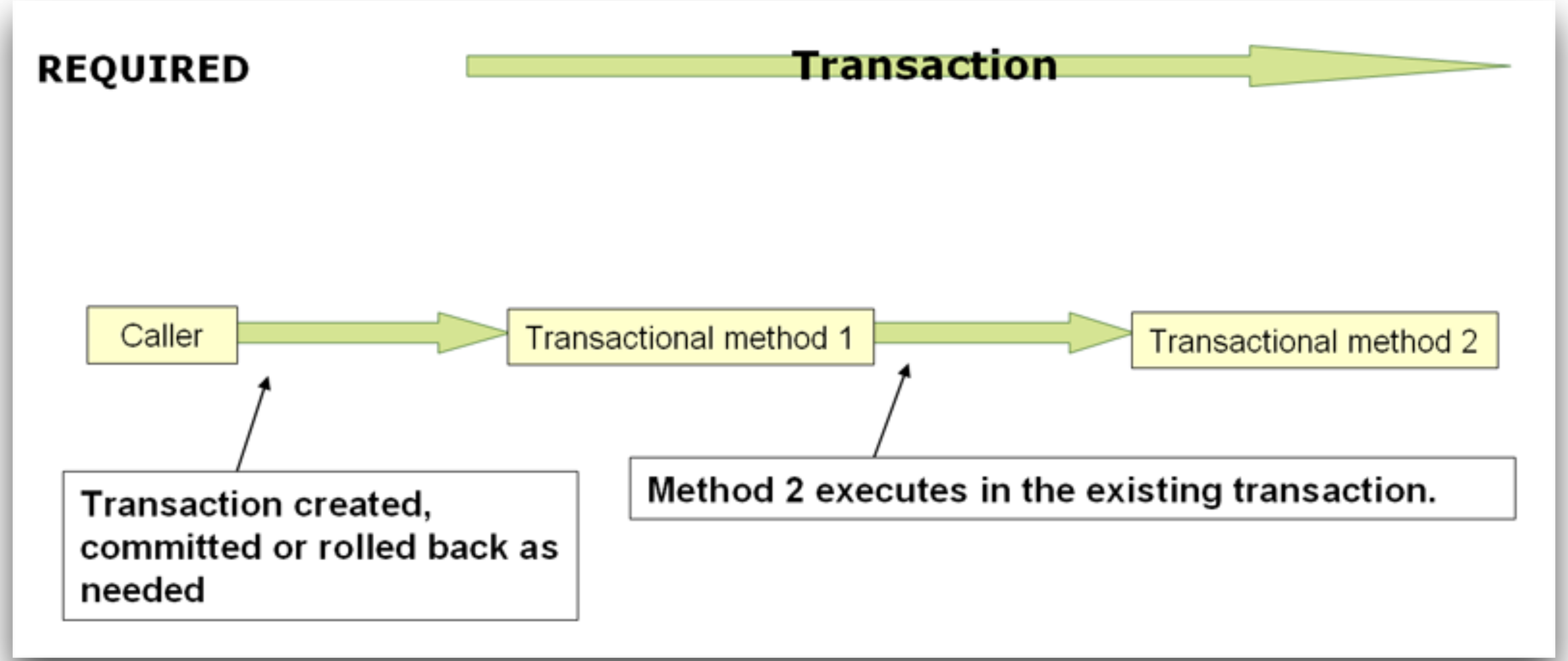
| Propagation | Existing tx | No existing tx |
|---|---|---|
| REQUIRED | Joins the transaction | Start a new transaction |
| REQUIRES_NEW | Suspend current, start new | Start a new transaction |
| SUPPORTS | Joins the transaction | Run with non-transaction |
| MANDATORY | Joins the transaction | Throw exception |
| NOT_SUOORTED | Suspend current, run with non-transaction | Run with non-transaction |
| NEVER | Throw exception | Run with non-transaction |

# Transaction Propagation (2)




https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/tx-propagation.html

# Transaction Propagation (3)




https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/tx-propagation.html

# Isolation Levels

Performance (concurrency) vs Consistency (data integrity)

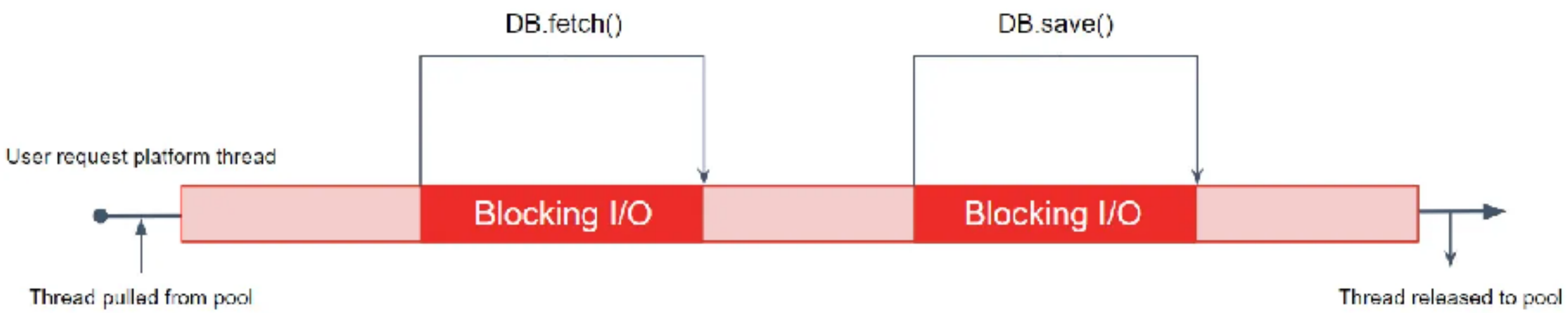
| Isolation Level | Dirty read | Non-repeatable read | Phantom read | Performance |
|---|---|---|---|---|
| READ_UNCOMMITTED | Allowed | Allowed | Allowed | Fastest |
| READ_COMMITED | Prevented | Allowed | Allowed | Fast |
| REPEATABLE_READ | Prevented | Prevented | Allowed* | Moderate |
| SERIALIZABLE | Prevented | Prevented | Prevented | Slowest |

# Spring AOP !!

https://docs.spring.io/spring-framework/reference/core/aop.html

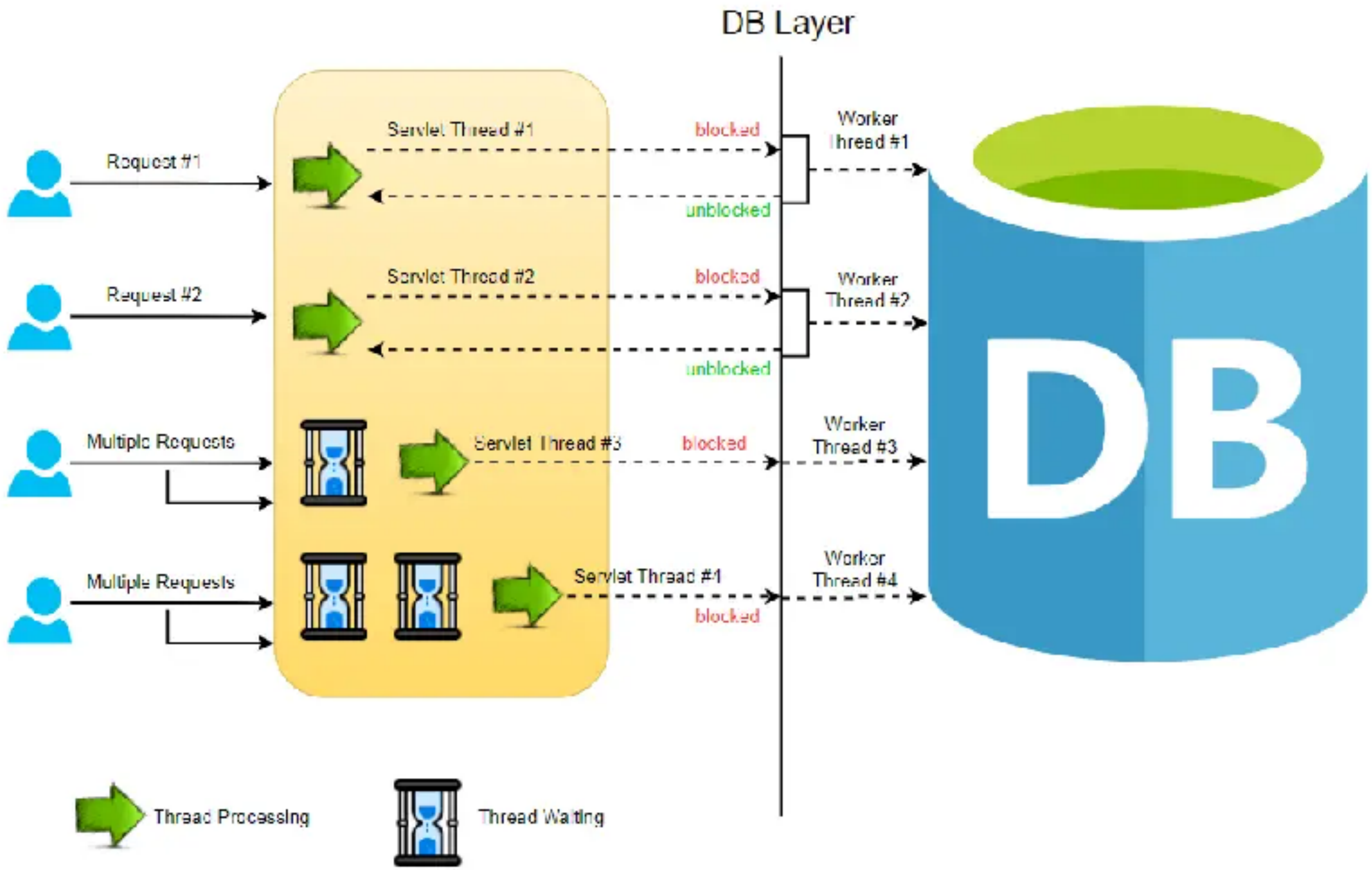
# Non-Blocking I/O

# Blocking I/O



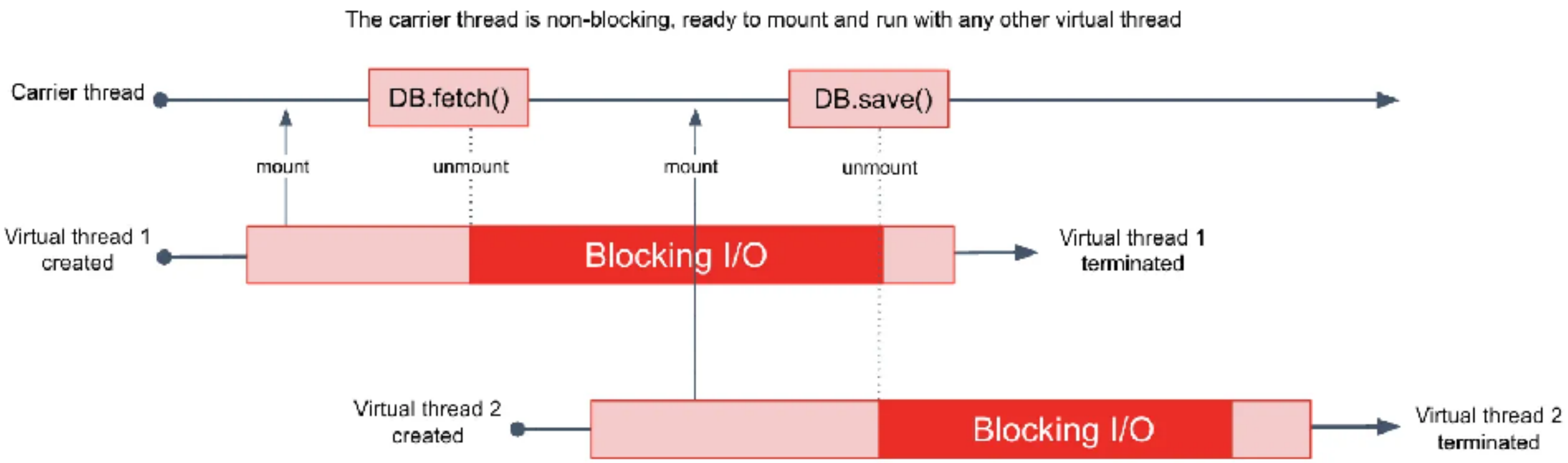

https://developers.ascendcorp.com/why-are-my-java-virtual-threads-slower-than-the-platform-threads-74612a1587f3

# Blocking I/O



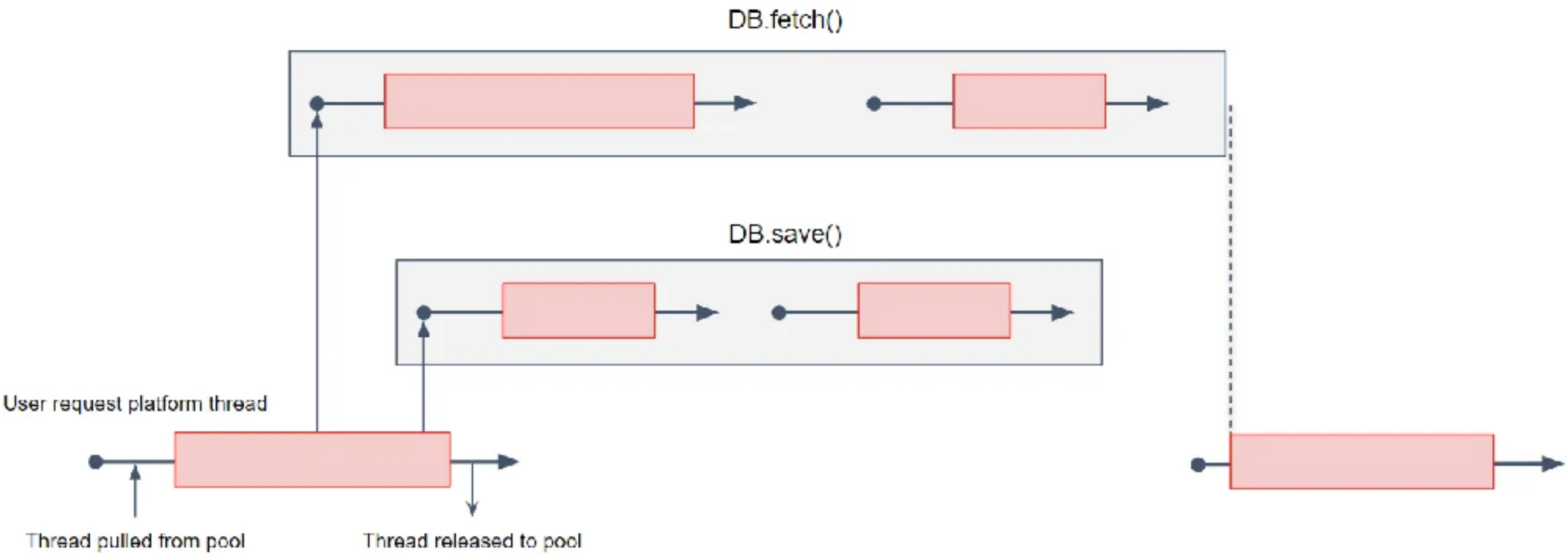

https://reflectoring.io/getting-started-with-spring-webflux/

# Blocking I/O with Virtual Thread

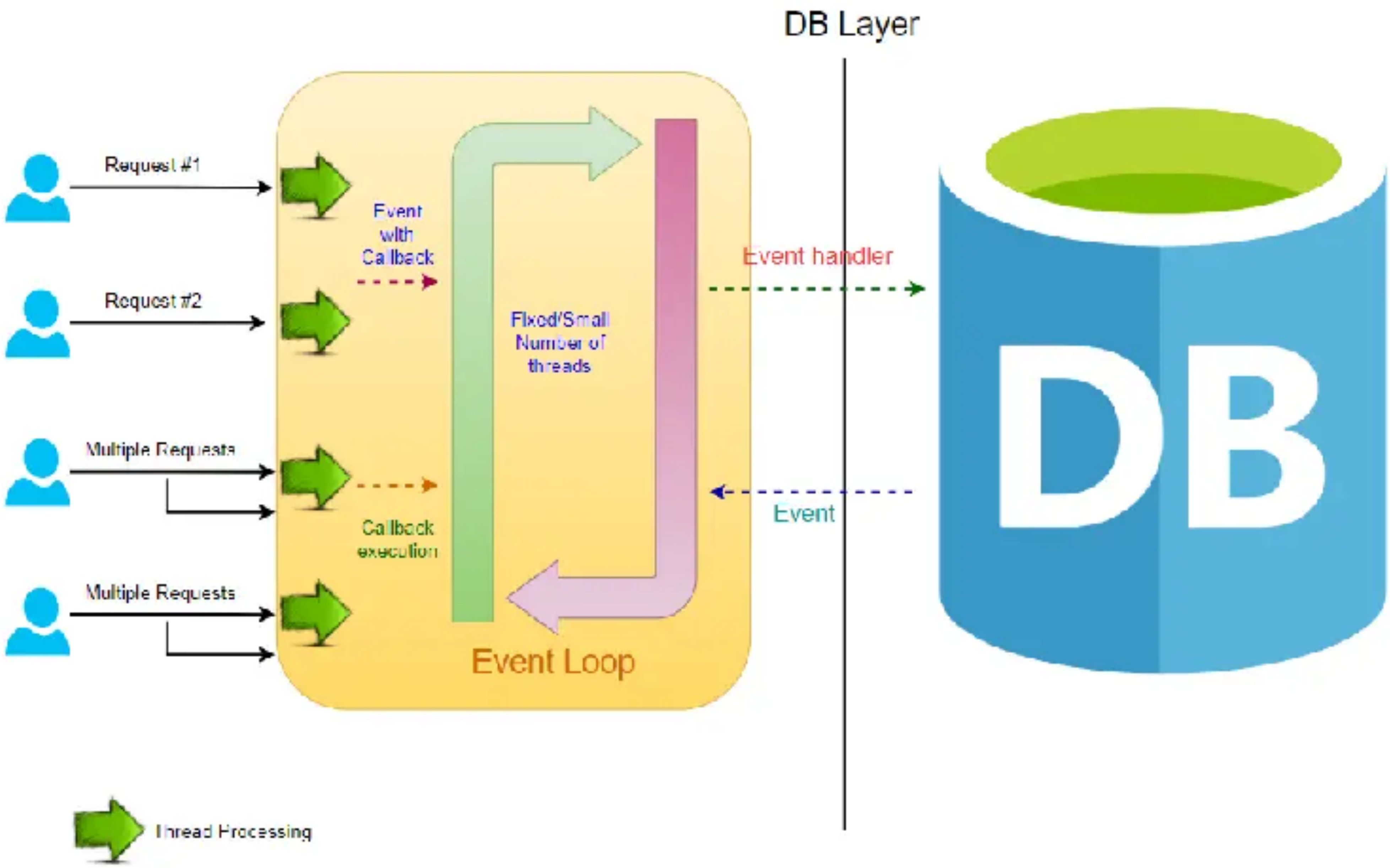


# Non-Blocking I/O

## Reactive framework



# Non-Blocking I/O




https://reflectoring.io/getting-started-with-spring-webflux/

# CompletableFuture

https://mobisoftinfotech.com/resources/blog/java-programming/java-async-programming-guide

# Reactive in Java

spring WebFlux

MICRONAUT®

VERT.X

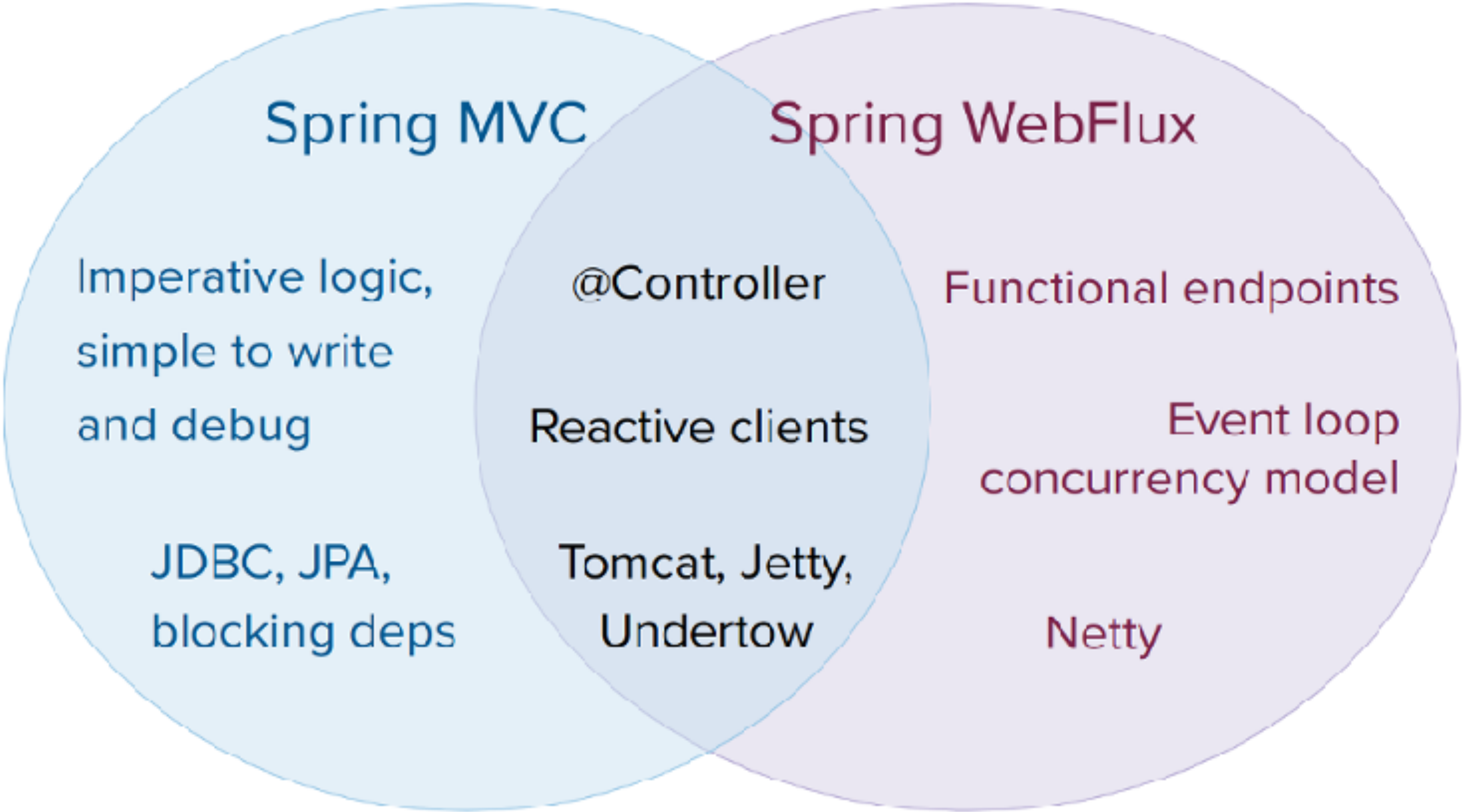
play

RxJava

QUARKUS

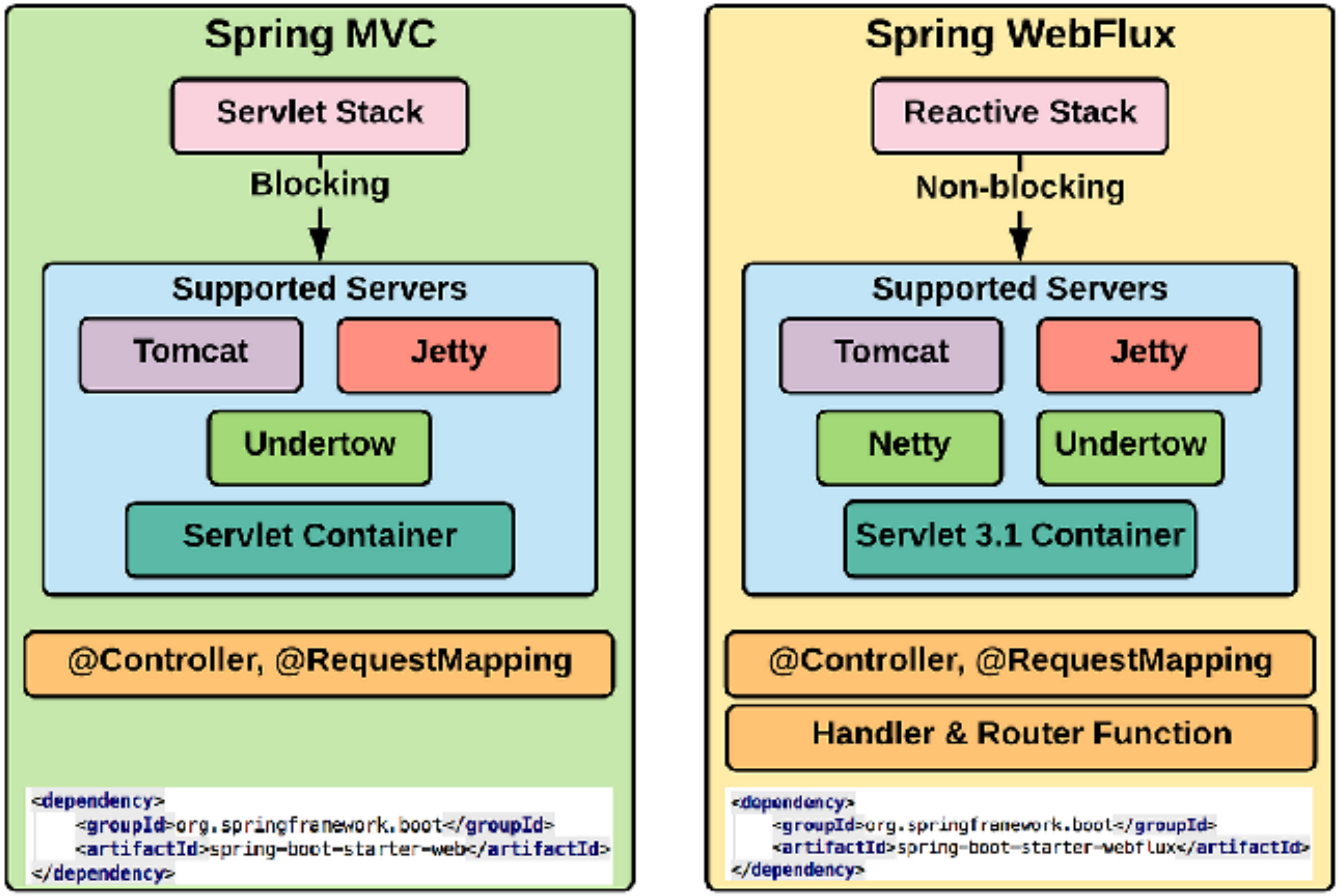
# Spring Webflux

# Spring WebFlux



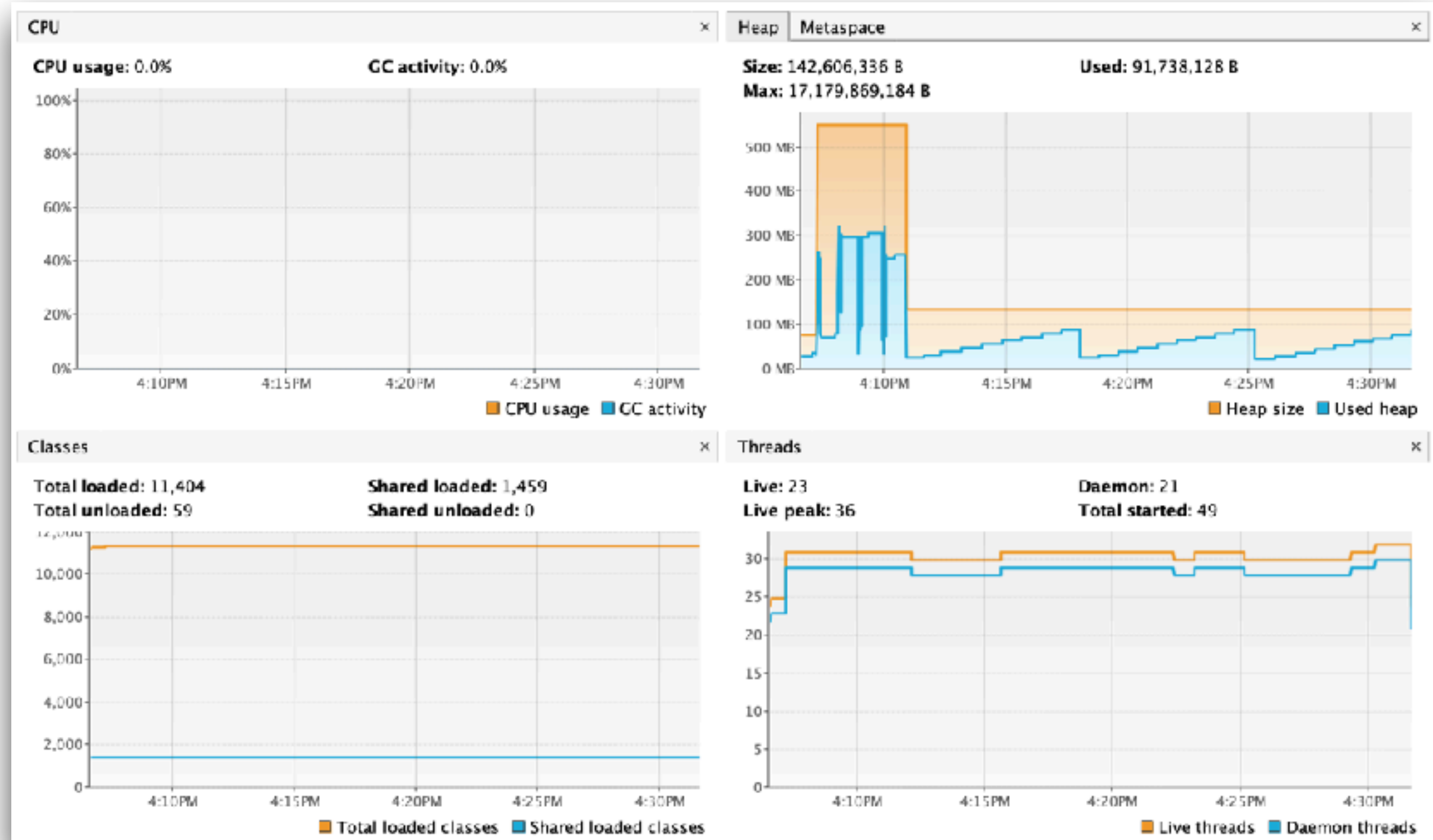

https://docs.spring.io/spring-framework/reference/web/webflux.html

# Spring MVC vs WebFlux



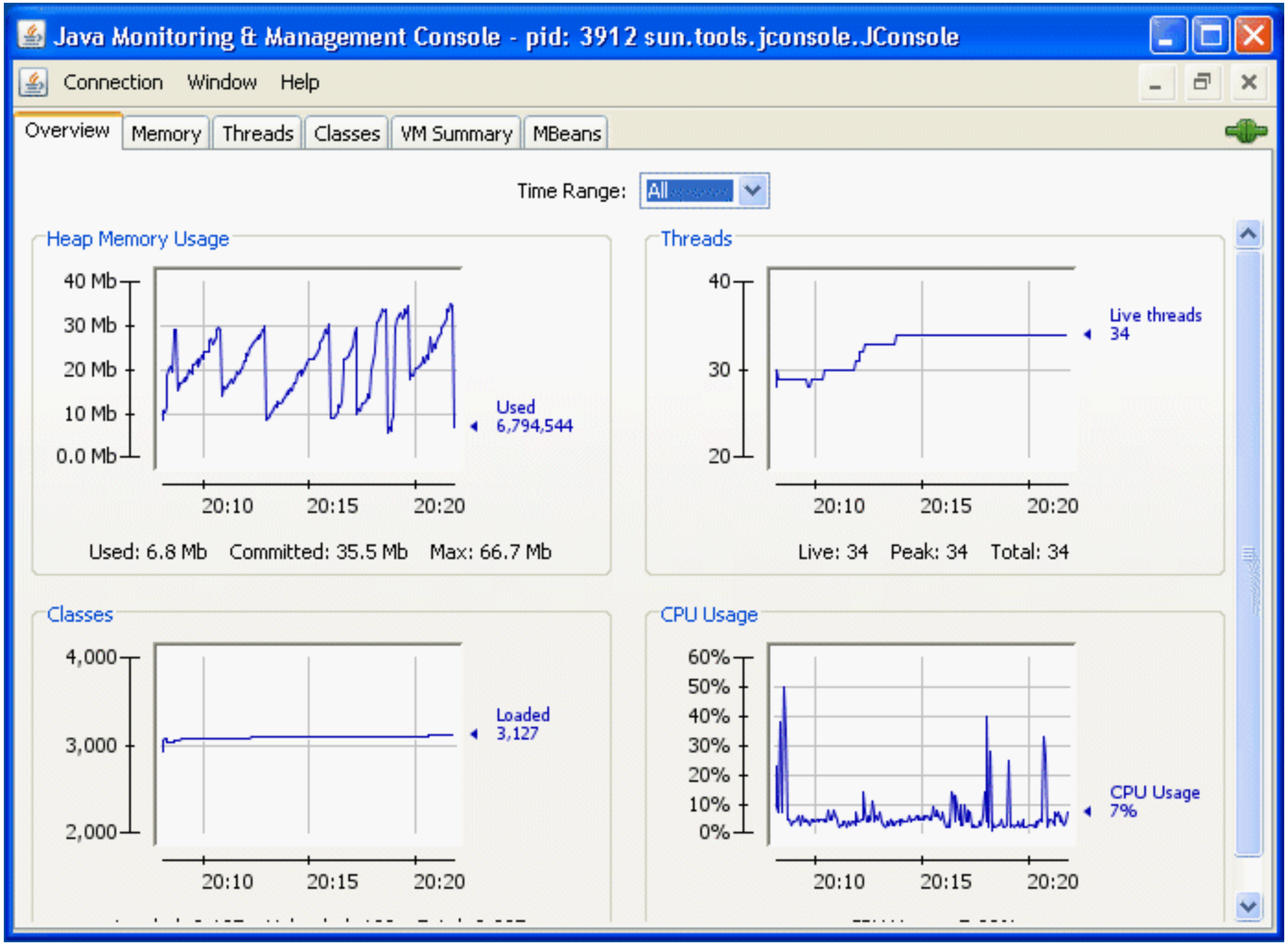

https://docs.spring.io/spring-framework/reference/web/webflux.html

# Monitoring with VisualVM




https://visualvm.github.io/

# Monitoring with JConsole




https://docs.oracle.com/en/java/javase/26/management/using-jconsole.html

# Learning …

# Q/A